รายงานโครงงานวิศวกรรมศาสตร์ (Final Project Report)

ThaiLaw Assistant: ระบบผู้ช่วยตอบคำถามกฎหมายแรงงานไทยด้วย Hybrid RAG (Dense + Graph) บน LINE

โดย

นายคีตศิลป์ คงสี รหัสนักศึกษา 6610110425

นายอลงกรณ์ จงยิ่งยศ รหัสนักศึกษา 6610110693

อาจารย์ผู้สอน: *อ.กฤษณ์วรา รัตนโอภาส*

รายวิชา: *241-351 Artificial Intelligences for Social Media Module*

โครงงานระดับปริญญาตรี หลักสูตรวิศวกรรมศาสตรบัณฑิต

ภาควิชาวิศวกรรมคอมพิวเตอร์

ปีการศึกษา: 2569

บทคัดย่อ (Abstract)

ภาษาไทย

โครงงานนี้พัฒนา ThaiLaw Assistant ระบบตอบคำถามกฎหมายแรงงานไทยบนแพลตฟอร์ม LINE โดยครอบคลุมบทบัญญัติ 187 มาตราของพระราชบัญญัติคุ้มครองแรงงาน พ.ศ. 2541 ระบบใช้แนวทาง Hybrid Retrieval-Augmented Generation (Hybrid RAG) ซึ่งผสาน Dense Retrieval ด้วย bge-m3 และ ChromaDB, การค้นคืนด้วย BM25 สำหรับภาษาไทย และ Graph RAG ร่วมกับ Query Router, Weighted Reciprocal Rank Fusion, Graph-seeded Expansion และ Provenance-aware Rerank เพื่อเพิ่มความถูกต้องและความน่าเชื่อถือของบริบทที่ส่งให้แบบจำลองภาษาขนาดใหญ่ นอกจากนี้ยังมีระบบ Fallback และการจัดการข้อผิดพลาดเพื่อรองรับการใช้งานจริง ระบบได้รับการประเมินด้วยชุดทดสอบจำนวน 50 คำถาม ครอบคลุม 7 ประเภท พร้อมคำตอบอ้างอิง

ผลการทดลองพบว่า Hybrid RAG มีประสิทธิภาพเหนือกว่า Dense RAG อย่างมีนัยสำคัญทางสถิติ โดยเมื่อประเมินด้วย RAGEngine จริง ค่า Recall@5 เพิ่มขึ้นจาก 0.625 เป็น 0.823 ($p = 0.001$) และค่า Hit@1 เพิ่มขึ้นจาก 0.565 เป็น 0.696 ($p = 0.014$) และค่า Citation Recall เพิ่มขึ้นจาก 0.63 เป็น 0.85 ($p = 0.003$) การรวมผลลัพธ์จาก Dense Retrieval และ Graph Retrieval โดยตรงไม่ให้ประโยชน์เพิ่มเติมในการทดลองนี้ แสดงให้เห็นว่าการออกแบบกลไก Fusion มีผลต่อประสิทธิภาพของระบบ นอกจากนี้ Provenance-aware Reranker ที่พัฒนาขึ้นช่วยเพิ่มค่า Hit@1 จาก 0.739 เป็น 0.826 เมื่อประเมินด้วยชุดข้อมูล Held-out (ชุด B, n=23) ทั้งนี้ตรวจพบการรั่วไหลของข้อมูลทดสอบในกราฟความรู้ จึงรายงานผลแยกตามชุดข้อมูลเพื่อป้องกันการประเมินประสิทธิภาพสูงเกินจริง การเปรียบเทียบแบบจำลองภาษาพบว่า API-based LLM ให้คะแนน Correctness สูงกว่า Local 4B ขณะที่คอขวดด้านเวลาเกิดจากการจัดอันดับด้วย Cross-encoder บน CPU โครงงานนี้ยังนำเสนอการวิเคราะห์ข้อผิดพลาดและแนวทางปรับปรุงที่สามารถประเมินผลซ้ำได้

คำสำคัญ: *การสร้างผลลัพธ์เสริมด้วยการดึงข้อมูล, การดึงข้อมูลแบบผสม, กราฟความรู้, Reciprocal Rank Fusion, กฎหมายแรงงานไทย, แบบจำลองภาษาขนาดใหญ่, แชตบอต LINE*

English

This project developed ThaiLaw Assistant, a LINE-based question-answering system for Thai labor law covering 187 sections of Thailand's Labor Protection Act, B.E. 2541 (1998). The system employs a Hybrid Retrieval-Augmented Generation (Hybrid RAG) approach that combines dense retrieval using bge-m3 and ChromaDB, Thai-language BM25 retrieval, and Graph RAG. The pipeline includes a query router, weighted Reciprocal Rank Fusion, graph-seeded expansion, and provenance-aware reranking to improve the accuracy and reliability of the context provided to the large language model. Fallback mechanisms and error handling were also implemented to support practical use. The system was evaluated using 50 questions covering seven categories with reference answers.

The results showed that Hybrid RAG significantly outperformed Dense RAG. When evaluated with the actual RAGEngine, Recall@5 increased from 0.625 to 0.823 ($p = 0.001$) and Hit@1 increased from 0.565 to 0.696 ($p = 0.014$), while citation recall increased from 0.63 to 0.85 ($p = 0.003$). Directly combining Dense Retrieval and Graph Retrieval outputs provided no additional benefit, indicating that the fusion mechanism affects system performance. The proposed provenance-aware reranker improved Hit@1 from 0.739 to 0.826 when evaluated on the held-out set (set B, $n = 23$). Test-set contamination was identified in the knowledge graph; therefore, the results were reported separately by dataset to avoid overstating performance. The comparison of language models showed that the API-based LLM achieved higher Correctness scores than the Local 4B model, while the main latency bottleneck was CPU-based cross-encoder reranking. The project also presents an in-depth error analysis and reproducible improvement strategies.

Keywords: *Retrieval-Augmented Generation, Hybrid RAG, Knowledge Graph, Reciprocal Rank Fusion, Thai Labor Law, Large Language Model, LINE Chatbot*

กิตติกรรมประกาศ (Acknowledgements)

โครงงานนี้เป็นส่วนหนึ่งของรายวิชา 241-351 Artificial Intelligences for Social Media Module คณะผู้จัดทำขอขอบพระคุณอาจารย์ผู้สอนที่ให้คำแนะนำเกี่ยวกับแนวคิด Retrieval-Augmented Generation และเกณฑ์การประเมินผล ซึ่งเป็นประโยชน์ต่อการออกแบบการทดลองเพื่อเปรียบเทียบ Dense RAG, Graph RAG และ Hybrid RAG อย่างเป็นระบบ

คณะผู้จัดทำขอขอบคุณผู้พัฒนาและองค์กรที่อยู่เบื้องหลังเครื่องมือ แบบจำลอง และบริการที่ใช้ในโครงงาน ได้แก่ Ollama, ChromaDB, Neo4j, PyThaiNLP, BAAI (BGE-M3 และ BGE-Reranker) และ dotBlue ซึ่งทีมเลือกใช้บริการ API ในการพัฒนาและประเมินระบบ รวมถึงกรมสวัสดิการและคุ้มครองแรงงาน สำหรับการเผยแพร่ข้อมูลและคู่มือที่นำมาใช้เป็นฐานความรู้ของระบบ

ท้ายที่สุดนี้ คณะผู้จัดทำขอขอบคุณสมาชิกทุกคนในกลุ่มที่ร่วมกันพัฒนา ทดสอบ และแก้ไขข้อบกพร่องจนโครงงานสำเร็จลุล่วงด้วยดี

สารบัญ (Table of Contents)

บทคัดย่อ (Abstract) ..... ก

กิตติกรรมประกาศ (Acknowledgements) ..... ค

สารบัญ (Table of Contents) ..... ง

บทที่ 1: บทนำ (Introduction) 1

1.1 ความเป็นมาและความสำคัญของปัญหา (Background and Rationale) 1

1.2 วัตถุประสงค์ของโครงงาน (Project Objectives) 2

1.3 ขอบเขตของโครงงาน (Scope of Work) 3

1.4 ประโยชน์ที่คาดว่าจะได้รับ (Expected Benefits) 4

บทที่ 2: ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง (Technical Background) 5

2.1 แนวคิดพื้นฐานด้าน Retrieval-Augmented Generation (RAG) 5

2.2 Dense Retrieval และ Vector Database 6

2.3 Graph RAG และ Knowledge Graph Structure 7

2.4 Hybrid RAG Architecture & Fusion Techniques 8

2.5 Large Language Models (Local LLM vs API LLM) 9

2.6 ตัวชี้วัดการประเมินการค้นคืนข้อมูล (Retrieval Evaluation Metrics) 11

บทที่ 3: การออกแบบและสถาปัตยกรรมระบบ (System Architecture & Design) 13

3.1 ภาพรวมสถาปัตยกรรมระบบ (System Architecture Overview) 13

3.2 การจัดเตรียมและการประมวลผลข้อมูล (Data Pipeline & Knowledge Base Construction) 15

3.3 การพัฒนาระบบค้นคืนข้อมูล (Retrieval Engine Implementation) 22

3.4 การบูรณาการโมเดลภาษาและการบำรุงรักษาระบบ (LLM Integration & Infrastructure) 25

บทที่ 4: การทดลอง ผลการทดสอบ และการวิเคราะห์ผล (Experiments, Results & Discussion) 28

4.1 สภาพแวดล้อมและระเบียบวิธีการทดลอง (Experimental Setup & Benchmark) 28

4.2 ผลการทดสอบเปรียบเทียบประสิทธิภาพการค้นคืน (Retrieval Performance Comparison) 32

4.3 ผลการทดสอบเปรียบเทียบโมเดลภาษา (Local LLM vs API LLM Evaluation)..... 41
4.4 การวิเคราะห์และอภิปรายผลเชิงลึก (In-depth Discussion & Error Analysis)..... 50
บทที่ 5: สรุปผลและข้อเสนอแนะ (Conclusion & Future Work)..... 59
5.1 สรุปผลการดำเนินโครงงาน (Project Conclusion)..... 59
5.2 ข้อจำกัดของระบบและปัญหาที่พบ (Limitations & Challenges)..... 62
5.3 ข้อเสนอแนะสำหรับการพัฒนาต่อยอด (Future Recommendations)..... 64
บรรณานุกรม (References)..... 66
ภาคผนวก A: รายงานสรุปผลงานและการวิเคราะห์ส่วนบุคคล (Individual Technical Report & Reflection)
..... 68
A-1 นักศึกษาคนที่ 1: Data / Knowledge Graph / Retrieval Evaluation 68
A-2 นักศึกษาคนที่ 2: Retrieval Fusion / LLM Integration / LINE Application 71
ภาคผนวก B: คู่มือการติดตั้งและการใช้งานระบบ (Installation & User Manual)..... 76
B-1 ข้อกำหนดระบบและ Dependencies (System Requirements and Dependencies)..... 76
B-2 ขั้นตอนการติดตั้งสภาพแวดล้อมและการเรียกใช้ระบบ (Setup Environment and Run Application)
..... 76
B-3 ตัวอย่าง User Interface..... 79

บทที่ 1: บทนำ (Introduction)

1.1 ความเป็นมาและความสำคัญของปัญหา (Background and Rationale)

แบบจำลองภาษาขนาดใหญ่ (Large Language Models: LLMs) สามารถสร้างคำตอบด้วยภาษาธรรมชาติได้อย่างมีประสิทธิภาพ อย่างไรก็ตาม การนำแบบจำลองดังกล่าวมาใช้ตอบคำถามด้านกฎหมายจำเป็นต้องคำนึงถึงความถูกต้อง ความสอดคล้องกับตัวบทกฎหมาย และการมีหลักฐานอ้างอิงที่ตรวจสอบได้ เนื่องจากแบบจำลองอาจสร้างข้อมูลที่ไม่ถูกต้องหรือไม่มีหลักฐานรองรับ ซึ่งเรียกว่าการสร้างข้อมูลหลอน (Hallucination) เช่น การระบุเลขมาตรา อัตราโทษ หรือเงื่อนไขทางกฎหมายคลาดเคลื่อน ปัญหาดังกล่าวอาจส่งผลให้ผู้ใช้นำข้อมูลไปใช้ผิดและเกิดความเข้าใจด้านกฎหมายที่ไม่ถูกต้อง

Retrieval-Augmented Generation (RAG) [1], [9] เป็นแนวทางที่ช่วยลดปัญหาดังกล่าว โดยระบบจะค้นคืนข้อมูลจากฐานความรู้ที่ผ่านการจัดเตรียมและตรวจสอบแล้ว จากนั้นนำข้อมูลที่ค้นคืนได้มาใช้เป็นบริบทประกอบการสร้างคำตอบของ LLM วิธีการนี้ช่วยให้คำตอบมีแหล่งที่มาและสามารถตรวจสอบย้อนกลับได้ อย่างไรก็ตาม Dense Retrieval ซึ่งค้นคืนข้อมูลจากความคล้ายคลึงเชิงความหมายของเวกเตอร์ (Vector Embedding) อาจมีข้อจำกัดในการค้นหาคำถามที่ระบุเลขมาตรา คำศัพท์เฉพาะ หรือรายละเอียดที่ต้องตรงกับข้อความต้นฉบับ นอกจากนี้ยังอาจไม่สามารถใช้ประโยชน์จากความสัมพันธ์ระหว่างมาตราต่าง ๆ ได้อย่างครบถ้วน

กฎหมายแรงงานมีลักษณะเป็นข้อมูลที่เชื่อมโยงกันหลายส่วน เช่น มาตราที่กำหนดสิทธิ มาตราที่กำหนดหน้าที่ มาตราที่กำหนดบทลงโทษ และมาตราที่มีการอ้างอิงถึงกัน ความสัมพันธ์เหล่านี้สามารถนำเสนอด้วยกราฟความรู้ (Knowledge Graph) ซึ่งช่วยเชื่อมโยงมาตรา หน่วยงานที่เกี่ยวข้อง หลักฐาน และขั้นตอนการร้องเรียน โดยความสัมพันธ์ในกราฟอาจรวมถึงการเชื่อมโยงระหว่างสิทธิกับบทลงโทษ เช่น PENALIZED_BY และการอ้างอิงข้ามมาตรา เช่น REFERS_TO อย่างไรก็ตาม Graph Retrieval เพียงอย่างเดียวอาจไม่ครอบคลุมคำถามที่ใช้ถ้อยคำแตกต่างจากคำหรือแนวคิดที่กำหนดไว้ในกราฟ

ด้วยเหตุนี้ โครงงานจึงนำเสนอแนวทาง Hybrid Retrieval-Augmented Generation (Hybrid RAG) ซึ่งผสานการค้นคืนหลายรูปแบบ ได้แก่ Dense Retrieval, BM25 สำหรับภาษาไทย และ Graph Retrieval เพื่อใช้จุดเด่นของแต่ละวิธีร่วมกัน Dense Retrieval ช่วยค้นคืนคำถามตามความหมาย BM25 ช่วยค้นคืนคำสำคัญและคำศัพท์เฉพาะได้อย่างแม่นยำ ส่วน Graph Retrieval ช่วยใช้ประโยชน์จากความสัมพันธ์ระหว่างข้อมูลและ

มาตราทางกฎหมาย การผสานวิธีการเหล่านี้มีเป้าหมายเพื่อเพิ่มความครอบคลุม ความถูกต้อง และความสามารถในการอ้างอิงแหล่งที่มาของคำตอบ

โครงงานนี้เลือกศึกษาพระราชบัญญัติคุ้มครองแรงงาน พ.ศ. 2541 เนื่องจากเป็นกฎหมายที่เกี่ยวข้องกับการทำงานในชีวิตประจำวัน เช่น การลาป่วย ค่าล่วงเวลา วันหยุด ค่าจ้าง และค่าชดเชยกรณีเลิกจ้าง อีกทั้งเนื้อหาของกฎหมายมีการอ้างอิงและเชื่อมโยงระหว่างมาตราหลายส่วน จึงเหมาะสำหรับการพัฒนาระบบตอบคำถามที่สามารถประเมินประสิทธิภาพได้อย่างเป็นรูปธรรม

ระบบที่พัฒนาขึ้นมีชื่อว่า ThaiLaw Assistant และให้บริการในรูปแบบแชตบอตบนแพลตฟอร์ม LINE เพื่อให้ผู้ใช้สามารถเข้าถึงข้อมูลกฎหมายแรงงานผ่านแอปพลิเคชันที่คุ้นเคยได้สะดวก ระบบมุ่งเน้นการตอบคำถามโดยอ้างอิงข้อมูลจากตัวบทกฎหมาย พร้อมแสดงแหล่งที่มาของข้อมูล เพื่อช่วยให้ผู้ใช้ตรวจสอบคำตอบและทำความเข้าใจสิทธิหรือหน้าที่ตามกฎหมายได้อย่างเหมาะสม

1.2 วัตถุประสงค์ของโครงงาน (Project Objectives)

1. *เพื่อออกแบบและพัฒนาระบบ Hybrid Retrieval-Augmented Generation (Hybrid RAG) ที่ผสานการค้นคืนข้อมูลแบบ Dense Retrieval ด้วย ChromaDB การค้นคืนแบบ Sparse Retrieval ด้วย BM25 และ Graph RAG โดยใช้กราฟความรู้ (จัดเก็บบน Neo4j และ graph snapshot) พร้อมพัฒนากระบวนการ Query Routing, Result Fusion, Reranking และ Context Aggregation อย่างเป็นระบบ*
2. *เพื่อเปรียบเทียบประสิทธิภาพของระบบ Dense RAG, Graph RAG และ Hybrid RAG ในการตอบคำถามกฎหมายแรงงานไทย โดยใช้ชุดทดสอบจำนวน 50 คำถาม ครอบคลุม 7 ประเภท และวิเคราะห์ปัจจัยเชิงเทคนิคที่ส่งผลต่อความแตกต่างของประสิทธิภาพระหว่างแต่ละแนวทาง*
3. *เพื่อประเมินและเปรียบเทียบประสิทธิภาพการทำงานของแบบจำลองภาษาขนาดใหญ่แบบ Local ผ่าน Ollama และแบบ API-based LLM ผ่านแพลตฟอร์ม dotBlue/PSU ในด้านคุณภาพและความถูกต้องของคำตอบ เวลาในการตอบสนอง การใช้ทรัพยากรหน่วยความจำ VRAM/RAM และปริมาณการใช้โทเคน*

1.3 ขอบเขตของโครงงาน (Scope of Work)

1.3.1 ขอบเขตด้านชุดข้อมูล (Dataset)

1. *ใช้ตัวบทพระราชบัญญัติคุ้มครองแรงงาน พ.ศ. 2541 ฉบับปรับปรุงล่าสุด [15] จำนวน 1 ฉบับ ครอบคลุม 18 หมวด และ 187 มาตรา แบ่งเป็นมาตราที่มีผลบังคับใช้จำนวน 177 มาตรา และมาตราที่ยกเลิก จำนวน 10 มาตรา โดยนำข้อมูลมาแบ่งเป็น 195 ชิ้นข้อมูล (Chunks)*
2. *ไม่ครอบคลุมกฎหมายแรงงานฉบับอื่น เช่น พระราชบัญญัติประกันสังคม และพระราชบัญญัติเงินทดแทน โดยข้อจำกัดดังกล่าวจะกล่าวถึงเพิ่มเติมในหัวข้อ 5.2*

1.3.2 ขอบเขตด้านระบบค้นคืนข้อมูล (Information Retrieval)

1. *ใช้ ChromaDB เป็นฐานข้อมูลเวกเตอร์ โดยใช้ HNSW และ Cosine Similarity ร่วมกับโมเดล Embedding BAAI/bge-m3*
2. *ใช้ BM25 เป็นการค้นคืนข้อมูลแบบ Sparse Retrieval จำนวน 2 รูปแบบ ได้แก่ Word-level Retrieval ซึ่งตัดคำด้วย PyThaiNLP newmm และ Character 3-gram Retrieval เพื่อรองรับคำสะกดที่แตกต่างกันและการค้นคืนจากเลขมาตรา*
3. *ใช้ Neo4j 5.24 Community Edition บน Docker เป็นฐานข้อมูลกราฟความรู้ โดยมีข้อมูลกราฟสำหรับการค้นคืนแบบออฟไลน์จำนวน 326 โหนด และ 720 เส้นเชื่อม ครอบคลุมโหนด 12 ประเภท*
4. *ใช้โมเดล BAAI/bge-reranker-v2-m3 เป็น Cross-encoder Reranker จัดอันดับผลลัพธ์ที่ค้นคืนได้*

1.3.3 ขอบเขตด้านแบบจำลองภาษา (Large Language Models)

1. *แบบจำลองภาษาแบบ Local ใช้งานผ่าน Ollama ได้แก่ qwen3.5:4b และ gemma3:4b โดยประมวลผลบน NVIDIA RTX 3050 Laptop ที่มี VRAM ขนาด 6 GB*
2. *แบบจำลองภาษาแบบ API ใช้งานผ่าน dotBlue ซึ่งเป็นเกตเวย์ของ PSU AI ที่รองรับรูปแบบการเรียกใช้งานแบบ OpenAI-compatible ได้แก่ qwen/qwen3.6-flash ซึ่งใช้เป็นแบบจำลองหลัก รวมถึง gpt-4o-mini และ deepseek-chat*
3. *ใช้ qwen/qwen3.6-plus เป็นแบบจำลองสำหรับการประเมินผลคำตอบโดยอัตโนมัติ*

1.3.4 ขอบเขตด้านช่องทางการใช้งาน (Application Interface)

ระบบให้บริการผ่าน LINE Messaging API [14] โดยพัฒนาด้วย Flask Webhook และใช้ Flex Message รวมถึง Rich Menu สำหรับการโต้ตอบกับผู้ใช้งาน

1.4 ประโยชน์ที่คาดว่าจะได้รับ (Expected Benefits)

1. **ด้านผู้ใช้งาน:** *ได้ระบบตอบคำถามกฎหมายแรงงานไทยผ่าน LINE ที่สามารถแสดงคำตอบพร้อมการอ้างอิงเลขมาตราที่เกี่ยวข้อง ช่วยให้ผู้ใช้เข้าถึงข้อมูลได้สะดวกโดยไม่จำเป็นต้องค้นหาจากตัวบทกฎหมายทั้งฉบับ*
2. **ด้านวิศวกรรมซอฟต์แวร์:** *ได้สถาปัตยกรรมระบบ Hybrid RAG ที่แบ่งหน้าที่การทำงานอย่างชัดเจน พร้อมชุดทดสอบเพื่อช่วยตรวจสอบความถูกต้องของระบบ และสามารถนำสถาปัตยกรรมไปประยุกต์ใช้กับกฎหมายหรือชุดข้อมูลอื่นได้*
3. **ด้านวิชาการ:** *ได้ผลการวิเคราะห์เชิงประจักษ์เกี่ยวกับข้อดี ข้อจำกัด และเงื่อนไขที่ส่งผลต่อประสิทธิภาพของ Hybrid RAG เมื่อเปรียบเทียบกับ Dense RAG รวมถึงข้อค้นพบจากผลการทดลองที่ไม่เป็นไปตามสมมติฐาน*
4. **ด้านการเลือกใช้แบบจำลองภาษา:** *ได้ข้อมูลสำหรับเปรียบเทียบแบบจำลองภาษาแบบ Local และแบบ API ในด้านคุณภาพคำตอบ เวลาในการตอบสนอง การใช้ทรัพยากร VRAM/RAM และปริมาณการใช้โทเคน เพื่อสนับสนุนการเลือกแนวทางติดตั้งและใช้งานระบบให้เหมาะสมกับข้อจำกัดด้านทรัพยากรและลักษณะการใช้งานจริง*

บทที่ 2: ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง (Technical Background)

2.1 แนวคิดพื้นฐานด้าน Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) คือสถาปัตยกรรมที่ผสานความสามารถของระบบค้นคืนข้อมูลเข้ากับแบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM) โดยแยกความรู้ที่ใช้ตอบคำถามออกจากพารามิเตอร์ของแบบจำลอง เมื่อระบบได้รับคำถาม ระบบจะค้นคืนเอกสารหรือข้อมูลที่เกี่ยวข้องจากฐานความรู้ภายนอก (Retrieval) จากนั้นนำข้อมูลที่ค้นคืนได้มาใช้เป็นบริบทประกอบคำถาม (Context Injection) และส่งให้ LLM สร้างคำตอบ (Generation)

กระบวนการทำงานของ RAG โดยทั่วไปประกอบด้วย 3 ขั้นตอนหลัก ได้แก่

1. *การค้นคืนข้อมูล (Retrieval): ค้นหาเอกสารหรือข้อมูลที่เกี่ยวข้องกับคำถามจากฐานความรู้*
2. *การจัดเตรียมบริบท (Context Construction): คัดเลือก จัดอันดับ และจัดรูปแบบข้อมูลที่ค้นคืนได้เพื่อส่งต่อให้แบบจำลองภาษา*
3. *การสร้างคำตอบ (Generation): ให้ LLM สร้างคำตอบโดยอาศัยคำถามและบริบทที่ค้นคืนมา*

ข้อดีของ RAG คือสามารถเพิ่มหรือปรับปรุงข้อมูลในฐานความรู้ได้โดยไม่จำเป็นต้องฝึกแบบจำลองภาษาใหม่ทั้งหมด อีกทั้งช่วยลดความเสี่ยงจากการสร้างข้อมูลที่ไม่ถูกต้อง (Hallucination) และสนับสนุนการอ้างอิงแหล่งที่มาของคำตอบ อย่างไรก็ตาม RAG ไม่สามารถรับประกันความถูกต้องของคำตอบได้โดยสมบูรณ์ เนื่องจากประสิทธิภาพของระบบขึ้นอยู่กับทั้งคุณภาพของการค้นคืนข้อมูลและความสามารถของ LLM ในการใช้บริบทอย่างถูกต้อง

ดังนั้น การประเมินระบบ RAG ควรพิจารณาแยกเป็น 2 ส่วน ได้แก่ คุณภาพของการค้นคืนข้อมูล (Retrieval Quality) ซึ่งพิจารณาว่าระบบสามารถค้นคืนมาตราหรือข้อมูลที่เกี่ยวข้องได้หรือไม่ และ คุณภาพของการสร้างคำตอบ (Generation Quality) ซึ่งพิจารณาความถูกต้อง ความครบถ้วน และความสอดคล้องของคำตอบกับบริบทที่ค้นคืนได้ แนวคิดดังกล่าวเป็นพื้นฐานสำคัญของการออกแบบและประเมินระบบ ThaiLaw Assistant

2.2 Dense Retrieval และ Vector Database

Dense Retrieval เป็นวิธีการค้นคืนข้อมูลโดยแปลงคำถามและเอกสารให้อยู่ในรูปเวกเตอร์ความหนาแน่นสูง (Dense Vector) ด้วยแบบจำลอง Embedding โดยข้อความที่มีความหมายใกล้เคียงกันจะมีตำแหน่งอยู่ใกล้กันในปริภูมิเวกเตอร์ โครงงานนี้ใช้แบบจำลอง BGE-M3 [2] ซึ่งรองรับหลายภาษา รวมถึงภาษาไทย และสามารถประมวลผลข้อความที่มีความยาวได้ถึง 8,192 โทเคน

การวัดความคล้ายคลึงระหว่างเวกเตอร์คำถามและเวกเตอร์เอกสารใช้ Cosine Similarity โดยมีสมการดังนี้

$$\mathrm{sim}(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$$

เมื่อ q คือเวกเตอร์ของคำถาม และ d คือเวกเตอร์ของเอกสาร ค่าความคล้ายคลึงที่สูงหมายถึงคำถามและเอกสารมีทิศทางของเวกเตอร์ใกล้เคียงกัน และมีแนวโน้มที่จะมีความเกี่ยวข้องกันมากกว่า

2.2.1 Vector Database และการค้นหา Nearest Neighbor: *Vector Database ใช้สำหรับจัดเก็บเวกเตอร์ของเอกสารและค้นหาเอกสารที่มีความใกล้เคียงกับเวกเตอร์ของคำถาม โครงงานนี้ใช้ ChromaDB [12] เป็นฐานข้อมูลเวกเตอร์ โดยใช้ดัชนี HNSW (Hierarchical Navigable Small World) สำหรับการค้นหา Approximate Nearest Neighbor การค้นหาในลักษณะนี้ช่วยลดเวลาในการค้นคืนเมื่อมีข้อมูลจำนวนมาก โดยแลกกับการยอมให้ผลลัพธ์เป็นค่าประมาณที่มีความคลาดเคลื่อนเล็กน้อย*

2.2.2 Top-K และ Similarity Threshold: *Top-K คือจำนวนเอกสารหรือ Candidate ที่ระบบเลือกจากผลการค้นคืนเพื่อนำไปประมวลผลในขั้นตอนถัดไป ส่วน Similarity Threshold คือค่าขั้นต่ำของความคล้ายคลึงที่ใช้คัดกรองเอกสาร หากคะแนนต่ำกว่าเกณฑ์ที่กำหนด ระบบจะไม่นำเอกสารดังกล่าวไปใช้เป็นบริบท การกำหนดค่า Top-K และ Similarity Threshold ต้องพิจารณาร่วมกันเพื่อให้ระบบมีบริบทเพียงพอ โดยไม่ส่งข้อมูลที่ไม่เกี่ยวข้องเข้าสู่ขั้นตอนการสร้างคำตอบมากเกินไป*

2.2.3 Reranking: *การค้นคืนด้วย Embedding หรือ Bi-encoder จะเข้ารหัสคำถามและเอกสารแยกจากกัน จึงมีความรวดเร็วและเหมาะสำหรับการค้นคืนในขั้นต้น แต่ความแม่นยำอาจไม่สูงเท่าการพิจารณาคำถามและเอกสารร่วมกัน Cross-encoder จะประมวลผลคู่คำถามและเอกสารพร้อมกัน ทำให้สามารถวิเคราะห์ความสัมพันธ์ระหว่างข้อมูลได้ละเอียดกว่า แต่ใช้ทรัพยากรและเวลาในการประมวลผลมากกว่า*

โครงงานนี้ใช้ BAAI/bge-reranker-v2-m3 [3] เป็น Cross-encoder Reranker โดยนำมาใช้กับ Candidate จำนวนจำกัดที่ผ่านการค้นคืนในขั้นแรกแล้ว แนวทางดังกล่าวเรียกว่า Retrieve-then-Rerank ซึ่งช่วยรักษาสมดุลระหว่างความเร็วในการประมวลผลและความแม่นยำของการจัดอันดับผลลัพธ์

2.2.4 BM25 และการค้นคืนเชิงคำศัพท์: *BM25 [4] เป็นวิธีการค้นคืนข้อมูลเชิงคำศัพท์ (Lexical Retrieval) ที่คำนวณความเกี่ยวข้องของเอกสารจากความถี่ของคำในเอกสาร ความถี่ของคำในคลังเอกสาร และความยาวของ*

เอกสาร วิธีการนี้เหมาะกับการค้นหาคำศัพท์เฉพาะ เลขมาตรา และข้อความที่ต้องตรงกับเอกสารต้นฉบับ เนื่องจากภาษาไทยไม่มีการเว้นวรรคระหว่างคำอย่างสม่ำเสมอ ระบบจึงต้องตัดคำก่อนประมวลผล BM25 โดยใช้ PyThaiNLP และอัลกอริทึม newmm [5] นอกจากนี้ยังใช้การแบ่งข้อความเป็น Character 3-gram เพื่อช่วยรองรับคำสะกดแตกต่างกัน คำที่ตัดคำได้ไม่สมบูรณ์ และคำศัพท์เฉพาะทางกฎหมาย การใช้ BM25 ร่วมกับ Dense Retrieval ช่วยให้ระบบค้นคืนได้ทั้งเอกสารที่มีความหมายใกล้เคียงและเอกสารที่มีคำสำคัญตรงกับคำถาม

2.3 Graph RAG และ Knowledge Graph Structure

กราฟความรู้ (Knowledge Graph: KG) เป็นโครงสร้างสำหรับจัดเก็บความรู้ในรูปแบบเครือข่าย ประกอบด้วยโหนด (Node) ซึ่งแทนเอนทิตีหรือข้อมูล และเส้นเชื่อม (Relationship หรือ Edge) ซึ่งแทนความสัมพันธ์ระหว่างโหนด โดยเส้นเชื่อมสามารถกำหนดทิศทางและประเภทของความสัมพันธ์ได้

Graph Retrieval-Augmented Generation (Graph RAG) [6] นำโครงสร้างกราฟความรู้มาใช้ร่วมกับกระบวนการค้นคืนข้อมูล เพื่อค้นหาข้อมูลที่มีความสัมพันธ์เชื่อมโยงกันหลายขั้น (Multi-hop Relationship) ซึ่งอาจไม่สามารถค้นคืนได้อย่างครบถ้วนด้วยการพิจารณาความคล้ายคลึงของข้อความเพียงอย่างเดียว แนวทางนี้เหมาะกับข้อมูลกฎหมายซึ่งมีความสัมพันธ์ระหว่างมาตรา สิทธิ หน้าที่ บทลงโทษ หน่วยงาน และขั้นตอนการดำเนินการ

โครงงานนี้ออกแบบ Ontology สำหรับกฎหมายแรงงาน โดยกำหนดประเภทของโหนดไว้ 13 ประเภท ได้แก่ Law, Chapter, Section, Term, Right, Duty, Penalty, Actor, Topic, Agency, Evidence, Form และ Step ทั้งนี้ ข้อมูลที่นำมาใช้ในการทดลองจริงมีโหนดที่ใช้งานจำนวน 12 ประเภทตามโครงสร้างของชุดข้อมูล ความสัมพันธ์ระหว่างโหนดประกอบด้วย HAS_CHAPTER, HAS_SECTION, REFERS_TO, PENALIZED_BY, DEFINES, ABOUT, HANDLED_BY, REQUIRES_EVIDENCE, USES_FORM, HAS_STEP, GRANTS_RIGHT, IMPOSES_DUTY, HELD_BY และ BINDS โดยรายละเอียดของโครงสร้าง Ontology แสดงไว้ในหัวข้อ 3.2.4

ระบบใช้ Neo4j เป็นฐานข้อมูลกราฟ และใช้ Cypher [11] ซึ่งเป็นภาษาสืบค้นข้อมูลแบบ Declarative สำหรับค้นหาและเชื่อมโยงโหนดในกราฟ ตัวอย่างคำสั่งสำหรับค้นหามาตราที่เกี่ยวข้องกับบทลงโทษมีดังนี้

```
MATCH (s:Section)-[r:PENALIZED_BY]->(p:Section)
WHERE s.section_no IN $section_nos
   OR p.section_no IN $section_nos
RETURN s, r, p
LIMIT $top_k
```

การใช้ Graph RAG มีประโยชน์ต่อระบบในหลายด้าน ได้แก่

1. *ช่วยค้นคืนข้อมูลตามความสัมพันธ์ที่กำหนดไว้อย่างชัดเจน เช่น ความสัมพันธ์ระหว่างมาตราที่กำหนด*

สิทธิกับมาตราที่กำหนดบทลงโทษ โดยไม่ต้องพึ่งพาความคล้ายคลึงของข้อความเพียงอย่างเดียว

2. *ช่วยเชื่อมโยงข้อมูลเชิงปฏิบัติ เช่น หน่วยงานที่เกี่ยวข้อง หลักฐาน แบบฟอร์ม และขั้นตอนการดำเนินการ เพื่อสนับสนุนการตอบคำถามที่ต้องใช้ข้อมูลจากหลายส่วน*
3. *ช่วยเพิ่มความสามารถในการอธิบายผลลัพธ์ เนื่องจากระบบสามารถตรวจสอบเส้นทางและความสัมพันธ์ของข้อมูลที่ใช้ในการค้นคืนได้ โดยแสดงรายละเอียดเพิ่มเติมผ่านโหมด /debug*

2.4 Hybrid RAG Architecture & Fusion Techniques

Hybrid RAG เป็นสถาปัตยกรรมที่ผสานผลลัพธ์จากระบบค้นคืนข้อมูลหลายรูปแบบ ซึ่งแต่ละรูปแบบมีจุดเด่นแตกต่างกัน โดย Dense Retrieval เหมาะกับการค้นคืนข้อมูลตามความหมาย BM25 เหมาะกับการค้นคืนคำสำคัญและคำศัพท์เฉพาะ ส่วน Graph Retrieval เหมาะกับการค้นคืนข้อมูลที่มีความสัมพันธ์เชื่อมโยงกัน โครงงานนี้ออกแบบกระบวนการ Hybrid RAG โดยประกอบด้วยองค์ประกอบหลัก 4 ส่วน ดังนี้

2.4.1 Weighted Reciprocal Rank Fusion: *Reciprocal Rank Fusion (RRF) [7] เป็นวิธีการรวมผลลัพธ์จาก Retriever หลายตัวโดยพิจารณาจากอันดับของเอกสารในแต่ละผลลัพธ์ ทำให้สามารถเปรียบเทียบและรวมอันดับจาก Retriever ที่มีช่วงคะแนนแตกต่างกันได้ โดยคำนวณคะแนนของเอกสาร d ดังนี้*

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{w_r}{c + \mathrm{rank}_r(d)}$$

เมื่อ R คือเซตของ Retriever, rank_r(d) คืออันดับของเอกสาร d จาก Retriever r, w_r คือน้ำหนักของ Retriever แต่ละตัว และ c คือค่าคงที่ที่ช่วยลดผลกระทบจากเอกสารที่มีอันดับต่ำ โดยโครงงานนี้กำหนด c=60 ตามแนวทางดั้งเดิมของ RRF

กรณีที่ Retriever ทุกตัวมีน้ำหนักเท่ากันจะเป็น RRF มาตรฐาน ส่วนการกำหนดน้ำหนักแตกต่างกันเรียกว่า Weighted RRF ซึ่งช่วยให้ระบบปรับความสำคัญของ Dense Retrieval, BM25 และ Graph Retrieval ให้เหมาะสมกับลักษณะของคำถามแต่ละประเภท

2.4.2 Query Routing: *Query Routing เป็นกระบวนการจำแนกประเภทคำถามก่อนเข้าสู่การค้นคืนข้อมูล โดยระบบแบ่งคำถามออกเป็นประเภทต่าง ๆ เช่น Lookup, Definition, Penalty, Procedure, Multi-hop และ General จากนั้นกำหนดน้ำหนักของ Retriever แต่ละประเภทให้แตกต่างกันตามลักษณะของคำถาม*

ตัวอย่างเช่น คำถามที่ระบุเลขมาตราหรือคำศัพท์เฉพาะจะให้น้ำหนักกับ BM25 และ Graph Retrieval สูงขึ้น

ขณะที่คำถามทั่วไปจะให้น้ำหนักกับ Dense Retrieval มากขึ้น แนวทางนี้ช่วยให้ระบบเลือกใช้ความสามารถของ Retriever แต่ละรูปแบบได้เหมาะสมกับอินพุต

2.4.3 Graph-seeded Expansion: *Graph-seeded Expansion เป็นกระบวนการขยายผลลัพธ์โดยนำเอกสารหรือมาตราที่มีอันดับสูงจากผลการค้นคืนที่ผ่านการรวมแล้วมาใช้เป็นจุดเริ่มต้น (Seed) จากนั้นระบบจะเดินกราฟตามความสัมพันธ์ที่กำหนดไว้ เช่น REFERS_TO และ PENALIZED_BY ในระยะ 1 Hop เพื่อค้นหามาตราหรือข้อมูลที่เกี่ยวข้องเพิ่มเติม และนำข้อมูลเหล่านั้นเข้าสู่ Candidate Pool ก่อนการจัดอันดับในขั้นตอนถัดไป*

2.4.4 Context Aggregation: *Context Aggregation เป็นกระบวนการรวบรวมและจัดรูปแบบข้อมูลที่ผ่านการคัดเลือกแล้ว เพื่อสร้างบริบทสำหรับส่งให้ LLM โดยบริบทประกอบด้วยเนื้อหาของมาตราที่เกี่ยวข้อง รวมถึงข้อมูลเชิงโครงสร้างจากกราฟ เช่น บทลงโทษ หน่วยงานที่เกี่ยวข้อง หลักฐาน แบบฟอร์ม และขั้นตอนการดำเนินการ*

โครงงานนี้จัดรูปแบบบริบทเป็น Section Card ซึ่งช่วยแบ่งข้อมูลตามมาตราและประเภทของข้อมูลอย่างเป็นระบบ พร้อมควบคุมความยาวของบริบทให้อยู่ภายใต้งบประมาณที่กำหนด เพื่อให้เหมาะสมกับข้อจำกัดด้าน Context Window ของแบบจำลองภาษาแต่ละประเภท

กระบวนการทั้งหมดทำงานตามลำดับตั้งแต่การจำแนกคำถาม การค้นคืนจากหลายแหล่ง การรวมอันดับ การขยายผลลัพธ์ด้วยกราฟ การจัดอันดับใหม่ และการรวมบริบท ก่อนส่งต่อให้ LLM สร้างคำตอบ

2.5 Large Language Models (Local LLM vs API LLM)

แบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM) เป็นองค์ประกอบที่ใช้สร้างคำตอบจากคำถามและบริบทที่ผ่านกระบวนการค้นคืนข้อมูลแล้ว ในโครงงานนี้มีการเปรียบเทียบการใช้งาน LLM 2 รูปแบบ ได้แก่ Local LLM ซึ่งประมวลผลภายในเครื่องผ่าน Ollama และ API LLM ซึ่งเรียกใช้งานผ่านเซิร์ฟเวอร์ของผู้ให้บริการ โดยแต่ละรูปแบบมีข้อดีและข้อจำกัดแตกต่างกัน ดังแสดงในตารางที่ 2.1

ตารางที่ 2.1 การเปรียบเทียบ Local LLM และ API LLM เชิงสถาปัตยกรรม

มิติ	**Local LLM (Ollama [13])**	**API LLM (dotBlue)**
ตำแหน่งการ	*ประมวลผลบนเครื่องของผู้ใช้งาน โดยใช้ GPU*	*ประมวลผลบนเซิร์ฟเวอร์ของผู้ให้บริการ*

มิติ	Local LLM (Ollama [13])	API LLM (dotBlue)
ประมวลผล	*หรือ CPU*	
ขนาดและข้อจำกัดของโมเดล	*ขึ้นอยู่กับความจุ VRAM/RAM ของเครื่อง โดยโครงงานนี้ใช้โมเดลขนาดประมาณ 4B และการบีบอัดแบบ Q4*	*ขึ้นอยู่กับโมเดลและทรัพยากรที่ผู้ให้บริการเปิดให้ใช้งาน*
ความเป็นส่วนตัว	*ข้อมูลสามารถประมวลผลภายในเครื่องได้ โดยไม่จำเป็นต้องส่งไปยังเซิร์ฟเวอร์ภายนอก*	*ข้อมูลคำถามและบริบทถูกส่งผ่านเครือข่ายไปยังเซิร์ฟเวอร์ของผู้ให้บริการ*
ต้นทุน	*มีต้นทุนด้านฮาร์ดแวร์และพลังงาน โดยทั่วไปไม่คิดค่าบริการตามจำนวนโทเคน*	*อาจคิดค่าบริการตามจำนวนโทเคน เครดิต หรือเงื่อนไขของผู้ให้บริการ*
ความหน่วง	*ขึ้นอยู่กับประสิทธิภาพของ GPU/CPU และเวลาในการโหลดโมเดลครั้งแรก*	*ขึ้นอยู่กับความเร็วเครือข่าย ภาระงาน และคิวของเซิร์ฟเวอร์*
ความเสี่ยงและข้อจำกัด	*อาจมีปัญหา VRAM/RAM ไม่เพียงพอ ความเร็วในการประมวลผลต่ำ หรือรองรับผู้ใช้พร้อมกันได้น้อย*	*อาจเกิด Rate Limit เซิร์ฟเวอร์ไม่พร้อมใช้งาน หรือการเชื่อมต่อและการส่งผลลัพธ์แบบสตรีมขัดข้อง*

แบบจำลองบางประเภทอาจมีกระบวนการให้เหตุผลภายใน (Hidden Reasoning) ก่อนสร้างคำตอบ กระบวนการดังกล่าวใช้ Completion Token เพิ่มเติม แม้โทเคนเหล่านั้นจะไม่แสดงในผลลัพธ์ที่ผู้ใช้เห็น ดังนั้น การกำหนดค่า num_predict หรือ max_tokens ต้องมีขนาดเพียงพอ มิฉะนั้นระบบอาจสร้างคำตอบได้ไม่ครบถ้วนหรือไม่ส่งคืนคำตอบ การวิเคราะห์ผลกระทบของกรณีดังกล่าวในโครงงานนี้นำเสนอไว้ในหัวข้อ 4.4.6

2.5.1 การประเมินผลด้วย LLM-as-a-Judge

LLM-as-a-Judge [8], [10] เป็นแนวทางที่ใช้แบบจำลองภาษาในการประเมินคุณภาพคำตอบของแบบจำลองภาษาอีกตัวหนึ่งตามเกณฑ์ที่กำหนดไว้ล่วงหน้า โครงงานนี้ใช้แนวคิดดังกล่าวเพื่อประเมินคำตอบในด้าน

Correctness, Faithfulness, Citation Accuracy, Thai Clarity และ Key-point Coverage

แนวทางนี้ช่วยให้สามารถประเมินคำตอบจำนวนมากด้วยเกณฑ์เดียวกันและลดเวลาเมื่อเทียบกับการประเมินโดยมนุษย์ อย่างไรก็ตาม ผลการประเมินอาจได้รับอิทธิพลจากอคติของแบบจำลองผู้ประเมิน เช่น ความแตกต่างด้านรูปแบบภาษา ความยาวของคำตอบ หรือความเอนเอียงต่อคำตอบบางลักษณะ ดังนั้นควรรายงานเกณฑ์การประเมินและข้อจำกัดของ LLM-as-a-Judge ร่วมกับผลการทดลอง โดยรายละเอียดข้อจำกัดแสดงไว้ในหัวข้อ 5.2

2.6 ตัวชี้วัดการประเมินการค้นคืนข้อมูล (Retrieval Evaluation Metrics)

กำหนดให้ G คือเซตของมาตราเฉลย (Gold Sections) ที่เกี่ยวข้องกับคำถาม และ L_k คือรายการมาตราที่ระบบค้นคืนได้จำนวน k รายการ โดยเรียงตามลำดับคะแนนจากมากไปน้อย ตัวชี้วัดที่ใช้ประเมินประสิทธิภาพของระบบค้นคืนมีดังนี้

2.6.1 Recall@k: *Recall@k วัดสัดส่วนของมาตราเฉลยที่ปรากฏอยู่ในผลลัพธ์ k อันดับแรก โดยคำนวณจาก*

$$\text{Recall@k} = \frac{|G \cap L_k|}{|G|}$$

ค่าที่สูงแสดงว่าระบบสามารถค้นคืนมาตราที่เกี่ยวข้องได้ครบถ้วนมากขึ้น

2.6.2 Hit@1: *Hit@1 ใช้ตรวจสอบว่ามาตราอันดับหนึ่งที่ระบบค้นคืนได้เป็นมาตราเฉลยหรือไม่ โดยกำหนดค่าเป็น*

$$\text{Hit@1} = \begin{cases} 1, & \text{ถ้า } L_1 \in G \\ 0, & \text{ถ้า } L_1 \notin G \end{cases}$$

ค่า Hit@1 ที่รายงานในระดับชุดข้อมูลคำนวณจากค่าเฉลี่ยของผลลัพธ์แต่ละคำถาม

2.6.3 Mean Reciprocal Rank (MRR): *MRR วัดลำดับของมาตราเฉลยรายการแรกที่ระบบค้นคืนพบ โดยให้คะแนนสูงเมื่อมาตราเฉลยปรากฏอยู่ในอันดับต้น ๆ คำนวณจาก*

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{\text{rank}_i}$$

เมื่อ N คือจำนวนคำถาม และ rank_i คือลำดับของมาตราเฉลยรายการแรกสำหรับคำถามที่ i หากไม่พบมาตรา

เฉลยในผลลัพธ์ ให้กำหนดค่า Reciprocal Rank เป็น 0

2.6.4 Normalized Discounted Cumulative Gain (nDCG@k): *nDCG@k ใช้วัดคุณภาพของการจัดอันดับผลลัพธ์ โดยให้คะแนนมากกว่าสำหรับมาตราเฉลยที่อยู่ในอันดับสูง คำนวณจาก*

$$\mathrm{nDCG@k} = \frac{\mathrm{DCG@k}}{\mathrm{IDCG@k}}$$

โดย

$$\mathrm{DCG@k} = \sum_{i=1}^{k} \frac{\mathrm{rel}_i}{\log_2(i+1)}$$

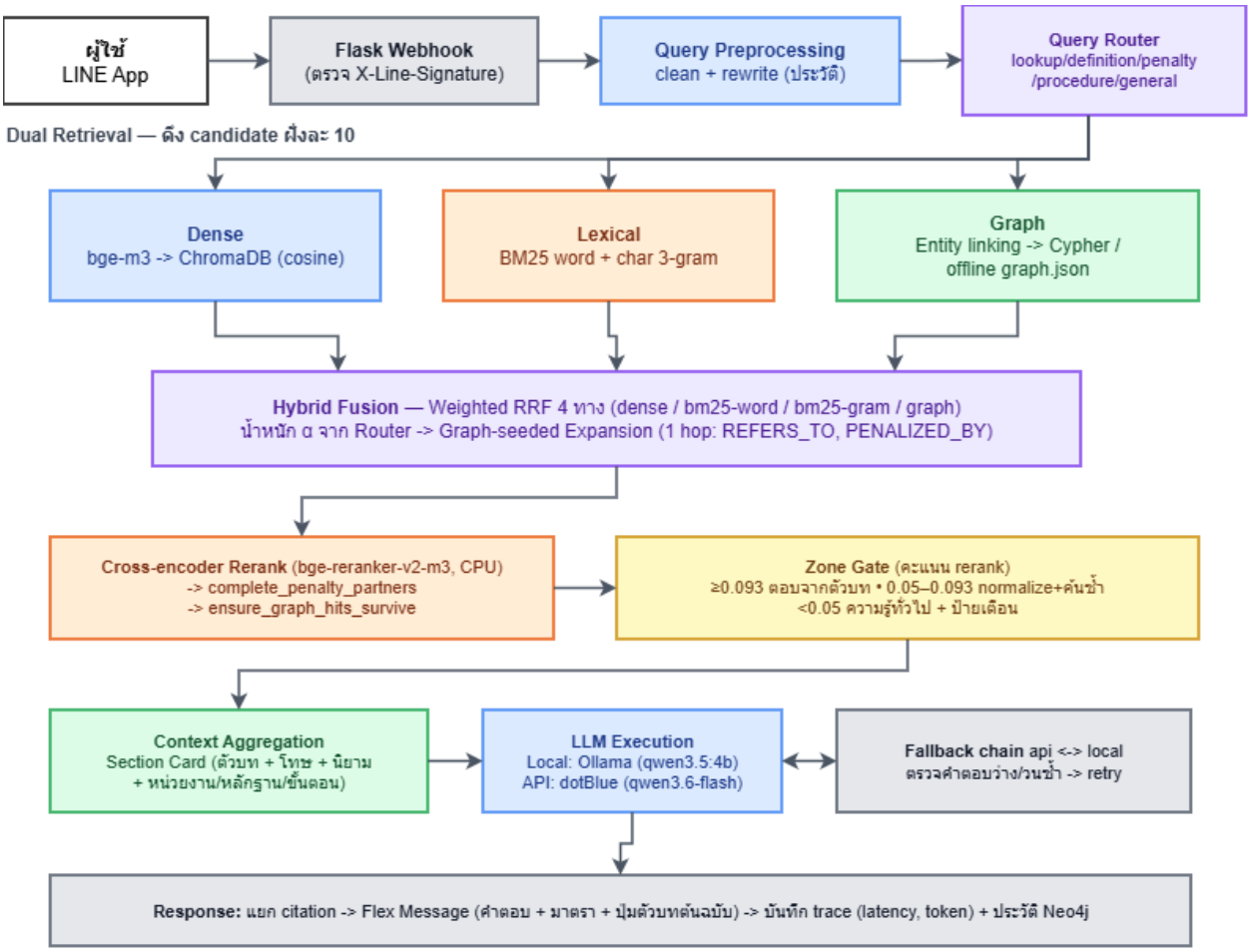
เมื่อ rel_i คือค่าความเกี่ยวข้องของผลลัพธ์ในอันดับที่ i และ IDCG@k คือค่า DCG สูงสุดที่เป็นไปได้จากการจัดอันดับมาตราเฉลยอย่างเหมาะสม ค่า nDCG@k อยู่ในช่วง 0 ถึง 1 โดยค่าที่สูงแสดงว่าระบบจัดอันดับมาตราที่เกี่ยวข้องไว้ในตำแหน่งที่เหมาะสมมากกว่า

สำหรับคำถามประเภท out_of_scope จำนวน 4 คำถาม ซึ่งไม่มีมาตราเฉลย ระบบจะไม่คำนวณตัวชี้วัด Retrieval เนื่องจากไม่มีเซต G สำหรับเปรียบเทียบ โดยจะรายงานผลของคำถามกลุ่มนี้แยกต่างหาก และคำนวณค่าเฉลี่ยของตัวชี้วัด Retrieval จากคำถามที่อยู่ในขอบเขตจำนวน 46 คำถาม (n=46) เพื่อให้ผลการประเมินสะท้อนประสิทธิภาพของการค้นคืนมาตราที่เกี่ยวข้องอย่างเหมาะสม

บทที่ 3: การออกแบบและสถาปัตยกรรมระบบ (System Architecture & Design)

3.1 ภาพรวมสถาปัตยกรรมระบบ (System Architecture Overview)

ระบบ ThaiLaw Assistant แบ่งการทำงานออกเป็น 2 ส่วนหลัก ได้แก่ **Offline Pipeline** *และ* **Online Pipeline** *โดย Offline Pipeline ใช้สำหรับเตรียมฐานความรู้และดัชนีค้นคืนข้อมูล ส่วน Online Pipeline ใช้สำหรับประมวลผลคำถามและสร้างคำตอบแบบเรียลไทม์ผ่าน LINE ดังแสดงในรูปที่ 3.1*



รูปที่ 3.1 สถาปัตยกรรมระบบ Hybrid RAG ตั้งแต่การรับคำถามจากผู้ใช้ การประมวลผลคำถาม การค้นคืนข้อมูล การหลอมรวมผลลัพธ์ การสร้างคำตอบ และการส่งผลลัพธ์กลับผ่าน LINE

3.1.1 Offline Pipeline

Offline Pipeline ประกอบด้วยกระบวนการเตรียมข้อมูลกฎหมายและสร้างองค์ประกอบสำหรับการค้นคืน ได้แก่

1. *เตรียมและแบ่งตัวบทกฎหมายออกเป็นชิ้นข้อมูล (Chunks)*
2. *สร้างเวกเตอร์แทนข้อความด้วยโมเดล BGE-M3 และจัดเก็บใน ChromaDB*
3. *สร้างดัชนี BM25 แบบ Word-level และ Character 3-gram*
4. *สร้างกราฟความรู้จากความสัมพันธ์ระหว่างมาตรา สิทธิ หน้าที่ บทลงโทษ หน่วยงาน หลักฐาน แบบฟอร์ม และขั้นตอน*
5. *จัดเก็บกราฟความรู้ใน Neo4j และเตรียม Graph Snapshot สำหรับการค้นคืนแบบออฟไลน์*

องค์ประกอบที่สร้างขึ้นจากกระบวนการดังกล่าวจะถูกนำไปใช้ใน Online Pipeline เพื่อค้นคืนข้อมูลและสร้างบริบทสำหรับ LLM

3.1.2 Online Pipeline

ลำดับการทำงานของ Online Pipeline มีขั้นตอนดังนี้

1. **การรับคำถามผ่าน LINE Webhook:** *Flask รับคำขอแบบ POST /callback ตรวจสอบ X-Line-Signature และแยกข้อความตาม Thread โดยใช้ Reply Token ในการส่งคำตอบกลับ นอกจากนี้ยังรองรับคำสั่งควบคุม เช่น /mode, /llm, /debug, /reset และ /intro*
2. **การเตรียมคำถาม (Query Preprocessing):** *ฟังก์ชัน clean_query ใช้ลบอักขระที่มองไม่เห็น เช่น Zero-width Space ซึ่งอาจเกิดจากการพิมพ์ภาษาไทย หากมีประวัติการสนทนาและคำถามมีลักษณะสั้นหรือมีคำสรรพนามอ้างอิง ระบบจะใช้ rewrite_with_history เพื่อปรับคำถามให้มีความสมบูรณ์มากขึ้น*
3. **การจำแนกประเภทคำถาม (Query Routing):** *ฟังก์ชัน classify_query จำแนกประเภทคำถามโดยใช้กฎจาก Regular Expression และคำสำคัญ เช่น เลขมาตรา จากนั้นกำหนดน้ำหนัก* **α** **ของแต่ละ** *Retriever ให้เหมาะสมกับประเภทคำถาม*
4. **การค้นคืนข้อมูลหลายรูปแบบ (Multi-Retrieval):** *ระบบค้นคืนข้อมูลจาก Dense Retrieval ด้วย BGE-M3 และ ChromaDB, Lexical Retrieval ด้วย BM25 แบบ Word-level และ Character 3-gram รวมถึง Graph Retrieval ด้วย Entity Linking และการสืบค้นผ่าน Cypher หรือ Graph Traversal โดยแต่ละแหล่งสร้าง Candidate Pool จำนวน 10 รายการ*
5. **การหลอมรวมผลลัพธ์ (Hybrid Fusion):** *ระบบใช้ Weighted Reciprocal Rank Fusion รวมผลลัพธ์จาก 4 แหล่ง ได้แก่ Dense Retrieval, BM25 Word-level, BM25 Character 3-gram และ Graph Retrieval จากนั้นใช้ Graph-seeded Expansion เดินกราฟระยะ 1 Hop ตามความสัมพันธ์ REFERS_TO และ PENALIZED_BY โดยใช้ผลลัพธ์อันดับต้น 3 รายการเป็น Seed และเพิ่มมาตราที่เกี่ยวข้องได้ไม่เกิน 5 มาตรา*
6. **การจัดอันดับใหม่และกลไกป้องกันข้อมูลสำคัญสูญหาย (Reranking and Safety Nets):** *Cross-*

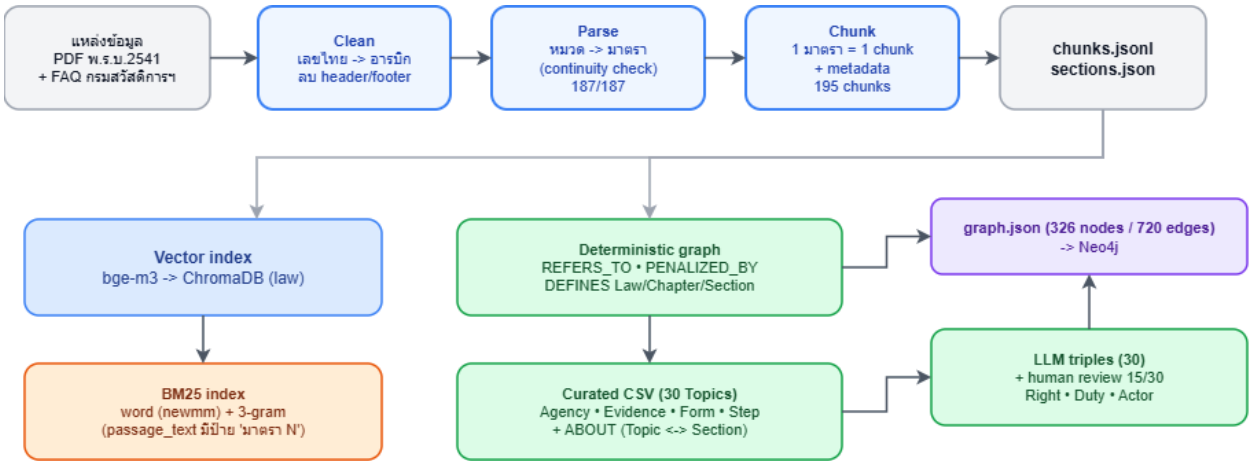
encoder BAAI/bge-reranker-v2-m3 จัดอันดับ Candidate ใหม่ จากนั้น complete_penalty_partners ใช้ดึงคู่มาตราที่เกี่ยวข้องกับบทลงโทษกลับมา และ ensure_graph_hits_survive ใช้รักษามาตราที่ Graph ยืนยันว่าเกี่ยวข้องไม่ให้ถูกตัดออกจากผลลัพธ์

7. **การคัดกรองระดับความเชื่อมั่น (Zone Gate):** *ระบบเปรียบเทียบคะแนน Rerank กับค่า TAU_ANSWER = 0.093 และ TAU_REJECT = 0.05 โดยแบ่งผลลัพธ์เป็น 3 ระดับ ได้แก่*
    - *คะแนนสูงกว่า TAU_ANSWER: ตอบโดยอ้างอิงจากตัวบทกฎหมาย*
    - *คะแนนอยู่ระหว่างสองค่า Threshold: ใช้ LLM ปรับแก้คำถามและค้นคืนข้อมูลซ้ำ 1 รอบ*
    - *คะแนนต่ำกว่า TAU_REJECT: ระบบจะส่งคำถามให้ LLM สร้างคำตอบจากความรู้ทั่วไป โดยไม่อ้างอิงเลขมาตราหรือแหล่งข้อมูลทางกฎหมาย และแสดงป้ายเตือน*
8. **การรวมบริบท (Context Aggregation):** *ระบบขยายข้อมูลเป็นเนื้อหามาตราแบบเต็มและสร้าง Section Card ซึ่งรวมข้อมูลจากมาตราและกราฟ เช่น บทลงโทษ หน่วยงาน หลักฐาน แบบฟอร์ม และขั้นตอน โดยจำกัดความยาวบริบทไม่เกิน 6,000 อักษรสำหรับ Local LLM และ 12,000 อักษรสำหรับ API LLM*
9. **การประมวลผลด้วย LLM:** *ระบบเรียกใช้งาน Local LLM ผ่าน Ollama หรือ API LLM ผ่าน dotBlue โดยใช้ Client Interface เดียวกัน และมี Fallback Chain สำหรับเปลี่ยนผู้ให้บริการเมื่อเกิดข้อผิดพลาด ได้คำตอบว่าง หรือได้คำตอบซ้ำผิดปกติ*
10. **การสร้างและส่งคำตอบ:** *ระบบแยกส่วน Citation Trailer ออกจากคำตอบ สร้าง Flex Message ซึ่งประกอบด้วยการ์ดคำตอบ ข้อมูลมาตรา และปุ่มสำหรับอ่านตัวบทต้นฉบับ จากนั้นบันทึก Trace และประวัติการทำงานลงในระบบจัดเก็บข้อมูลที่กำหนด*

สถาปัตยกรรมดังกล่าวช่วยให้ระบบสามารถใช้ประโยชน์จากการค้นคืนหลายรูปแบบร่วมกัน พร้อมเพิ่มกลไกตรวจสอบความเกี่ยวข้องของข้อมูลก่อนส่งบริบทให้ LLM ซึ่งช่วยสนับสนุนความถูกต้อง ความสามารถในการอ้างอิง และความปลอดภัยของคำตอบ

3.2 การจัดเตรียมและการประมวลผลข้อมูล (Data Pipeline & Knowledge Base Construction)

ระบบ ThaiLaw Assistant ใช้กระบวนการเตรียมข้อมูลแบบ Offline Data Pipeline เพื่อแปลงตัวบทกฎหมายและข้อมูลเชิงปฏิบัติให้เป็นฐานความรู้สำหรับการค้นคืน โดยผลลัพธ์ของกระบวนการประกอบด้วย Vector Index, BM25 Index และ Knowledge Graph ดังแสดงในรูปที่ 3.2



รูปที่ 3.2 Offline Data Pipeline ตั้งแต่การนำเข้าตัวบทกฎหมายจนถึงการสร้าง Vector Index, BM25 Index และ Knowledge Graph

3.2.1 การทำความสะอาดและปรับรูปแบบข้อมูล (Data Cleaning and Text Normalization)

ข้อมูลหลักประกอบด้วยตัวบทพระราชบัญญัติคุ้มครองแรงงาน พ.ศ. 2541 จากเอกสาร PDF โดยมีกระบวนการเตรียมข้อมูลดังตารางที่ 3.1

ตารางที่ 3.1 ขั้นตอนการเตรียมข้อมูล (Data Cleaning & Parsing)

ขั้นตอน	การดำเนินการ
การดึงข้อมูล	*ดึงข้อความจากเอกสาร PDF ด้วย pdfplumber และรวบรวมคู่มือจากแหล่งข้อมูลทางการด้วย Requests และ BeautifulSoup*
การทำความสะอาด	*แปลงเลขไทยเป็นเลขอารบิก ลบ Header, Footer, ข้อมูลหน้าเอกสาร ข้อความ about:blank และ Footnote ที่ไม่เกี่ยวข้อง รวมถึงตัดบล็อกประวัติการแก้ไขท้ายเอกสาร*
การแยกโครงสร้าง	*แยกข้อมูลตามลำดับ หมวดและมาตรา ตรวจสอบสถานะของมาตรา และตรวจสอบความต่อเนื่องของเลขมาตรา*
การแบ่งข้อมูล	*แบ่งตัวบทออกเป็น Chunk ตามขอบเขตของมาตรา โดยรายละเอียดแสดงในหัวข้อ 3.2.2*

การตรวจสอบโครงสร้างพบว่าสามารถแยกข้อมูลได้ครบ 187 มาตรา และตรวจพบรายการอ้างอิงข้ามมาตรา จำนวน 213 รายการ อย่างไรก็ตาม เอกสาร PDF มีปัญหา Glyph Corruption บางส่วน เช่น สระหรือวรรณยุกต์

อยู่ผิดตำแหน่ง ซึ่งเป็นปัญหาจากชั้นข้อความของเอกสารต้นฉบับ ไม่ใช่ข้อผิดพลาดของ Regular Expression ปัญหาดังกล่าวพบอย่างน้อย 3 จาก 195 Chunks และอาจส่งผลให้การค้นคืนด้วยคำสำคัญแบบตรงตัวไม่พบข้อมูลบางรายการ จึงมีการใช้ Topic Alias ในกราฟความรู้เพื่อช่วยรองรับรูปแบบคำที่แตกต่างกัน

3.2.2 การแบ่งข้อความ (Chunking Strategy)

โครงงานนี้กำหนดให้หนึ่งมาตราเป็นหนึ่ง Chunk เนื่องจากมาตราเป็นหน่วยอ้างอิงตามธรรมชาติของกฎหมาย และเป็นหน่วยที่ผู้ใช้สามารถตรวจสอบย้อนกลับได้โดยตรง การแบ่งข้อมูลตามจำนวนโทเคนแบบตายตัวอาจทำให้เนื้อหามาตราถูกตัดแยกจากกันและทำให้การอ้างอิงไม่สมบูรณ์

มาตราที่มีความยาวเกิน 1,600 อักษรจะถูกแบ่งย่อย โดยพยายามแบ่งตามขอบย่อหน้าและข้อย่อย เช่น (1), (2) และ (3) หากไม่สามารถแบ่งได้อย่างเหมาะสมจะเก็บเนื้อหาไว้เป็น Chunk เดียว ในชุดข้อมูลนี้มี 2 มาตราที่ถูกแบ่งย่อย ส่งผลให้จากตัวบทจำนวน 187 มาตรา ได้ข้อมูลทั้งหมด 195 Chunks

การแบ่งความยาวของ Chunk มีรายละเอียดดังนี้

ความยาว Chunk	**จำนวน**
ไม่เกิน 200 อักษร	*59*
201–500 อักษร	*80*
501–1,000 อักษร	*44*
1,001–1,600 อักษร	*10*
มากกว่า 1,600 อักษร	*2*

ความยาวของ Chunk มีค่าต่ำสุด 8 อักษร ค่าเฉลี่ย 415 อักษร ค่ามัธยฐาน 280 อักษร และค่าสูงสุด 3,213 อักษร ระบบไม่ใช้ Overlap เนื่องจากขอบเขตของ Chunk สอดคล้องกับขอบเขตของมาตรา ทั้งนี้ผลการเปรียบเทียบประสิทธิภาพของการแบ่งข้อมูลแสดงไว้ในบทที่ 4

ทุก Chunk จะถูกกำกับด้วย Metadata ได้แก่ chunk_id, law_id, law_name, chapter, section_no, section_key, status, source_url และ refs_out โดย Metadata เหล่านี้ใช้สำหรับการอ้างอิงคำตอบ การสร้างกราฟความรู้ และการประเมินผลระบบ

3.2.3 การสร้างดัชนีค้นคืนข้อมูล (Index Construction)

ระบบสร้างดัชนีค้นคืนข้อมูล 2 รูปแบบ ได้แก่ Vector Index และ BM25 Index ดังนี้

1. **Vector Index:** *นำข้อความของแต่ละ Chunk ไปสร้างเวกเตอร์ด้วยโมเดล BGE-M3 และจัดเก็บใน ChromaDB Collection ชื่อ law โดยใช้ดัชนี HNSW และ Cosine Similarity*
2. **BM25 Index:** *สร้างดัชนีจากข้อความของแต่ละ Passage โดยเพิ่มป้ายกำกับเลขมาตรา เช่น "มาตรา 61" ไว้ด้านหน้าข้อความก่อนทำดัชนี การเพิ่มป้ายกำกับดังกล่าวช่วยให้ระบบค้นคืนคำถามที่ระบุเลขมาตราได้ แม้เนื้อหาภายในมาตราจะไม่ได้กล่าวถึงเลขมาตราของตนเองซ้ำ*

3.2.4 การสกัดเอนทิตีและการสร้างกราฟความรู้ (Entity and Relationship Extraction)

กราฟความรู้สร้างจากข้อมูล 3 แหล่ง ซึ่งมีระดับความน่าเชื่อถือแตกต่างกัน ดังตารางที่ 3.2

ตารางที่ 3.2 แหล่งที่มาของกราฟความรู้

แหล่งข้อมูล	**วิธีการสร้าง**	**ผลลัพธ์**
Deterministic	*ใช้ Regular Expression จากตัวบทใน graph_core.py*	*Law, Chapter, Section รวมถึงความสัมพันธ์ HAS_CHAPTER, HAS_SECTION, REFERS_TO, PENALIZED_BY และ DEFINES*
Curated	*คัดเลือกและตรวจสอบข้อมูลจากไฟล์ data/curated/topic_agency_evidence_form_step.csv และเว็บไซต์กรมสวัสดิการและคุ้มครองแรงงาน*	*Topic, Agency, Evidence, Form, Step และความสัมพันธ์ด้านหน่วยงาน หลักฐาน แบบฟอร์ม ขั้นตอน และการเชื่อมโยง Topic กับ Section*
LLM Extraction และ Human Review	*ใช้แบบจำลองภาษาขนาดเล็กสกัด Triple ใน triples.py แล้วตรวจสอบโดยมนุษย์*	*Right, Duty, Actor และความสัมพันธ์ GRANTS_RIGHT, IMPOSES_DUTY, HELD_BY และ BINDS เฉพาะรายการที่ผ่านการตรวจสอบ*

ผลการตรวจสอบการสกัดข้อมูลด้วย LLM พบว่าผ่านการตรวจสอบ 15 จาก 30 รายการ หรือคิดเป็นร้อยละ 50 โดยความสัมพันธ์ GRANTS_RIGHT, IMPOSES_DUTY และ ABOUT มีความถูกต้องสูงกว่า ขณะที่ BINDS และ

HELD_BY มีปัญหาจากการจำแนกประเภทของ Subject ผิด ระบบจึงไม่นำความสัมพันธ์ที่ยังไม่ผ่านการตรวจสอบมาใช้โดยตรง แต่สร้างความสัมพันธ์ใหม่จากข้อมูลที่ผ่านการตรวจสอบแล้ว

หลังจากตรวจสอบและรวมข้อมูลจากทุกแหล่ง กราฟความรู้มีทั้งหมด 326 โหนด และ 720 เส้นเชื่อม โดยไม่มีโหนดที่แยกออกจากกราฟ ดังแสดงในตารางที่ 3.3

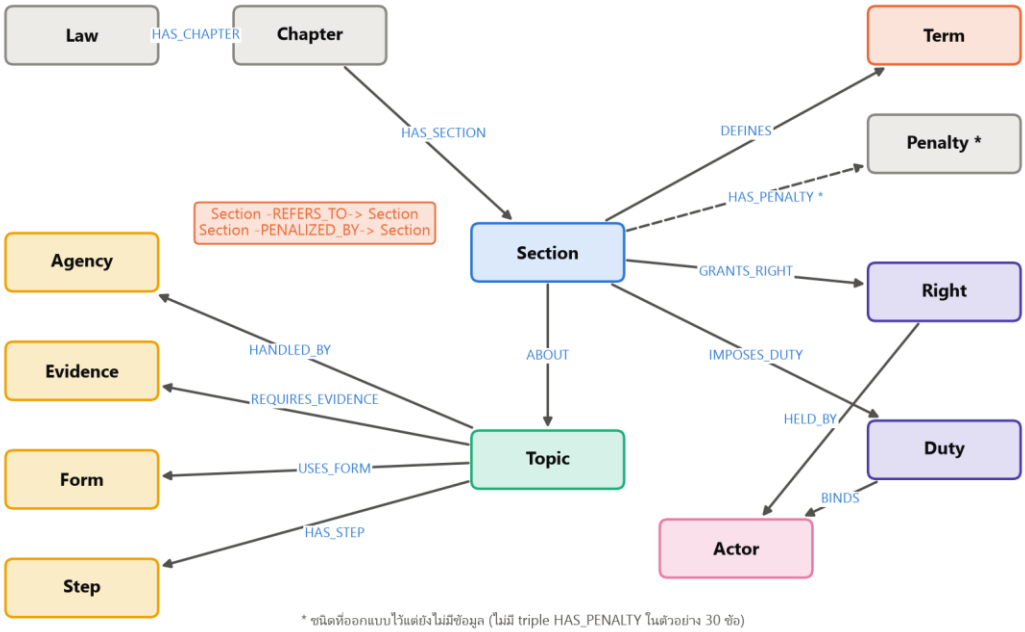
ตารางที่ 3.3 จำนวนโหนดและความสัมพันธ์ในกราฟความรู้

ประเภทโหนด	**จำนวน**	**ประเภทความสัมพันธ์**	**จำนวน**
Section	*188*	*REFERS_TO*	*211*
Topic	*30*	*HAS_SECTION*	*187*
Evidence	*24*	*PENALIZED_BY*	*98*
Step	*19*	*ABOUT*	*35*
Term	*23*	*HANDLED_BY, REQUIRES_EVIDENCE, USES_FORM, HAS_STEP*	*30 ต่อประเภท*
Chapter	*18*	*DEFINES*	*23*
Right	*7*	*HAS_CHAPTER*	*18*
Duty	*7*	*GRANTS_RIGHT, IMPOSES_DUTY, HELD_BY, BINDS*	*7 ต่อประเภท*
Actor	*4*		
Agency	*3*		
Form	*2*		
Law	*1*		
รวม 326 โหนด		**รวม 720 เส้นเชื่อม**	

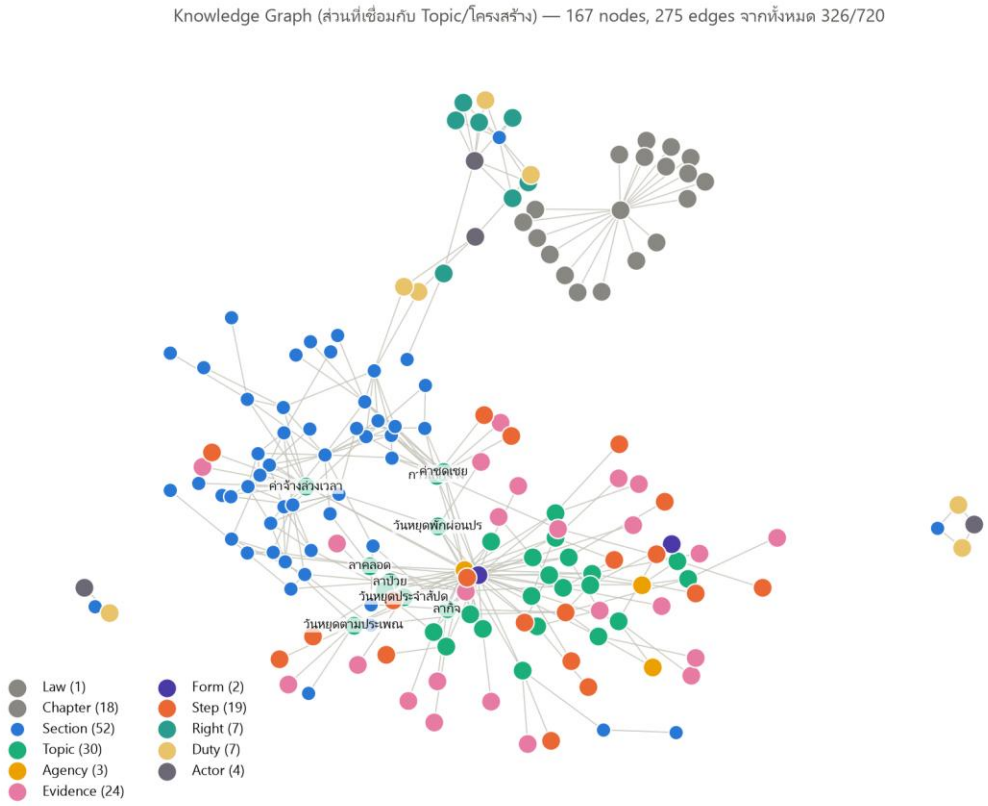
จำนวนโหนดประเภท Section เท่ากับ 188 เนื่องจากประกอบด้วยมาตราจริงจำนวน 187 มาตรา และโหนดที่เกิดจากการอ้างถึงมาตรา 583 ของกฎหมายอื่น นอกจากนี้ บางโหนดถูกสร้างเป็น Stub จากการอ้างอิงล่วงหน้า แม้ไม่มีคุณสมบัติข้อความหรือสถานะของมาตราในกราฟ แต่เนื้อหาสำหรับการตอบคำถามจะดึงจากไฟล์ sections.json จึงไม่กระทบต่อการค้นคืนเนื้อหาหลัก

การออกแบบกราฟความรู้มีแนวคิดสำคัญ 3 ประการ ได้แก่

1. *แบ่งระดับความน่าเชื่อถือของข้อมูล โดยข้อมูลที่สร้างจากกฎเชิงกำหนดและข้อมูลที่คัดเลือกจากแหล่งทางการมีความน่าเชื่อถือสูงกว่า ขณะที่ข้อมูลที่สกัดด้วย LLM ต้องผ่านการตรวจสอบโดยมนุษย์ก่อนนำไปใช้*
2. *ใช้ Topic เป็นตัวกลางในการเชื่อมโยงภาษาที่ผู้ใช้ใช้กับมาตราและขั้นตอนปฏิบัติ เช่น การเชื่อมคำว่า “ลากิจ” ไปยังมาตราที่เกี่ยวข้อง*
3. *ใช้ความสัมพันธ์ PENALIZED_BY และ REFERS_TO เป็นเส้นทางหลักในการค้นคืนข้อมูลแบบหลายขั้น (Multi-hop Retrieval)*



รูปที่ 3.3 Ontology ของกราฟความรู้กฎหมายแรงงาน



รูปที่ 3.4 กราฟความรู้จริงจากไฟล์ data/graph.json แสดงโครงสร้าง Law ⟶ Chapter ⟶ Section และการเชื่อมโยง Topic กับ Section ผ่านความสัมพันธ์ ABOUT

3.3 การพัฒนาระบบค้นคืนข้อมูล (Retrieval Engine Implementation)

3.3.1 Dense Retrieval Pipeline

กระบวนการ Dense Retrieval ใน Retriever.search_raw() ประกอบด้วยการเข้ารหัสคำถามด้วย BGE-M3 การค้นคืนจาก ChromaDB และ BM25 แบบ Word-level กับ Character 3-gram อย่างละ 10 รายการ จากนั้นจึงหลอมรวมผลลัพธ์ด้วย RRF ในโหมด Dense ของ Engine และคัดเลือกจำนวน Candidate ด้วย dynamic_k() ตามความยาวและความซับซ้อนของคำถาม โดยคำถามสั้นจะจัดอันดับใหม่ 3 จาก 4 รายการ คำถามยาวหรือมีคำเชื่อมจะจัดอันดับ 8 จาก 10 รายการ และคำถามทั่วไปจะจัดอันดับ 5 จาก 6 รายการ ขั้นตอนสุดท้ายใช้ Cross-encoder บน CPU และใช้คะแนนอันดับหนึ่งเป็นสัญญาณสำหรับ Zone Gate ทั้งนี้ Dense-only ที่ไม่ใช้ BM25 เป็น Configuration แยกในการทดลอง Ablation

ระบบใช้ Threshold จากคะแนน Rerank ซึ่งอยู่ในช่วง 0–1 หลังผ่าน Sigmoid แทนการใช้ Cosine Similarity ดิบ เนื่องจากผลการทดลองในหัวข้อ 4.2.3 แสดงว่า Absolute Threshold บน Cosine ดิบไม่เหมาะกับโดเมนกฎหมายไทย

3.3.2 Graph Retrieval Pipeline

กระบวนการประกอบด้วยขั้นตอนสำคัญดังต่อไปนี้

1. **Entity Linking:** *ตรวจจับเลขมาตราด้วย Regular Expression ซึ่งรองรับเลขไทยและรูปแบบ เช่น มาตรา 57/1 รวมถึงจับ Topic Alias จากคอลัมน์ aliases ใน Curated CSV และตรวจจับคำถามเกี่ยวกับคำนิยาม*
2. **Query-type Inference:** *จำแนกคำถามเป็น Lookup, Procedure, Penalty, Definition หรือ Aggregation เพื่อเลือกแม่แบบ Cypher ที่เหมาะสม*
3. **Graph Traversal:** *ใช้แม่แบบ Cypher แบบ Parameterized โดยไม่ต่อข้อความจากอินพุตของผู้ใช้โดยตรง เพื่อลดความเสี่ยงจากการโจมตีแบบ Injection*

ตารางที่ 3.4 แม่แบบ Cypher ตามประเภทคำถาม

ชนิด	Pattern
lookup	*(s:Section) WHERE s.section_no IN $nos + OPTIONAL (s)-[:REFERS_TO|PENALIZED_BY]->(related)*
procedure	*(t:Topic)<-[:ABOUT]-(s:Section) + (t)-[:HANDLED_BY|REQUIRES_EVIDENCE|USES_FORM|HAS_STEP*1..2]-(n)*
penalty	*(s:Section)-[:PENALIZED_BY]->(p:Section)*
definition	*(s:Section)-[:DEFINES]->(t:Term)*
aggregation	*(t:Topic)<-[:ABOUT]-(s:Section)-[:GRANTS_RIGHT]->(r:Right)*

4. **In-memory Mode:** *ระบบโหลด data/graph.json เข้าหน่วยความจำและใช้เป็นเส้นทางหลักของ Graph Retrieval ทั้งในการประเมินและการให้บริการผ่าน LINE*
   โดยเดินกราฟด้วยตรรกะเดียวกับแม่แบบ Cypher ข้างต้น (โค้ดรองรับการเรียกผ่าน Neo4j Driver แต่ Engine ไม่ได้ใช้) ทำให้ Graph Retrieval ยังทำงานได้โดยไม่ต้องพึ่งพา Neo4j ในขณะรันระบบ ส่วน Neo4j ใช้สำหรับจัดเก็บประวัติสนทนาและสำรวจกราฟตามการตั้งค่าของระบบ
5. **Aggregation Fallback:** *สำหรับคำถามรวบยอด เช่น “ลาประเภทใดบ้าง” ระบบจะขยายไปยัง Topic ที่มี Root เดียวกัน เช่น “ลา” และ “วันหยุด” พร้อมเพิ่มค่า Top-K เป็นอย่างน้อย 20 รายการ เพื่อป้องกันการตัดข้อมูลที่เกี่ยวข้องออกเร็วเกินไป*

3.3.3 Hybrid Fusion Engine

1. **Query Router:** *ใน router.py ระบบจำแนกคำถามตามลำดับความสำคัญ ได้แก่ คำทักทายซึ่งส่งตรงไปยัง LLM คำถามที่มีเลขมาตราซึ่งจัดเป็น Lookup คำถามคำนิยามซึ่งจัดเป็น Definition คำถามเกี่ยวกับบทลงโทษซึ่งจัดเป็น Penalty คำถามเกี่ยวกับขั้นตอนซึ่งจัดเป็น Procedure และคำถามที่เหลือซึ่งจัดเป็น General จากนั้นจึงเลือกน้ำหนักของ Retriever ตามตารางที่ 3.5*

ตารางที่ 3.5 ตารางน้ำหนักของ Query Router

ประเภทคำถาม	α_dense	α_bm25	α_graph	เหตุผล
Lookup	*0.2*	*0.4*	*0.4*	*BM25 และ Graph เหมาะกับการค้นคืนเลขมาตรา*
Definition	*0.3*	*0.2*	*0.5*	*นิยามอยู่ใน Node Term ผ่าน DEFINES*
Penalty / Procedure / Multi-hop	*0.3*	*0.1*	*0.6*	*ต้องใช้ความสัมพันธ์ในกราฟ*
General	*0.6*	*0.3*	*0.1*	*คำถามมีถ้อยคำกว้าง จึงให้น้ำหนัก Dense สูงกว่า*

ในปัจจุบันกฎใน _classify_type ยังไม่คืนค่า Multi-hop โดยคำถามที่ต้องใช้หลายมาตราส่วนใหญ่จะถูกจัดเป็น Penalty, Procedure หรือ General น้ำหนักของ Multi-hop ที่แสดงในตารางจึงเป็นการเตรียมไว้สำหรับการจำแนกด้วย LLM ในอนาคต

2. **Weighted RRF:** *คำนวณคะแนนรวมของแต่ละ Chunk จาก*

$$\text{Score}(d) = \sum_{r} \frac{w_r}{60 + \text{rank}_r(d)}$$

โดยรวมผลลัพธ์จาก Dense, BM25 Word-level, BM25 Character 3-gram และ Graph การกำหนดน้ำหนัก **α_bm25 ของ BM25** *จะแบ่งให้ Word-level และ Character 3-gram และใช้ลำดับแทนคะแนนดิบเพื่อหลีกเลี่ยงปัญหาช่วงคะแนนของแต่ละ Retriever แตกต่างกัน*

3. **Graph-seeded Expansion:** *ใช้ผลลัพธ์ 3 อันดับแรกเป็น Seed แล้วเดินกราฟระยะ 1 Hop ตามความสัมพันธ์ REFERS_TO และ PENALIZED_BY เพื่อเพิ่ม Candidate ได้ไม่เกิน 5 รายการ พร้อมบันทึก graph_paths สำหรับการตรวจสอบผ่านโหมด /debug*

4. **Post-rerank Safety Nets:** *เป็นกลไกหลังการจัดอันดับใหม่ ซึ่งพัฒนาจากการวิเคราะห์ข้อผิดพลาดในหัวข้อ 4.4 ประกอบด้วย*

- *complete_penalty_partners ใช้ดึงคู่มาตราที่เกี่ยวข้องกับบทลงโทษกลับมาจาก Candidate Pool เดิม เมื่อมาตราเนื้อหาถูกคงไว้แต่มาตราคู่บทลงโทษถูกตัดออก*
- *ensure_graph_hits_survive ใช้รักษามาตราที่ Graph ยืนยันจากเลขมาตราหรือ Alias ไม่ให้ถูกตัดออกหลังการ Rerank และปรับสัญญาณสำหรับ Zone Gate ตามเงื่อนไขที่กำหนด เพื่อไม่ให้ผลลัพธ์ที่ Graph ยืนยันถูกส่งไปเป็นคำตอบความรู้ทั่วไปโดยไม่จำเป็น*

3.4 การบูรณาการโมเดลภาษาและการบำรุงรักษาระบบ (LLM Integration & Infrastructure)

3.4.1 การออกแบบ Prompt Engineering สำหรับ Context Injection

พรอมต์หลักใน generator.SYSTEM_PROMPT กำหนดให้ระบบตอบจากข้อมูลอ้างอิงเท่านั้น หากบริบทไม่เพียงพอให้แจ้งว่าไม่พบข้อมูลในตัวบทกฎหมาย ห้ามสร้างเลขมาตรา วันที่ หรือชื่อหน่วยงานขึ้นเอง และต้องปิดท้ายด้วยบรรทัดอ้างอิงรูปแบบที่กำหนด เช่น “ใช้ข้อมูลจาก: [มาตรา X], [มาตรา Y]” เพื่อให้โปรแกรม parse_citation แยก Citation ได้อย่างสม่ำเสมอ โดยบรรทัดดังกล่าวจะถูกตัดออกจากข้อความที่แสดงแก่ผู้ใช้ เนื่องจากข้อมูลซ้ำกับการ์ด Flex Message

บริบทที่ส่งให้ LLM จัดรูปแบบเป็น Section Card รายมาตรา ประกอบด้วยหัวมาตรา สถานะ ลิงก์แหล่งที่มา ตัวบท บทลงโทษ นิยาม และข้อมูลจาก Topic เช่น หน่วยงาน หลักฐาน แบบฟอร์ม และขั้นตอน ระบบควบคุมความยาวบริบทโดยไม่ตัดกลางการ์ด และกำหนดงบประมาณประมาณ 6,000 อักษรสำหรับ Local LLM และ 12,000 อักษรสำหรับ API LLM

สำหรับคำถามนอกขอบเขต ระบบใช้ GENERAL_KNOWLEDGE_SYSTEM_PROMPT เพื่อให้ LLM ตอบด้วยความรู้ทั่วไปโดยไม่อ้างเลขมาตราหรือแหล่งข้อมูลกฎหมาย และโค้ดจะเพิ่มป้ายเตือนว่าเป็นความรู้ทั่วไปของ AI ให้กับคำตอบทุกครั้ง

3.4.2 การเชื่อมต่อ Local LLM และ API LLM (Error Handling & Fallback Mechanisms)

ไฟล์ src/llm/client.py กำหนด Interface กลาง .chat(messages) -> (text, usage) เพื่อให้ระบบเรียกใช้ Local LLM และ API LLM ด้วยรูปแบบเดียวกัน

ตารางที่ 3.6 การเปรียบเทียบ LLM Client

มิติ	OllamaClient (Local)	DotBlueClient (API)
ไลบรารี	*Ollama Native โดยใช้ think=False ตามการตั้งค่าของระบบ*	*OpenAI SDK ผ่าน Endpoint ของ PSU AI*
ข้อควรระวัง	*ต้องกำหนด num_ctx และ num_predict ให้เหมาะสม*	*DotBlue ส่งข้อมูลแบบ SSE จึงต้องประมวลผล Stream และรวม Delta*
Retry	*ใช้ Fallback ตามจุดเรียกใช้งาน*	*ใช้ Tenacity สูงสุด 3 ครั้งแบบ Exponential Backoff*
Token Usage	*prompt_eval_count และ eval_count*	*prompt_tokens และ completion_tokens*

ตารางที่ 3.7 กลไก Error Handling และ Fallback

ปัญหา	กลไกจัดการ	แหล่งที่เกี่ยวข้อง
LLM ล่มหรือเกิด Exception	*เปลี่ยนผู้ให้บริการระหว่าง API และ Local พร้อมบันทึก provider_fallback*	*_generate_with_fallback*
คำตอบว่าง	*ตรวจด้วย _is_empty_body แล้วเรียกผู้ให้บริการอื่นซ้ำ พร้อมเพิ่ม NUM_PREDICT จาก 512 เป็น 2,500*	*หัวข้อ 4.4.6*
คำตอบซ้ำหรือวนซ้ำ	*ใช้ _is_degenerate_body ตรวจสัดส่วนบรรทัดซ้ำ แล้วเรียกใช้ Fallback*	*การทดสอบบน LINE*
ผู้ให้บริการทั้งสอง	*ส่งข้อความปฏิเสธแบบสำเร็จรูปโดยไม่แสดง*	*_general_knowledge_with_fallback*

ปัญหา	กลไกจัดการ	แหล่งที่เกี่ยวข้อง
รายไม่พร้อมใช้งาน	*รายละเอียด Error*	
Graph Snapshot สูญหายหรือเสียหาย	*เปลี่ยน Graph Retriever เป็น None และลดรูปการค้นคืนเป็น Dense + BM25*	*_load_graph_retriever*
Neo4j ไม่พร้อมใช้งาน	*ใช้ Graph Snapshot แบบ Offline และครอบการจัดการประวัติด้วย try/except*	*Offline Design*
Judge SSE เสียหาย	*เรียกซ้ำไม่เกิน 3 ครั้งต่อรายการ*	*judge.py*
Race Condition ของประวัติสนทนา	*ใช้ threading.Lock และคืนสำเนาของประวัติ*	*engine._get_history*
Webhook ไม่ผ่านการยืนยัน	*ตรวจสอบ X-Line-Signature ก่อนประมวลผล*	*app_line.py*

ระบบมี Observability ผ่าน tracing.log_trace ซึ่งบันทึกคำถาม คะแนน Zone เวลาในแต่ละขั้นตอน ได้แก่ Retrieve, Graph Expansion, Rerank และ Generate รวมถึงจำนวน Prompt และ Completion Token จากการเรียก LLM ทั้งหมดลงใน data/traces.jsonl ซึ่งมีข้อมูลจากการใช้งานและการทดสอบจริงมากกว่า 2,400 บรรทัด

โครงสร้างโค้ดแยกเป็นโมดูล ingest/, index/, retrieval/, llm/ และ app/ พร้อม Unit Test จำนวน 133 กรณี ครอบคลุม Parser, Chunker, Graph Construction, Dense/BM25/Graph Retrieval, Fusion, Rerank, Zone, Concurrency, Safety Nets, Citation, Flex Message, Tracing, LINE Commands และ Token Accounting

บทที่ 4: การทดลอง ผลการทดสอบ และการวิเคราะห์ผล (Experiments, Results & Discussion)

4.1 สภาพแวดล้อมและระเบียบวิธีการทดลอง (Experimental Setup & Benchmark)

4.1.1 สภาพแวดล้อมฮาร์ดแวร์และซอฟต์แวร์ (Hardware and Software Specification)

ตารางที่ 4.1 สภาพแวดล้อมที่ใช้ทดสอบ

รายการ	**รายละเอียด**
เครื่องทดสอบ	*โน้ตบุ๊กระบบปฏิบัติการ Windows 11, RAM 16 GB และ GPU NVIDIA GeForce RTX 3050 Laptop GPU ขนาด 6 GB VRAM*
Embedding	*BAAI/bge-m3 โดยประมวลผลจริงบน CPU เนื่องจาก PyTorch 2.14.0+cpu เป็นรุ่น CPU-only และใช้ ChromaDB จำนวน 1 collection รวม 195 chunks*
Reranker	*BAAI/bge-reranker-v2-m3 ประมวลผลบน CPU เพื่อสงวน VRAM สำหรับ Local LLM และเป็นคอขวดสำคัญของเวลาในการตอบสนอง*
Graph	*Graph snapshot ในไฟล์ data/graph.json มี 326 nodes และ 720 edges สำหรับ Offline Retrieval พร้อม Neo4j 5.24 Community ผ่าน Docker*
Local LLM	*Ollama ได้แก่ qwen3.5:4b และ gemma3:4b โดยใช้การควอนไทซ์ Q4, think=False และ num_ctx=8192*
API LLM	*dotBlue ได้แก่ qwen/qwen3.6-flash, gpt-4o-mini และ deepseek-chat*
ตัวประเมิน	*qwen/qwen3.6-plus ผ่าน dotBlue โดยแยกจากโมเดลที่นำมาเปรียบเทียบเป็นส่วนใหญ่*
พารามิเตอร์ร่วม	*temperature=0.2, NUM_PREDICT=2500, POOL=10, RRF c=60, TAU_ANSWER=0.093, TAU_REJECT=0.05 และงบประมาณบริบท 6,000 อักษรสำหรับ Local LLM และ 12,000 อักษรสำหรับ API LLM*

4.1.2 ชุดข้อมูลทดสอบและคำตอบอ้างอิง (Test Dataset & Ground Truth Questions)

ชุดทดสอบ eval/testset.jsonl ประกอบด้วยคำถามจำนวน 50 ข้อ แบ่งให้สมาชิกทั้งสองคนจัดทำคนละ 25 ข้อ โดยหลีกเลี่ยงการใช้มาตราซ้ำกันและตรวจสอบไขว้ระหว่างผู้จัดทำ ชุดทดสอบครอบคลุมคำถาม 7 ประเภท เพื่อประเมินจุดแข็งและข้อจำกัดของวิธีค้นคืนแต่ละรูปแบบ ดังแสดงในตารางที่ 4.2

ตารางที่ 4.2 องค์ประกอบของชุดทดสอบ

ประเภท	จำนวน	วัตถุประสงค์	ตัวอย่างคำถาม
lookup	*8*	*ค้นหามาตราที่ระบุโดยตรง ซึ่ง BM25 และ Graph Retrieval คาดว่าจะช่วยได้*	*“มาตรา 61 กำหนดเรื่องใด”*
definition	*6*	*ค้นหานิยามของคำศัพท์หรือเอนทิตีในกราฟ*	*“ลูกจ้างคือใคร”*
single_hop	*10*	*ใช้มาตราเดียวในการตอบคำถามที่ใช้ภาษาทั่วไป*	*“ลูกจ้างมีสิทธิลากิจหรือไม่”*
multi_hop	*10*	*เชื่อมโยงอย่างน้อย 2 มาตรา เช่น มาตราเนื้อหากับมาตราบทลงโทษ*	*“ฝ่าฝืนเรื่องเวลาทำงานมีโทษอย่างไร”*
procedure	*8*	*ค้นหาขั้นตอน หน่วยงาน หรือหลักฐานที่เกี่ยวข้อง*	*“ต้องใช้หลักฐานอะไรเมื่อร้องเรียนค่าจ้างค้างจ่าย”*
aggregation	*4*	*รวบรวมข้อมูลจากหลายมาตราภายใต้หัวข้อเดียวกัน*	*“ลูกจ้างมีสิทธิการลาประเภทใดบ้าง”*
out_of_scope	*4*	*คำถามนอกขอบเขต พ.ร.บ.คุ้มครองแรงงาน ซึ่งต้องระบุว่าเป็นความรู้ทั่วไปและไม่สร้างเลขมาตรา*	*คำถามที่ไม่เกี่ยวข้องกับ พ.ร.บ.คุ้มครองแรงงาน*
รวม	*50*		

แต่ละคำถามมีคำตอบอ้างอิง (Ground Truth) ประกอบด้วย gold_sections ซึ่งระบุ chunk ของมาตราที่ควรค้นคืน โดยมีค่าเฉลี่ย 1.43 chunk ต่อคำถามในกลุ่ม In-scope จำนวน 46 ข้อ นอกจากนี้ยังประกอบด้วย key_points, gold_agency และ expected_query_type

คำถามประเภท Out-of-scope ไม่มี gold section จึงไม่นำมาคำนวณ Retrieval Metrics โดยรายงานผลทั้ง n=50 สำหรับภาพรวม และ n=46 สำหรับการวิเคราะห์เฉพาะคำถามในขอบเขตและการทดสอบนัยสำคัญ

4.1.3 รูปแบบการทดลอง (Experimental Configurations)

ก) การทดลอง Retrieval Ablation (eval/run_retrieval.py --production) ใช้ Backend จริง ได้แก่ BGE-M3, ChromaDB, Cross-encoder Reranker และ GraphRetriever เดียวกับระบบ Production จากนั้นสร้าง Configuration จากองค์ประกอบย่อย เพื่อวิเคราะห์ผลของการเพิ่มแต่ละชั้น ดังแสดงในตารางที่ 4.3

ตารางที่ 4.3 นิยาม Configuration ของ Ablation

Config	องค์ประกอบ
D	*Dense Retrieval เท่านั้น (BGE-M3 ⟶ ChromaDB) โดยไม่ใช้ BM25*
D+R	*D และ Cross-encoder Rerank*
G	*Graph Retrieval เท่านั้น (GraphRetriever: entity linking + ABOUT/REFERS_TO/PENALIZED_BY)*
H1	*นำผล Dense และ Graph มาต่อกันโดยตรง (Concatenation) โดยไม่ใช้ Fusion เป็น Baseline ระดับพื้นฐาน*
H2	*RRF ระหว่าง Dense และ Graph โดยใช้น้ำหนักเท่ากัน*
H3	*H2 และ Graph-seeded Expansion โดยใน Harness นี้ไม่ได้ฉีด Edge เพิ่ม จึงไม่คาดว่าจะมีผลเพิ่ม*
H4	*Weighted RRF โดยกำหนดน้ำหนัก Dense:Graph เท่ากับ 0.4:0.6 และไม่ใช้ Rerank*
H5	*H4 และ Rerank*
H6	*H5 และ Penalty-partner โดยจำลองการทำงานของ complete_penalty_partners*

ข) การประเมิน Engine เรียกใช้ RAGEngine เดียวกับระบบ LINE ใน 3 โหมด ได้แก่ Dense ซึ่งใช้ Dense และ BM25 ต่อด้วย Rerank, Graph ซึ่งใช้ Graph Retrieval ต่อด้วย Rerank และ Safety Nets และ Hybrid ซึ่งใช้

Router-weighted 4-way RRF ต่อด้วย Expansion, Rerank และ Safety Nets เพื่อให้ผลสามารถเปรียบเทียบกับการสร้างคำตอบได้โดยตรง

ค) การประเมิน Generation ใช้ RAGEngine.answer_with_debug() ครบทั้ง Pipeline กับคำถาม 50 ข้อต่อ Configuration และสร้าง Session ID ใหม่สำหรับแต่ละข้อ เพื่อป้องกันประวัติสนทนาปะปน การทดลองประกอบด้วยการเปรียบเทียบ LLM 5 รุ่นบน Hybrid การเปรียบเทียบโหมด Dense, Graph และ Hybrid การทดลอง No-RAG และ Full-context รวมถึงการทดลอง num_ctx sweep ที่ 2,048, 4,096 และ 8,192 บน Local LLM

4.1.4 ดัชนีชี้วัดประสิทธิภาพ (Evaluation Metrics)

ตารางที่ 4.4 ดัชนีชี้วัดที่ใช้

กลุ่ม	ดัชนี	ความหมาย
Retrieval	*Recall@5, Hit@1, MRR, nDCG@5*	*วัดจำนวนมาตราเฉลยที่ค้นพบ ความถูกต้องของอันดับแรก อันดับของมาตราที่ถูกต้องตัวแรก และคุณภาพการจัดอันดับ*
Generation (LLM-as-a-Judge, 1–5)	*Correctness, Faithfulness, Citation Accuracy, Thai Clarity*	*ผู้ประเมินให้คะแนนโดยเปรียบเทียบกับ key_points ของคำถามนั้น*
Generation (0–1)	*Key-point Coverage; OOS handled*	*วัดสัดส่วนประเด็นสำคัญที่ครอบคลุม และตรวจว่าคำถามนอกขอบเขตมีป้ายความรู้ทั่วไปของ AI และไม่อ้างเลขมาตราที่ไม่มีหลักฐาน*
Citation (อัตโนมัติ)	*Citation Precision / Recall*	*เปรียบเทียบเลขมาตราที่อ้างในคำตอบกับ gold_sections โดยใช้ Regular Expression และไม่ใช้ LLM ในการคำนวณ*
ประสิทธิภาพระบบ	*Latency, Token, Token cost, VRAM/RAM สูงสุด*	*วัดเวลาแฝง จำนวน Prompt/Completion Token ต้นทุน Token และทรัพยากรสูงสุดจากการรันจริงใน eval/results/generation_*.csv*

ข้อควรระวังเชิงระเบียบวิธีมี 3 ประการ ได้แก่ ผลของ LLM ไม่เป็น Deterministic และแต่ละ Configuration รันเพียง 1 ครั้ง จึงควรตีความเป็นแนวโน้มมากกว่าค่าที่มีช่วงความเชื่อมั่น การประเมินคุณภาพคำตอบใช้ LLM-as-a-Judge ซึ่งยังไม่ได้ตรวจสอบเทียบกับผู้ประเมินที่เป็นมนุษย์ จึงใช้เพื่อเปรียบเทียบสัมพัทธ์ระหว่าง Configuration เป็นหลัก และขนาดตัวอย่างในแต่ละประเภทมีเพียง 4–10 ข้อ จึงอาจมีความแปรปรวนสูงและควรพิจารณาร่วมกับผลการทดสอบนัยสำคัญ ทั้งนี้ Correctness และ Key-point Coverage ใช้ประเมินความตรงประเด็นของคำตอบ (สอดคล้องกับ Answer Relevancy) ส่วน Citation Precision/Recall ใช้วัด Precision/Recall ของการอ้างอิง

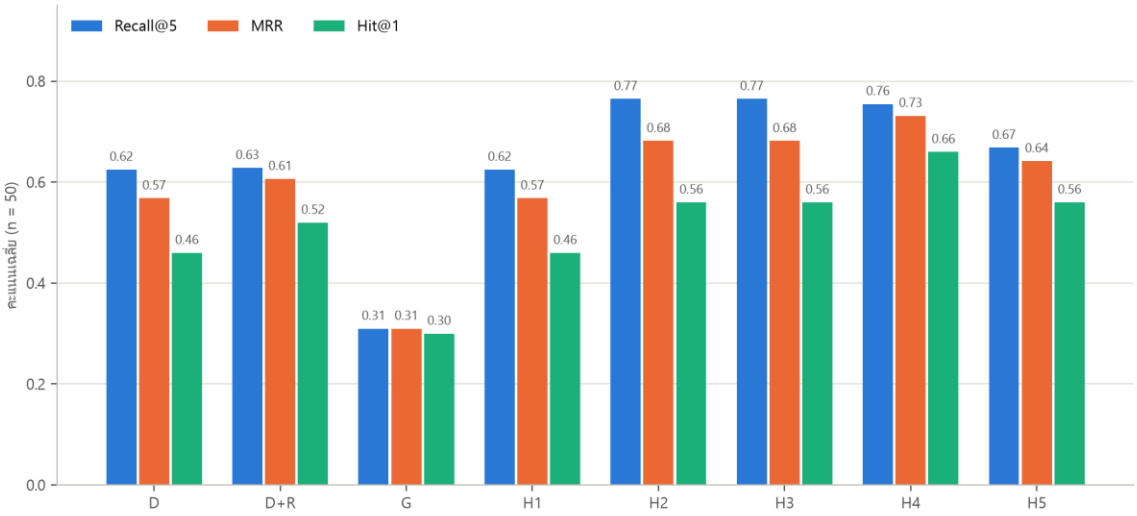
4.2 ผลการทดสอบเปรียบเทียบประสิทธิภาพการค้นคืน (Retrieval Performance Comparison)

4.2.1 ภาพรวม Dense vs Graph vs Hybrid (Ablation)

ตารางที่ 4.5 ผลการค้นคืนของแต่ละ Configuration

Configuration	Recall@5 (n=50)	MRR (n=50)	nDCG@5 (n=50)	Hit@1 (n=50)	Recall@5 (In-scope n=46)	Hit@1 (In-scope n=46)
D (Dense เท่านั้น)	*0.625*	*0.568*	*0.553*	*0.460*	*0.679*	*0.500*
D+R (Dense + Rerank)	*0.628*	*0.607*	*0.583*	*0.520*	*0.683*	*0.565*
G (Graph เท่านั้น)	*0.310*	*0.310*	*0.306*	*0.300*	*0.337*	*0.326*
H1 (Concat Dense+Graph)	*0.625*	*0.568*	*0.553*	*0.460*	*0.679*	*0.500*
H2 (+ RRF)	*0.765*	*0.682*	*0.676*	*0.560*	*0.832*	*0.609*
H3 (+ Graph-seeded Expansion)	*0.765*	*0.682*	*0.676*	*0.560*	*0.832*	*0.609*

Configuration	Recall@5 (n=50)	MRR (n=50)	nDCG@5 (n=50)	Hit@1 (n=50)	Recall@5 (In-scope n=46)	Hit@1 (In-scope n=46)
H4 (+ Router weight, ไม่ Rerank)	*0.755*	*0.732*	*0.710*	*0.660*	*0.821*	*0.717*
H5 (H4 + Rerank)	*0.668*	*0.642*	*0.621*	*0.560*	*0.726*	*0.609*
H6 (H5 + Penalty-partner)	*0.668*	*0.641*	*0.620*	*0.560*	*0.726*	*0.609*



รูปที่ 4.1 ผลการค้นคืนเฉลี่ยของแต่ละ Configuration

ข้อสังเกตสำคัญมีดังนี้

1. *Hybrid RAG ให้ผลดีกว่า Dense RAG เมื่อใช้กลไก Fusion ที่เหมาะสม โดย H4 ซึ่งใช้ Weighted RRF โดยไม่ผ่าน Rerank ให้ Recall@5 เท่ากับ 0.821 และ Hit@1 เท่ากับ 0.717 เทียบกับ Dense ที่ 0.679 และ 0.500 ตามลำดับ (เฉพาะคำถามในขอบเขต n=46) ความแตกต่างของ Hit@1 มีนัยสำคัญทางสถิติ โดยเพิ่มขึ้น 0.217, Bootstrap 95% CI เท่ากับ [0.109, 0.348] และมีค่า p=0.002*
2. *การนำรายการผลลัพธ์มาต่อกันโดยไม่ใช้กลไก Fusion ไม่ช่วยเพิ่มประสิทธิภาพ โดย H1 ให้ผลเท่ากับ*

Dense เนื่องจากรายการ Dense อยู่ด้านหน้าและผลลัพธ์ของ Graph ถูกตัดออกเมื่อจำกัดที่ Top-5 การใช้ RRF ใน H2 ช่วยเพิ่ม Recall@5 จาก 0.625 เป็น 0.765 หรือเพิ่มขึ้น 0.140 แสดงให้เห็นว่าการมีทั้ง Vector และ Graph ยังไม่เพียงพอ หากไม่มีกลไก Fusion ที่เหมาะสม

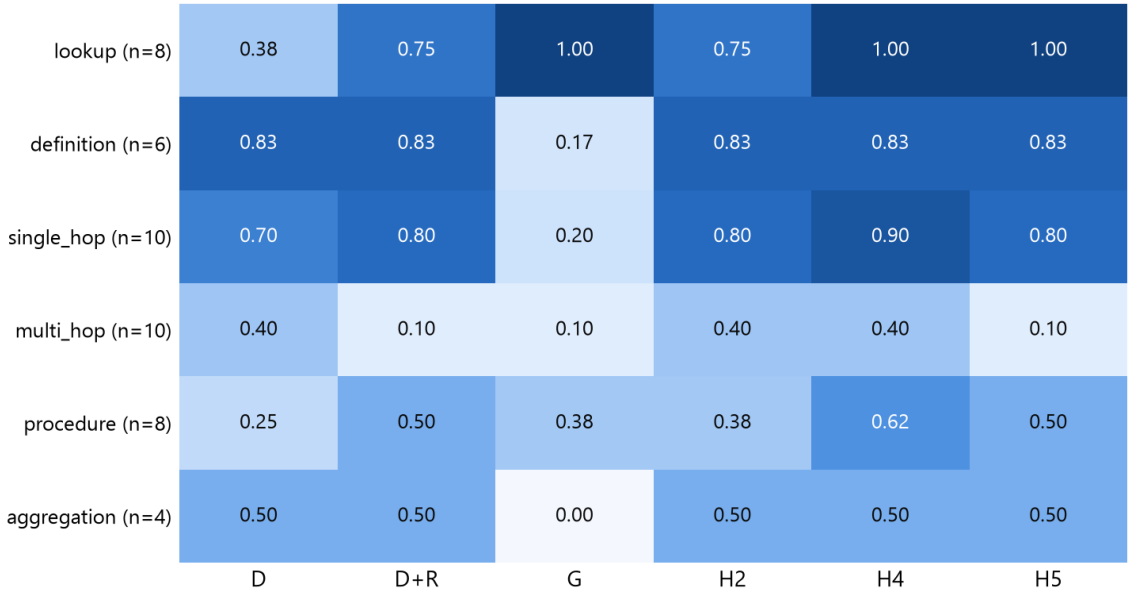
3. *Graph Retrieval เพียงอย่างเดียวมีข้อจำกัด โดยมี Recall@5 เท่ากับ 0.337 ซึ่งต่ำกว่า Dense ที่ 0.679 อย่างมีนัยสำคัญทางสถิติ (n=46) (p=0.006) อย่างไรก็ตาม Graph ยังช่วยค้นคืนข้อมูลที่ Dense ไม่พบในคำถามบางประเภท จึงควรใช้เป็นองค์ประกอบร่วมกับ Retriever อื่น*
4. *Rerank ไม่ได้ช่วยเพิ่มประสิทธิภาพของ Hybrid ในชุด Ablation โดย H5 มี Hit@1 เท่ากับ 0.609 ต่ำกว่า H4 ที่ 0.717 (n=46) แม้ความแตกต่างยังไม่มีนัยสำคัญทางสถิติ (p=0.166) สาเหตุเชิงเทคนิคอภิปรายในหัวข้อ 4.4.2*

4.2.2 ผลแยกตามประเภทคำถามและการทดสอบนัยสำคัญ

ตารางที่ 4.6 Hit@1 แยกตามประเภทคำถาม

ประเภท	D	D+R	G	H1	H2	H3	H4	H5	H6
lookup (n=8)	*0.38*	*0.75*	*1.00*	*0.38*	*0.75*	*0.75*	*1.00*	*1.00*	*1.00*
definition (n=6)	*0.83*	*0.83*	*0.17*	*0.83*	*0.83*	*0.83*	*0.83*	*0.83*	*0.83*
single_hop (n=10)	*0.70*	*0.80*	*0.20*	*0.70*	*0.80*	*0.80*	*0.90*	*0.80*	*0.80*
multi_hop (n=10)	*0.40*	*0.10*	*0.10*	*0.40*	*0.40*	*0.40*	*0.40*	*0.10*	*0.10*
procedure (n=8)	*0.25*	*0.50*	*0.38*	*0.25*	*0.38*	*0.38*	*0.62*	*0.50*	*0.50*
aggregation (n=4)	*0.50*	*0.50*	*0.00*	*0.50*	*0.50*	*0.50*	*0.50*	*0.50*	*0.50*

หมายเหตุ: คำถามประเภท out_of_scope ไม่มี gold section จึงไม่รวมในการคำนวณ



รูปที่ 4.2 Heatmap ของ Hit@1 แยกตามประเภทคำถามและ Configuration

ตารางที่ 4.7 การทดสอบนัยสำคัญแบบจับคู่

เปรียบเทียบ	Δ Recall@5 [95% CI]	p	Δ Hit@1 [95% CI]	p	W/L/T (Hit@1)
D ⟶ D+R	*+0.004 [-0.076, +0.087]*	*0.864*	*+0.065 [-0.087, +0.217]*	*0.405*	*8/5/33*
D ⟶ G	*-0.342 [-0.531, -0.138]*	*0.006*	*-0.174 [-0.391, +0.065]*	*0.144*	*11/19/16*
D ⟶ H2	*+0.152 [+0.065, +0.261]*	*0.008*	*+0.109 [+0.022, +0.196]*	*0.025*	*5/0/41*
D ⟶ H4	*+0.141 [+0.054, +0.250]*	*0.011*	*+0.217 [+0.109, +0.348]*	*0.002*	*10/0/36*
D ⟶ H5	*+0.047 [-0.051, +0.152]*	*0.304*	*+0.109 [-0.043, +0.261]*	*0.197*	*10/5/31*
H4 ⟶ H5	*-0.094 [-0.214, +0.022]*	*0.159*	*-0.109 [-0.261, +0.043]*	*0.166*	*4/9/33*

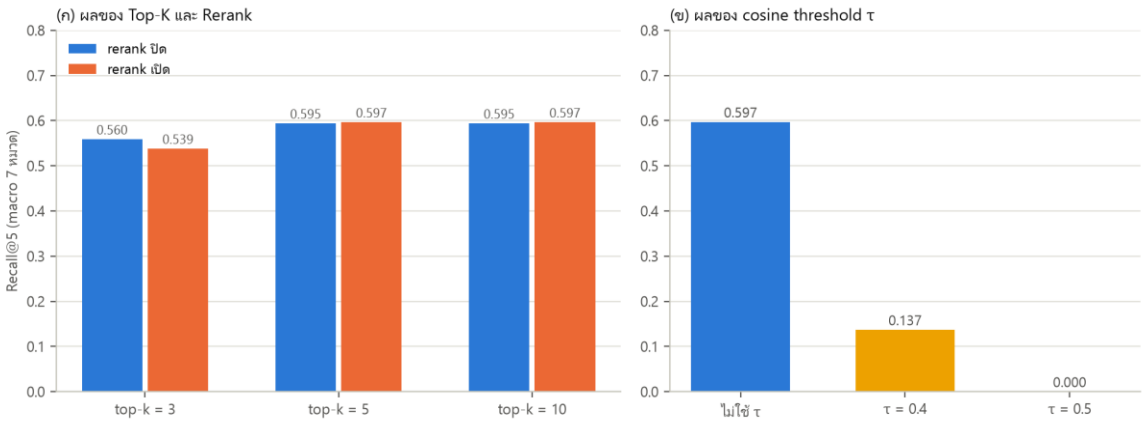
การพิจารณาค่าเฉลี่ยรวมเพียงอย่างเดียวอาจบดบังความแตกต่างระหว่างประเภทคำถาม โดย H5 เมื่อเทียบกับ Dense ไม่มีนัยสำคัญในภาพรวม เนื่องจากผลดีในกลุ่ม Lookup และ Procedure ถูกหักล้างด้วยผลด้อยในกลุ่ม

Multi-hop ขณะที่ H4 ชนะ Dense โดยไม่แพ้ในคำถามที่นำมาเปรียบเทียบ จึงควรรายงานผลแยกตามประเภทคำถามร่วมกับผลรวม

4.2.3 การปรับจูน Dense Retrieval

ตารางที่ 4.8 การปรับ Dense Retrieval (ค่าเฉลี่ยแบบ Macro ของ 7 ประเภทคำถามโดยนับ Out-of-scope เป็น 0 จึงต่ำกว่าตารางที่ 4.5 ที่เฉลี่ยรายข้อ)

Top-K	τ (Cosine)	Rerank	Recall@5	MRR	Hit@1
3	*ไม่ใช้*	*ปิด*	*0.559*	*0.536*	*0.437*
3	*ไม่ใช้*	*เปิด*	*0.539*	*0.570*	*0.498*
5	*ไม่ใช้*	*ปิด*	*0.595*	*0.539*	*0.437*
5	*ไม่ใช้*	*เปิด*	*0.597*	*0.582*	*0.498*
10	*ไม่ใช้*	*ปิด*	*0.595*	*0.549*	*0.437*
10	*ไม่ใช้*	*เปิด*	*0.597*	*0.590*	*0.498*
ทุกค่า	*0.4*	*ทุกค่า*	*0.137*	*0.155*	*0.148*
ทุกค่า	*0.5*	*ทุกค่า*	*0.000*	*0.000*	*0.000*



รูปที่ 4.3 ผลของ Top-K, Rerank และ cosine threshold ต่อ Recall@5 ของ Dense

ผลการปรับจูนมีข้อสังเกตดังนี้

- **Top-K:** *การเพิ่มจาก 3 เป็น 5 ช่วยเพิ่มประสิทธิภาพอย่างชัดเจน ขณะที่การเพิ่มจาก 5 เป็น 10 ให้ผลใกล้เคียงกัน โดยค่า Recall@5 เมื่อเปิด Rerank เท่ากับ 0.539, 0.597 และ 0.597 ตามลำดับ จึงเลือกใช้ k=5 เป็นค่าพื้นฐาน และใช้ dynamic_k ปรับเป็น 3, 5 หรือ 8 ตามความยาวของคำถาม*
- **Similarity Threshold:** *ค่า Threshold แบบ Absolute บน Cosine มีผลในทางลบ โดย* **τ***=0.4 ลด Recall@5 เหลือ 0.137 และ* **τ***=0.5 ตัดผลลัพธ์ของทุกคำถามจน Recall@5 เท่ากับ 0 เนื่องจากคะแนน Cosine ระหว่างภาษาพูดกับตัวบทกฎหมายไม่สูงพอ แม้เป็น Chunk เฉลย ระบบจึงใช้คะแนน Cross-encoder และค่า TAU_ANSWER กับ TAU_REJECT เป็นเกณฑ์ตัดสินแทน*
- **Rerank:** *ช่วยปรับปรุงลำดับผลลัพธ์ โดยเพิ่ม Hit@1 จาก 0.437 เป็น 0.498 และเพิ่ม MRR ประมาณ 0.04 แต่ไม่ได้เพิ่ม Recall@5 อย่างชัดเจน กล่าวคือ Rerank ช่วยจัดลำดับข้อมูลที่ค้นพบแล้วมากกว่าช่วยค้นหาข้อมูลใหม่*
- **Chunking:** *ผลการเปรียบเทียบแบบ Per-section กับ Fixed-512 จาก Offline Scorer ซึ่งเป็น Proxy แบบ Lexical พบว่า Per-section ให้ลำดับผลลัพธ์ดีกว่า โดยมี Recall@5=0.246, MRR=0.224 และ Hit@1=0.164 เทียบกับ Fixed-512 ที่ 0.228, 0.188 และ 0.130 ตามลำดับ อย่างไรก็ตาม ผลดังกล่าวเป็นหลักฐานประกอบ เนื่องจากไม่ได้ใช้ Embedding จริง การเลือก Per-section ยังเหมาะสมเพราะสอดคล้องกับหน่วยอ้างอิงของกฎหมายและช่วยให้ตรวจสอบ Citation ได้ง่าย*

4.2.4 ผลการประเมิน RAGEngine จริง: Dense, Graph และ Hybrid

การทดลอง Ablation สร้างแต่ละ Configuration จากส่วนประกอบย่อย จึงอาจไม่สะท้อนการทำงานของระบบจริงทั้งหมด ดังนั้นจึงมีการประเมินซ้ำด้วย RAGEngine จริงทั้ง 3 โหมด โดย Hybrid v2 คือระบบที่ปรับปรุงจาก

การวิเคราะห์ข้อผิดพลาดในหัวข้อ 4.4.4 ส่วน Hybrid v1 คือระบบก่อนการปรับปรุง ซึ่งใช้คะแนนจาก Cross-encoder เป็นหลักและเก็บไว้เพื่อใช้เป็นคู่เปรียบเทียบ ทั้งนี้โหมด Dense ของ Engine ใช้ Dense ร่วมกับ BM25 เพื่อให้เป็น Baseline ที่แข็งแรง ส่วน Dense ล้วนรายงานไว้ใน Config D ของตารางที่ 4.5

ตารางที่ 4.9 ผลการค้นคืนของ RAGEngine จริง 3 โหมด (พร้อมจำนวน chunk เฉลี่ยในบริบท)

โหมด / เวอร์ชัน	**Recall@5**	**MRR**	**nDCG@5**	**Hit@1**	**Context Recall**	**จำนวน Chunk เฉลี่ย**
Dense (Dense + BM25 ⟶ Rerank)	*0.625*	*0.623*	*0.603*	*0.565*	*0.663*	*4.5*
Graph	*0.835*	*0.810*	*0.787*	*0.739*	*0.891*	*6.6*
Hybrid v1 (Router + RRF + Expansion ⟶ Rerank ⟶ Safety Nets)	*0.748*	*0.696*	*0.677*	*0.587*	*0.917*	*6.5*
Hybrid v2 (Provenance-aware Rerank + Zone Gate/Alias ที่ปรับปรุงแล้ว)	*0.823*	*0.778*	*0.759*	*0.696*	*0.906*	*6.7*

หมายเหตุ: ผลการทดลองใช้คำถามในขอบเขตจำนวน 46 ข้อ ค่า Recall@5, MRR, nDCG@5 และ Hit@1 คำนวณจากผลลัพธ์ 5 อันดับแรก ส่วน Context Recall คือสัดส่วนของมาตราเฉลยที่ปรากฏอยู่ในบริบทจริงที่ส่งให้ LLM ซึ่งอาจมีจำนวนมากกว่า 5 chunks

ตารางที่ 4.10 การทดสอบนัยสำคัญแบบจับคู่ของ Engine จริง

การเปรียบเทียบ	**Δ Recall@5 [95% CI]**	**p**	**Δ Hit@1 [95% CI]**	**p**	**W/L/T (Hit@1)**
Dense ⟶ Hybrid v1	*+0.123 [+0.051, +0.210]*	*0.007*	*+0.022 [+0.000, +0.065]*	*0.317*	*1/0/45*

การเปรียบเทียบ	Δ Recall@5 [95% CI]	p	Δ Hit@1 [95% CI]	p	W/L/T (Hit@1)
Dense ⟶ Hybrid v2	*+0.197 [+0.105, +0.304]*	*0.001*	*+0.130 [+0.043, +0.239]*	*0.014*	*6/0/40*
Hybrid v1 ⟶ Hybrid v2	*+0.074 [+0.022, +0.139]*	*0.026*	*+0.109 [+0.022, +0.196]*	*0.025*	*5/0/41*

ตารางที่ 4.11 Hit@1 ของ Engine จริงแยกตามประเภทคำถาม

ประเภท	Dense	Graph	Hybrid v1	Hybrid v2
lookup	*0.88*	*1.00*	*1.00*	*1.00*
definition	*0.83*	*0.83*	*0.83*	*0.83*
single_hop	*0.70*	*0.80*	*0.70*	*0.80*
multi_hop	*0.20*	*0.40*	*0.20*	*0.30*
procedure	*0.50*	*0.75*	*0.50*	*0.62*
aggregation	*0.25*	*0.75*	*0.25*	*0.75*

ข้อสังเกตสำคัญมี 3 ประการ ได้แก่ (1) Hybrid v2 ชนะ Dense อย่างมีนัยสำคัญทั้ง Recall@5 และ Hit@1 (ตารางที่ 4.10) (2) การปรับปรุงจาก Hybrid v1 เป็น Hybrid v2 ช่วยเพิ่ม Hit@1 อย่างมีนัยสำคัญ โดยเพิ่มขึ้น 0.109 และมีค่า p=0.025 รวมถึงชนะ 5 ข้อและไม่แพ้ในคำถามที่นำมาเปรียบเทียบ และ (3) โหมด Graph ในการประเมินจริงไม่ใช่ Graph Retrieval เพียงอย่างเดียว เนื่องจากยังผ่าน Graph-seeded Expansion, Rerank และ Safety Nets เช่นเดียวกับกระบวนการประเมินของ Hybrid ดังนั้น ผลที่ Graph มีค่าสูงกว่า Hybrid v2 เล็กน้อยในชุดรวมควรตีความร่วมกับผลการวิเคราะห์การปนเปื้อนในหัวข้อถัดไป

4.2.5 ข้อควรระวังด้านการปนเปื้อนของข้อมูลทดสอบและการแยกชุด Held-out

ระหว่างการพัฒนา ทีมได้เพิ่ม Edge ABOUT ระหว่าง Topic และ Section โดยอ้างอิงจากกรณีที่ระบบค้นคืนผิดในรอบแรก เช่น คำถามเกี่ยวกับการลาป่วย การลากิจ การลาคลอด การร้องเรียนแรงงาน และการเลิกจ้าง เมื่อตรวจสอบย้อนหลังพบว่า Edge เหล่านี้ครอบคลุมมาตราเฉลยของ 16 จาก 23 ข้อในชุด A แต่ครอบคลุมเพียง 3 จาก 23 ข้อในชุด B

จึงมีความเป็นไปได้ว่าชุด A ได้รับประโยชน์จากการปรับปรุง Graph และอาจทำให้ผลประเมินสูงกว่าความสามารถในการใช้งานกับคำถามใหม่ การเปรียบเทียบที่เป็นธรรมกว่าจึงควรพิจารณาชุด B ซึ่งไม่ได้ใช้ทั้งการปรับ Graph และการจูนพารามิเตอร์ Rerank

ตารางที่ 4.12 ผลของ RAGEngine จริงแยกตามชุดคำถาม

โหมด	**A: Recall@5**	**A: Hit@1**	**B (Held-out): Recall@5**	**B (Held-out): Hit@1**	**รวม: Recall@5**	**รวม: Hit@1**
Dense	*0.413*	*0.391*	*0.837*	*0.739*	*0.625*	*0.565*
Graph	*0.790*	*0.696*	*0.880*	*0.783*	*0.835*	*0.739*
Hybrid v1	*0.638*	*0.435*	*0.859*	*0.739*	*0.748*	*0.587*
Hybrid v2	*0.732*	*0.565*	*0.913*	*0.826*	*0.823*	*0.696*

บนชุด B ซึ่งเป็น Held-out, Hybrid v2 ให้ผลสูงสุดทั้ง Recall@5 เท่ากับ 0.913 และ Hit@1 เท่ากับ 0.826 สูงกว่า Graph ที่ 0.880 และ 0.783 และ Dense ที่ 0.837 และ 0.739 ตามลำดับ ส่วนในชุด A ซึ่ง Graph ได้รับการปรับตามกรณีที่ระบบเคยค้นคืนผิด Graph ให้ผลสูงกว่า Hybrid

ดังนั้น ความแตกต่างระหว่าง Graph และ Hybrid ในชุดรวมจึงไม่ควรใช้เป็นหลักฐานว่า Graph ดีกว่าในทุกสถานการณ์ โดยสรุปอย่างระมัดระวังได้ว่า Hybrid ให้ผลดีกว่า Dense และให้ผลเทียบเท่าหรือดีกว่า Graph บน

ชุดที่ไม่ปนเปื้อน อย่างไรก็ตาม ชุด B มีขนาดเพียง 23 ข้อ ทำให้ความแตกต่างระหว่าง Hybrid และ Graph ยังไม่มีนัยสำคัญทางสถิติ และถือเป็นข้อจำกัดของการทดลองที่ควรกล่าวถึงในหัวข้อ 5.2

4.3 ผลการทดสอบเปรียบเทียบแบบจำลองภาษา (Local LLM vs API LLM Evaluation)

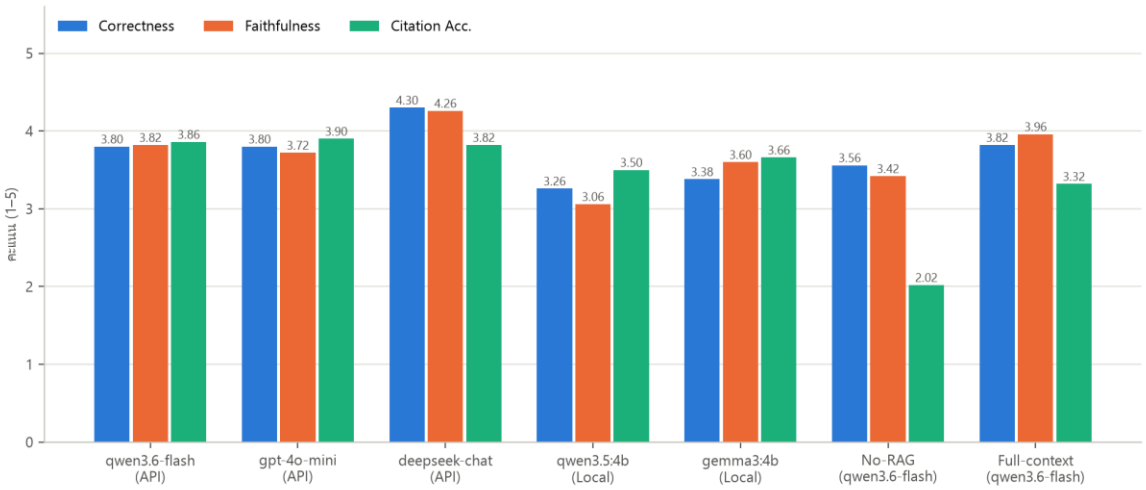
แบบจำลองภาษาทุกตัวได้รับการทดสอบบน Hybrid RAG v2 โดยใช้ Engine, Context และ Prompt เดียวกัน และเปลี่ยนเฉพาะแบบจำลองภาษา การทดลองใช้คำถามจำนวน 50 ข้อ และประเมินคุณภาพด้วย LLM-as-a-Judge รุ่น qwen/qwen3.6-plus เพียงรุ่นเดียว เนื่องจากแต่ละแบบจำลองถูกรันเพียง 1 ครั้ง ความแตกต่างประมาณ 0.2 คะแนนจึงควรตีความเป็นแนวโน้มและอาจมีความแปรปรวนจากการสุ่มของ LLM

4.3.1 คุณภาพคำตอบ

ตารางที่ 4.13 คุณภาพคำตอบของแบบจำลองภาษาแต่ละรุ่นบน Hybrid RAG

ระบบ	n	Correctness	Faithfulness	Citation Accuracy	Thai Clarity	Key-point	OOS Handled	Citation Precision	Citation Recall
qwen3.6-flash (API)	*50/50*	*3.80*	*3.82*	*3.86*	*4.80*	*0.78*	*1.00*	*0.75*	*0.85*
gpt-4o-mini (API)	*50/50*	*3.80*	*3.72*	*3.90*	*4.64*	*0.81*	*1.00*	*0.66*	*0.84*
deepseek-chat (API)	*50/50*	*4.30*	*4.26*	*3.82*	*4.94*	*0.82*	*1.00*	*0.63*	*0.88*
qwen3.5:4b (Local)	*50/50*	*3.26*	*3.06*	*3.50*	*4.50*	*0.70*	*1.00*	*0.67*	*0.79*
gemma3:4b (Local)	*50/50*	*3.38*	*3.60*	*3.66*	*3.86*	*0.66*	*1.00*	*0.63*	*0.79*

ระบบ	n	Correctness	Faithfulness	Citation Accuracy	Thai Clarity	Key-point	OOS Handled	Citation Precision	Citation Recall
No-RAG (qwen3.6-flash)	*50/50*	*3.56*	*3.42*	*2.02*	*4.76*	*0.76*	*0.25*	*0.17*	*0.17*
Full-context (qwen3.6-flash)	*50/50*	*3.82*	*3.96*	*3.32*	*4.80*	*0.82*	*0.00*	*0.47*	*0.50*



รูปที่ 4.4 คะแนนคุณภาพของแบบจำลองภาษาแต่ละรุ่นบน Hybrid RAG เทียบกับ Baseline

ตารางที่ 4.14 Correctness แยกตามประเภทคำถาม

ประเภท	qwen3.6-flash	gpt-4o-mini	deepseek-chat	qwen3.5:4b	gemma3:4b
lookup (n=8)	*4.12*	*4.50*	*4.62*	*4.38*	*3.38*

ประเภท	qwen3.6-flash	gpt-4o-mini	deepseek-chat	qwen3.5:4b	gemma3:4b
definition (n=6)	*4.33*	*4.00*	*4.33*	*3.67*	*3.67*
single_hop (n=10)	*4.00*	*3.50*	*4.20*	*2.50*	*3.60*
multi_hop (n=10)	*3.50*	*3.10*	*3.90*	*2.60*	*2.30*
procedure (n=8)	*3.00*	*3.88*	*4.38*	*3.38*	*3.50*
aggregation (n=4)	*3.25*	*4.00*	*4.50*	*3.25*	*3.75*
out_of_scope (n=4)	*4.75*	*4.25*	*4.50*	*3.75*	*4.50*

ผลการทดลองที่สำคัญมีดังนี้

1. *RAG ช่วยเพิ่มความน่าเชื่อถือของการอ้างอิง โดย No-RAG มี Citation Accuracy เท่ากับ 2.02 และ Citation Precision/Recall เท่ากับ 0.17/0.17 ขณะที่ระบบ RAG มีค่า 3.86 และ 0.75/0.85 ตามลำดับ แสดงว่า No-RAG อาจสรุปเนื้อหาโดยรวมได้ แต่มีความเสี่ยงต่อการอ้างเลขมาตราผิดหรือการคาดเดา นอกจากนี้ No-RAG ยังจัดการคำถามนอกขอบเขตได้ไม่สม่ำเสมอ โดยมีค่า OOS Handled เท่ากับ 0.25 เทียบกับ 1.00 ในระบบ RAG*
2. *Hybrid RAG ให้คุณภาพใกล้เคียง Full-context แต่ใช้ Token น้อยกว่าประมาณ 13 เท่า โดย Full-context มี Correctness เท่ากับ 3.82 ใกล้เคียง Hybrid ที่ 3.80 แต่ต้องส่ง Prompt ประมาณ 31,334 Token ต่อคำถาม เทียบกับ 2,442 Token ของ Hybrid และมี Citation Accuracy ต่ำกว่า คือ 3.32 เทียบกับ 3.86 เนื่องจาก LLM ต้องค้นหามาตราที่เกี่ยวข้องจากข้อความยาวด้วยตนเอง อีกทั้ง Full-context ไม่มีกลไกจัดการคำถามนอกขอบเขต จึงมีค่า OOS Handled เท่ากับ 0.00*
3. *API LLM ให้คะแนน Correctness สูงกว่า Local 4B ในการทดลอง โดย deepseek-chat ได้คะแนน 4.30, gpt-4o-mini และ qwen3.6-flash ได้ 3.80 ขณะที่ qwen3.5:4b และ gemma3:4b ได้ 3.26 และ 3.38 ตามลำดับ ช่องว่างเห็นได้ชัดในคำถามที่ต้องใช้บริบทหลายมาตรา โดยเฉพาะกลุ่ม Multi-hop และ Single-hop ซึ่ง qwen3.5:4b ได้คะแนน 2.60 และ 2.50 ตามลำดับ*

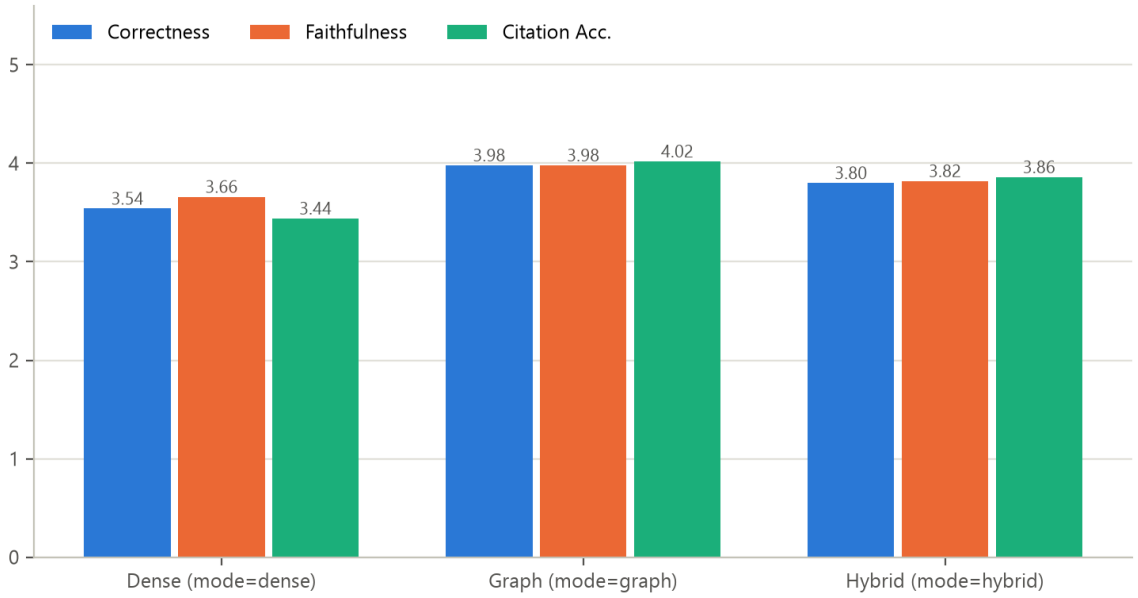
4. *ทุกแบบจำลองจัดการคำถามนอกขอบเขตได้ครบในระบบเวอร์ชันสุดท้าย โดยมีค่า OOS Handled เท่ากับ 1.00 เนื่องจากโค้ดเพิ่มป้ายเตือนว่าเป็นความรู้ทั่วไปของ AI ให้กับคำตอบโดยอัตโนมัติ และไม่ได้ให้ LLM สร้างป้ายเตือนดังกล่าวด้วยตนเอง*

ควรระวังการตีความอันดับระหว่าง qwen3.6-flash และ gpt-4o-mini เนื่องจากได้ Correctness เท่ากันที่ 3.80 และผู้ประเมิน qwen/qwen3.6-plus เป็นโมเดลในตระกูลเดียวกับ qwen3.6-flash ซึ่งอาจก่อให้เกิด Self-preference Bias อย่างไรก็ตาม deepseek-chat ซึ่งเป็นคนละตระกูลกลับได้คะแนนสูงสุด จึงไม่พบหลักฐานเพียงพอที่จะสรุปว่าผลทั้งหมดถูกชี้นำโดยอคติดังกล่าว ทั้งนี้ deepseek-chat มี Citation Precision ต่ำที่สุดในกลุ่ม API ที่ 0.63 สะท้อนแนวโน้มการอ้างมาตรามากกว่าที่จำเป็น

4.3.2 ผลของโหมด Retrieval ต่อคำตอบแบบ End-to-end

ตารางที่ 4.15 การเปรียบเทียบโหมด Retrieval บน LLM เดียวกัน

โหมด	n	Correctness	Faithfulness	Citation Accuracy	Thai Clarity	Key-point	Citation Precision	Citation Recall
Dense	*50/50*	*3.54*	*3.66*	*3.44*	*4.66*	*0.72*	*0.54*	*0.63*
Graph	*50/50*	*3.98*	*3.98*	*4.02*	*4.84*	*0.78*	*0.75*	*0.84*
Hybrid	*50/50*	*3.80*	*3.82*	*3.86*	*4.80*	*0.78*	*0.75*	*0.85*



รูปที่ 4.5 คุณภาพคำตอบเมื่อใช้ Retrieval แต่ละโหมด

ตัวชี้วัดที่คำนวณโดยตรงโดยไม่ใช้ LLM-as-a-Judge ให้ภาพที่ชัดเจนกว่า โดย Hybrid มี Citation Recall สูงกว่า Dense จาก 0.63 เป็น 0.85 หรือเพิ่มขึ้น 0.221 (Wilcoxon p=0.003; ชนะ 16 ข้อ แพ้ 2 ข้อจาก 46 ข้อ) และมี Citation Precision สูงกว่าจาก 0.54 เป็น 0.75 หรือเพิ่มขึ้น 0.214 (p=0.002)

*ขณะที่ Hybrid เทียบกับ Graph ไม่แตกต่างกันอย่างมีนัยสำคัญ (**Δ**Recall=+0.011, p=0.71; **Δ**Precision=-0.001, p=0.97) ในมุม LLM-as-a-Judge ลำดับคือ Graph 3.98 > Hybrid 3.80 > Dense 3.54 แต่ความแตกต่างระหว่างคู่ Dense–Hybrid และ Graph–Hybrid ยังไม่มีนัยสำคัญทางสถิติ (p=0.30 และ p=0.26 ตามลำดับ, n=46)*

จึงสรุปได้ว่า Graph และ Hybrid ให้คำตอบที่มีการอ้างอิงถูกต้องกว่า Dense ขณะที่ Hybrid ไม่แตกต่างจาก Graph อย่างมีนัยสำคัญในระดับคำตอบของชุดทดสอบนี้ ผลดังกล่าวควรพิจารณาร่วมกับข้อควรระวังเรื่อง Test-set Contamination ในหัวข้อ 4.2.5

4.3.3 Response Time และ Resource Usage

ตารางที่ 4.16 Latency และทรัพยากรจากการวัดแบบเครื่องว่าง (n=10 คำถาม; เวลาเป็นวินาที, Token/s เป็นค่ามัธยฐาน ,VRAM/RAM เป็น MiB)

LLM	n	Total มัธยฐาน	Total เฉลี่ย	Total สูงสุด	Retrieve	Rerank	Generate	Completion Token	Token/s	VRAM สูงสุด	RAM (RSS)
qwen3.6-flash (API)	*10*	*31.4*	*32.1*	*56.8*	*0.23*	*20.15*	*11.7*	*1,596*	*–*	*–*	*4,130*
gpt-4o-mini (API)	*10*	*23.7*	*22.9*	*46.0*	*0.21*	*20.11*	*2.6*	*77*	*–*	*–*	*4,078*
deepseek-chat (API)	*10*	*26.4*	*24.7*	*46.9*	*0.21*	*20.13*	*4.4*	*82*	*–*	*–*	*4,092*
qwen3.5:4b (Local)	*10*	*25.5*	*23.7*	*48.3*	*0.25*	*20.54*	*2.9*	*60*	*19.9*	*4,072*	*4,126*
gemma3:4b (Local)	*10*	*25.5*	*24.1*	*48.2*	*0.25*	*20.38*	*3.4*	*96*	*25.8*	*3,946*	*4,120*

ผลการวัดแสดงว่าคอขวดหลักอยู่ที่ Cross-encoder Rerank บน CPU ไม่ใช่ LLM โดยขั้น Retrieve ใช้เวลาประมาณ 0.2 วินาที ขณะที่ขั้น Rerank ใช้เวลาประมาณ 20 วินาที หรือประมาณร้อยละ 63–88 ของเวลารวม ส่วน Local LLM ใช้เวลา Generate ประมาณ 3 วินาที จึงทำให้เวลาโดยรวมของ Local LLM อยู่ที่ประมาณ 25 วินาที และใกล้เคียงกับ API LLM ที่ตอบสนองเร็ว เช่น gpt-4o-mini ซึ่งใช้เวลาประมาณ 24 วินาที

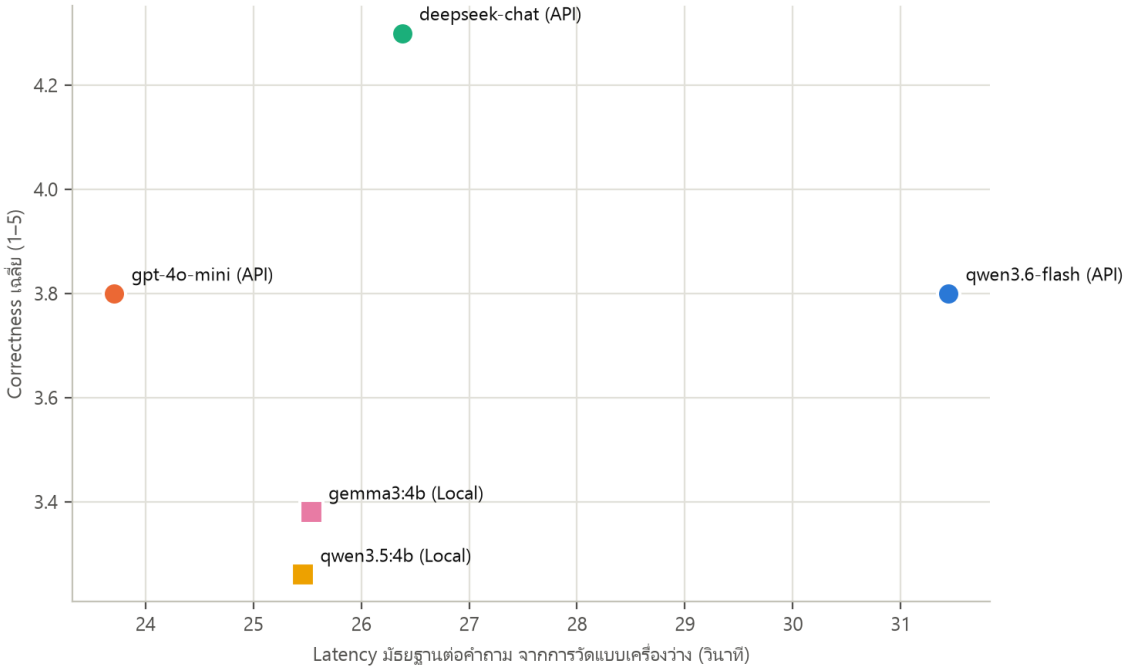
qwen3.6-flash ใช้เวลา Generate ประมาณ 11.7 วินาที และใช้ Completion Token เฉลี่ยประมาณ 1,600 Token ต่อคำถาม (การรัน 50 คำถามในตารางที่ 4.18 ได้ประมาณ 1,650 Token) เนื่องจากมีกระบวนการให้

เหตุผลภายในที่ไม่แสดงผล แต่ยังใช้เวลาและ Token ในการประมวลผล นอกจากนี้ยังพบการ Fallback 4 จาก 50 ข้อ หรือร้อยละ 8 ขณะที่ gpt-4o-mini และ deepseek-chat ไม่พบการ Fallback ในการทดลองนี้

Local LLM ใช้ VRAM สูงสุดประมาณ 3.9–4.1 GB จาก GPU ขนาด 6 GB และใช้ RAM ประมาณ 4.1 GB โดยค่า RAM นี้รวมการใช้ทรัพยากรของ Embedding, Reranker, ChromaDB, BM25 และ Graph ผลการทดลองปรับ num_ctx แสดงว่า Throughput เปลี่ยนแปลงไม่มาก จึงเลือกใช้ num_ctx=8,192 เพื่อรองรับคำถามที่ต้องใช้บริบทจำนวนมาก

ตารางที่ 4.17 ผลของ num_ctx ต่อ Local LLM

โมเดล	num_ctx	Latency เฉลี่ย (วินาที)	Token/s	VRAM สูงสุด (MiB)
qwen3.5:4b	*2,048*	*4.96*	*24.7*	*3,774*
qwen3.5:4b	*4,096*	*3.82*	*22.4*	*3,840*
qwen3.5:4b	*8,192*	*3.82*	*22.5*	*3,972*
gemma3:4b	*2,048*	*4.45*	*27.2*	*3,972*
gemma3:4b	*4,096*	*3.84*	*26.5*	*3,744*
gemma3:4b	*8,192*	*4.65*	*29.0*	*3,828*



รูปที่ 4.6 ความสัมพันธ์ระหว่างคุณภาพกับเวลาตอบของแบบจำลองภาษาแต่ละรุ่น

4.3.4 Token Usage และต้นทุน

การคำนวณใช้จำนวน Token ที่วัดได้จริงร่วมกับราคาอ้างอิงสาธารณะต่อ 1 ล้าน Token ของผู้ให้บริการ ณ ช่วงที่ทำการทดลอง ทั้งนี้ ราคาของผู้ให้บริการอาจเปลี่ยนแปลง จึงควรตรวจสอบราคาอีกครั้งก่อนนำไปใช้งานจริง

ตารางที่ 4.18 Token Usage และต้นทุนโดยประมาณต่อ 1,000 คำถาม

LLM	Prompt Token/ คำถาม	Completion Token/คำถาม	ต้นทุนเมื่อใช้ราคา gpt-4o-mini (USD)	ต้นทุนตามราคาโมเดล (USD)	ราคา Input/Output ($/1M)
qwen3.6-flash (API)	*2,442*	*1,650*	*1.356*	*ไม่มีราคาสาธารณะที่ตรวจสอบได้*	–
gpt-4o-mini (API)	*2,907*	*113*	*0.504*	*0.504*	*0.15/0.60*

LLM	Prompt Token/ คำถาม	Completion Token/คำถาม	ต้นทุนเมื่อใช้ราคา gpt-4o-mini (USD)	ต้นทุนตามราคาโมเดล (USD)	ราคา Input/Output ($/1M)
deepseek-chat (API)	*2,784*	*125*	*0.493*	*0.889*	*0.27/1.10*
qwen3.5:4b (Local)	*1,877*	*110*	*0*	*–*	*–*
gemma3:4b (Local)	*2,184*	*138*	*0*	*–*	*–*

- *Token ต่อคำถามวัดจากการรันคำถาม 50 ข้อ และรวมการเรียก LLM ทุกครั้ง ได้แก่ Rewrite, Normalize, Generate และ Fallback*
- *qwen3.6-flash ใช้ Completion Token สูงที่สุดประมาณ 1,650 Token ต่อคำถาม เมื่อประมาณด้วยราคา gpt-4o-mini ปริมาณ Token ที่มากทำให้ต้นทุนโดยประมาณสูงกว่า ส่วน gpt-4o-mini และ deepseek-chat ใช้ Completion Token ประมาณ 110–125 Token ต่อคำถาม*
- *Local LLM ไม่มีค่าบริการตามจำนวน Token แต่มีต้นทุนด้านฮาร์ดแวร์และพลังงาน และให้ Correctness ต่ำกว่า API LLM ในการทดลองนี้*
- *ในการทดลอง deepseek-chat ให้ Correctness สูงสุดที่ 4.30 และใช้ Completion Token ต่ำ จึงเป็นตัวเลือก API ที่ให้คุณภาพสูงสุดในชุดการทดลองนี้ อย่างไรก็ตาม การพิจารณาความคุ้มค่าควรคำนึงถึงราคาจริงของผู้ให้บริการร่วมด้วย ส่วน Local LLM เหมาะกับกรณีที่ต้องการประมวลผลข้อมูลภายในเครื่องหรือใช้เป็น Fallback โดยยอมรับข้อจำกัดด้านคุณภาพคำตอบ*

4.4 การวิเคราะห์และอภิปรายผลเชิงลึก (In-depth Discussion & Error Analysis)

4.4.1 เหตุใด Hybrid จึงชนะ Dense: Graph เติมสิ่งที่ Embedding มองไม่เห็น

เมื่อวิเคราะห์คำถามที่ H4 ให้ผลดีกว่า Dense พบว่าเป็นคำถามที่ Dense ไม่สามารถเชื่อมโยงคำถามกับมาตราเฉลยด้วยความคล้ายคลึงของข้อความ แต่ Graph สามารถเชื่อมโยงได้จากโครงสร้างความสัมพันธ์ โดยในกลุ่มที่ H4 มี Recall@5 ดีขึ้นพบ 7 ข้อ และไม่พบข้อที่ผลแย่ลง ลักษณะตัวอย่างแสดงในตารางที่ 4.19

ตารางที่ 4.19 ตัวอย่างข้อที่ Hybrid (H4) ชนะ Dense และกลไกที่ทำให้ชนะ

ข้อ	ประเภท	คำถาม	Gold	Recall@5: D ⟶ G ⟶ H4	กลไกที่ทำให้ Graph/Hybrid ได้เปรียบ
A-003	*lookup*	*มาตรา 61 กำหนดเรื่องใด*	*s61*	*0.0 ⟶ 1.0 ⟶ 1.0*	*Embedding แยกเลขมาตรา 61 ออกจากเลขมาตราอื่นได้ไม่ชัดเจน ขณะที่ Graph ใช้ Regex ของเลขมาตราเพื่อเชื่อมกับ Node Section โดยตรง*
B-003	*lookup*	*มาตรา 76 ว่าด้วยเรื่องอะไร*	*s76*	*0.0 ⟶ 1.0 ⟶ 1.0*	*Graph เชื่อมเลขมาตราเข้ากับ Section ได้โดยตรง*
A-009	*single_hop*	*ลูกจ้างมีสิทธิลากิจหรือไม่*	*s34*	*0.0 ⟶ 1.0 ⟶ 1.0*	*ตัวบทมาตรา 34 ไม่ใช้คำว่า “ลากิจ” โดยตรง และเอกสาร PDF มี Glyph Corruption แต่ Topic “ลากิจ” มี Alias และ Edge ABOUT ไปยังมาตรา 34*
A-008	*single_hop*	*ลูกจ้างลาป่วยได้กี่วัน*	*s32, s57*	*0.5 ⟶ 1.0 ⟶ 1.0*	*Topic “ลาป่วย” เชื่อมผ่าน ABOUT ไปยังมาตรา 32 และมาตรา 57 ได้ครบทั้งสองมาตรา*
A-018	*procedure*	*นายจ้างไม่จ่ายค่าล่วงเวลาต้องร้องเรียนที่ไหน*	*s123*	*0.0 ⟶ 1.0 ⟶ 1.0*	*มาตรา 123 ซึ่งเกี่ยวข้องกับการยื่นคำร้องเชื่อมผ่าน Topic “การร้องเรียนแรงงาน” ซึ่งมีข้อมูลหน่วยงาน หลักฐาน และขั้นตอน*

ข้อ	ประเภท	คำถาม	Gold	Recall@5: D ⟶ G ⟶ H4	กลไกที่ทำให้ Graph/Hybrid ได้เปรียบ
A-019	*procedure*	*ต้องใช้หลักฐานอะไรเมื่อร้องเรียนค่าจ้างค้างจ่าย*	*s123*	*0.0 ⟶ 1.0 ⟶ 1.0*	*ใช้ความสัมพันธ์เดียวกับ A-018 เพื่อเชื่อมคำถามกับมาตรา 123 และข้อมูลเชิงปฏิบัติ*
A-020	*procedure*	*ขั้นตอนยื่นคำร้องเรื่องเลิกจ้างทำอย่างไร*	*s123*	*0.0 ⟶ 1.0 ⟶ 1.0*	*Topic "การร้องเรียนแรงงาน" เชื่อมโยงไปยังมาตรา 123 และข้อมูลขั้นตอนการดำเนินการ*

กลไกสำคัญมี 3 ประการ ได้แก่ (1) Dense Retrieval มีข้อจำกัดกับ Identifier เช่น เลขมาตรา ซึ่ง BM25 และ Graph ช่วยจับคู่ได้โดยตรง (2) Dense Retrieval มีข้อจำกัดกับภาษาพูดที่ไม่ปรากฏในตัวบท ซึ่ง Topic Alias ใน Graph ทำหน้าที่เป็นสะพานเชื่อม และ (3) ข้อมูลเชิงปฏิบัติ เช่น หน่วยงาน หลักฐาน แบบฟอร์ม และขั้นตอน อยู่นอกเนื้อหามาตราโดยตรง จึงต้องอาศัยความสัมพันธ์ Topic⟶Agency/Evidence/Form/Step

ในทางกลับกัน Graph Retrieval เพียงอย่างเดียวแพ้ Dense ในประเภท Definition โดยมี Hit@1 เท่ากับ 0.17 เทียบกับ 0.83 และประเภท Single-hop เท่ากับ 0.20 เทียบกับ 0.70 เนื่องจาก Entity Linking ของ Graph อาศัย Regex เลขมาตราและ Alias ที่จัดทำไว้ล่วงหน้า โดยมี 30 Topics และเชื่อม ABOUT ได้ 11 Topics จึงยังครอบคลุมภาษาและคำถามได้จำกัด ผลดังกล่าวแสดงว่า Dense และ Graph เป็นวิธีที่เสริมกัน ไม่ได้ทดแทนกัน และเป็นเหตุผลที่การหลอมรวมด้วย RRF ให้ผลดีกว่าการใช้ Retriever ใดเพียงตัวเดียว

4.4.2 เหตุใด Cross-encoder Rerank จึงฉุด Hybrid

H4 ซึ่งไม่ใช้ Rerank มี Hit@1 เท่ากับ 0.717 สูงกว่า H5 ซึ่งใช้ Rerank ที่ 0.609 (n=46) โดยเฉพาะในคำถามประเภท Single-hop, Procedure และ Multi-hop ตัวอย่างที่ชัดเจนคือ A-018, A-019 และ A-020 ซึ่ง Graph สามารถค้นพบมาตรา 123 ได้ และ H4 มีค่า Recall@5 เท่ากับ 1.0 แต่หลังผ่าน Rerank มาตรา 123 หลุดจาก Top-5 ทั้งสามข้อ ทำให้ H5 มีค่าเท่ากับ 0.0 ในข้อเหล่านี้

สาเหตุเชิงเทคนิคคือ Cross-encoder BAAI/bge-reranker-v2-m3 ตัดสินจากความคล้ายเชิงข้อความระหว่างคำถามกับเนื้อหา Chunk เป็นหลัก แต่ไม่ทราบที่มาของ Candidate จาก Graph ซึ่งยืนยันความเกี่ยวข้องผ่าน Topic และความสัมพันธ์ในกราฟ

ตัวอย่าง A-018 ซึ่งถามว่า "นายจ้างไม่จ่ายค่าล่วงเวลาต้องร้องเรียนที่ไหน" พบว่ามาตรา 123 ได้คะแนน Cross-encoder เพียง 0.031 ต่ำกว่ามาตรา 72 ที่ 0.128 มาตรา 64 ที่ 0.095 และมาตรา 61 ที่ 0.069 เนื่องจากมาตราเหล่านั้นมีคำเกี่ยวกับค่าจ้างหรือค่าล่วงเวลาซ้ำกับคำถามมากกว่า ทั้งที่มาตรา 123 เป็นมาตราที่ตอบเรื่องการยื่นคำร้องโดยตรง นอกจากนี้ ตัวบทจาก PDF มีปัญหา Glyph Corruption เช่น "ฝาฝนหรอื" และ "สทิ ธไิ ด้" ซึ่งลดความคล้ายเชิงข้อความลงอีก

สำหรับ A-019 คะแนนของ Candidate ทุกมาตราต่ำมากอยู่ในช่วง 0.001–0.004 จน Cross-encoder แยกแยะได้ยาก ปัญหาเดียวกันเกิดกับคำถาม Multi-hop ซึ่งต้องเชื่อมมาตราเนื้อหากับมาตราบทลงโทษที่มีถ้อยคำแตกต่างกันมาก ทำให้ Rerank อาจลดอันดับของมาตราที่จำเป็นต่อคำตอบ

หลักฐานทางอ้อมที่สนับสนุนสาเหตุนี้คือ D+R เทียบกับ D ไม่ได้เพิ่ม Recall@5 อย่างมีนัยสำคัญ โดยเพิ่มเพียง 0.004 และมีค่า p=0.864 แสดงว่า Rerank ช่วยปรับลำดับมากกว่าช่วยค้นคืนข้อมูลใหม่ ขณะที่ H4 ซึ่งเพิ่มสัญญาณจาก Graph ทำให้ Recall เพิ่มขึ้น นอกจากนี้ การทดลองปรับน้ำหนัก Router ของ Route General ไม่ทำให้ผลเปลี่ยนแปลงแม้แต่ข้อเดียว เนื่องจาก Rerank จัดอันดับใหม่โดยไม่พิจารณาลำดับ RRF เดิม ซึ่งอภิปรายเพิ่มเติมในหัวข้อ 4.4.3

ระบบยังคงใช้ Rerank เนื่องจาก (1) คะแนน Rerank เป็นสัญญาณของ Zone Gate ที่ใช้ตัดสินว่าควรตอบจากตัวบทหรือไม่ และ (2) Rerank ช่วยเพิ่มประสิทธิภาพในคำถามประเภท Lookup โดย Dense มี Hit@1 เพิ่มจาก 0.38 เป็น 0.75 เมื่อใช้ D+R

อย่างไรก็ตาม ระบบเพิ่ม Safety Nets หลัง Rerank ได้แก่ complete_penalty_partners และ ensure_graph_hits_survive เพื่อรักษามาตราที่ Graph ยืนยันไว้ และแก้ที่ต้นเหตุเพิ่มเติมในระบบเวอร์ชันปรับปรุง (v2) ด้วย Provenance-aware Rerank ซึ่งให้โบนัสแก่ Candidate ที่ Graph ยืนยันและรวมอันดับจาก Weighted RRF ดังรายละเอียดในหัวข้อ 4.4.4

4.4.3 ผลลบที่มีคุณค่า: การปรับน้ำหนัก Router ไม่มีผล

คำถาม เช่น "ลูกจ้างมีสิทธิลากิจหรือไม่" ไม่ตรงกับ Keyword ของ Router จึงถูกจัดอยู่ใน Route General ซึ่งเดิมใช้น้ำหนัก Dense 0.6, BM25 0.3 และ Graph 0.1 ทีมจึงทดลองเพิ่มน้ำหนัก Graph เป็น Dense 0.35, BM25 0.25 และ Graph 0.40 แล้วรันซ้ำบนคำถาม 26 ข้อ

ผลคือ Hit@1 ไม่เปลี่ยนแปลงแม้แต่ข้อเดียว โดยไม่มีข้อที่ดีขึ้นหรือแย่ลง การทดลองนี้เป็นผลระหว่างการพัฒนาและไม่ได้รันซ้ำในรอบสุดท้าย

สาเหตุคือระบบดึง Candidate จากแต่ละ Retriever ในจำนวนคงที่โดยไม่ขึ้นกับน้ำหนัก Router ดังนั้นน้ำหนักจึงมีผลเฉพาะลำดับก่อนเข้าสู่ Rerank แต่ Rerank จัดอันดับใหม่จากเนื้อหาโดยไม่ใช้ลำดับเดิม Candidate

Pool จึงเหมือนเดิมและให้ผลลัพธ์เหมือนกัน

การค้นพบนี้ชี้ว่าต้นเหตุอยู่ที่ Rerank มากกว่าน้ำหนัก Router และนำไปสู่การเพิ่ม Safety Nets หลัง Rerank แทนการปรับน้ำหนักต่อไป

4.4.4 การปรับปรุงระบบจากผลการวิเคราะห์ (v1 ⟶ v2) และผลของการปรับปรุง

จากการวิเคราะห์ข้อผิดพลาดของระบบเวอร์ชันแรก (v1) ได้เลือกปรับปรุง 3 จุดที่ส่งผลต่อผู้ใช้มากที่สุด ได้แก่ Provenance-aware Rerank, Zone Gate และการจัดการ Alias จากนั้นวัดผลซ้ำด้วยกระบวนการเดิม โดยใช้คำถาม 50 ข้อ แบบจำลองภาษา 5 รุ่น และผู้ประเมินรุ่นเดียวกัน เพื่อให้สามารถเปรียบเทียบผลก่อนและหลังการปรับปรุงได้

(1) Provenance-aware Rerank แทนที่จะให้คะแนน Cross-encoder ตัดสินอันดับ Top-k เพียงอย่างเดียว ระบบใช้คะแนนรวมดังนี้

$$\text{final} = (1 - w) \cdot CE + w \cdot prior + b \cdot I_{\text{Graph}}$$

โดย CE คือคะแนน Cross-encoder, prior=1-rank/N คือตำแหน่งของ Candidate ในลำดับ Weighted RRF ซึ่งช่วยรักษาหลักฐานจาก Router และ Fusion ไว้ และ I_Graph เป็นตัวบ่งชี้ว่า Candidate ได้รับการยืนยันจาก Graph หรือไม่ โดยมีค่า 1 เมื่อ Graph ยืนยัน และมีค่า 0 ในกรณีอื่น

คะแนน CE ดิบสูงสุดยังคงถูกส่งให้ Zone Gate เพื่อไม่ให้ค่า TAU ที่ปรับไว้สูญเสียความหมาย เพื่อป้องกัน Overfitting จึงเลือกค่า (w,b) จากตาราง 6×6 โดยใช้เฉพาะชุด A จำนวน 23 ข้อ ตามกฎที่กำหนดไว้ล่วงหน้า คือ เลือกค่าที่ทำให้ Hit@1 และ Recall@5 สูงสุด และหากผลเท่ากันให้เลือกค่าพารามิเตอร์ที่ต่ำกว่า จากนั้นประเมินผลบนชุด B จำนวน 23 ข้อซึ่งไม่เคยใช้ในการจูน

ตารางที่ 4.20 ผลของ Provenance-aware Rerank (วัดด้วย eval/tune_rerank_blend.py ที่ replay candidate pool เดิม ค่า "ก่อน" จึงต่างจาก Hybrid v1 ในตารางที่ 4.9 เล็กน้อย)

ชุดข้อมูล	Hit@1 ก่อน	Hit@1 หลัง	Recall@5 ก่อน	Recall@5 หลัง
A (ชุดที่ใช้จูน, n=23)	*0.478*	*0.565*	*0.659*	*0.725*
B (Held-out, n=23)	*0.739*	*0.826*	*0.859*	*0.913*

ชุดข้อมูล	Hit@1 ก่อน	Hit@1 หลัง	Recall@5 ก่อน	Recall@5 หลัง
รวม (n=46)	*0.609*	*0.696*	*0.759*	*0.819*

ผลบนชุด Held-out ดีขึ้นในทิศทางเดียวกับชุดที่ใช้จูน และตารางการจูนมีช่วงค่าที่ให้ประสิทธิภาพใกล้เคียงกันหลายค่า โดยที่ w=0.3 Hit@1 เท่ากันทุกค่า b และ Recall@5 คงที่ตั้งแต่ b=0.3 ซึ่งเป็นค่าที่เลือกใช้ (w=0.3, b=0.3) สะท้อนว่าผลไม่ไวต่อการเลือกพารามิเตอร์มากนัก

อย่างไรก็ตาม Context Recall บนชุด A ลดลงเล็กน้อยจาก 0.855 เป็น 0.812 เนื่องจาก Cross-encoder เดิมสามารถดันมาตราที่เกี่ยวข้องบางส่วนเข้าสู่ Top-k ได้ ผลที่ระดับ Engine ในตารางที่ 4.10 ซึ่งรวมการแก้ Zone Gate และ Alias ทำให้ Hit@1 เพิ่มขึ้น 0.109 (p=0.025) ขณะที่ผลของ Provenance-aware Rerank เพียงอย่างเดียวเพิ่มขึ้น 0.087 ตามตารางที่ 4.20

(2) การปรับปรุง Zone Gate

ตรวจพบกรณีที่ Zone Gate ตัดสินไม่สอดคล้องกับมาตราเฉลยจำนวน 2 ข้อ ดังนี้

- **False out-of-scope:** *คำถาม "มาตรา 1 กำหนดเรื่องใด" ถูกส่งไปตอบด้วยความรู้ทั่วไป ทั้งที่ Graph ยืนยันมาตรา 1 และมาตราดังกล่าวอยู่ในผลลัพธ์ที่ค้นคืนแล้ว สาเหตุคือ ensure_graph_hits_survive() เดิมส่งสัญญาณ Graph ยืนยันเฉพาะกรณีที่มีการเพิ่มมาตราใหม่ จึงแก้ให้ฟังก์ชันนับทุกมาตราที่ Graph ยืนยัน ไม่ว่าจะมีอยู่ในผลลัพธ์เดิมหรือถูกเพิ่มเข้ามาใหม่*
- **False in-scope:** *คำถาม "ฟ้องหย่าต้องทำอย่างไร" ซึ่งอยู่นอกขอบเขตถูกตอบเป็นกฎหมายแรงงาน เนื่องจาก Alias "ฟ้อง" ของ Topic "การร้องเรียนแรงงาน" กว้างเกินไป จึงปรับ Alias ให้เฉพาะเจาะจงเป็น "ฟ้องนายจ้าง"*

(3) ผลรวมของการปรับปรุง

ข้อคำถามในขอบเขตที่ถูกส่งไปตอบด้วยความรู้ทั่วไปลดลงเหลือ 0 ในโหมด Hybrid และ Graph ทุกโมเดล ขณะที่โหมด Dense ยังคงมี 4 ข้อ เนื่องจากไม่มี Safety Net จาก Graph และค่า OOS Handled เพิ่มเป็น 1.00 ในทุกโมเดล จากเดิม 0.75 (โหมด Dense เดิมเป็น 1.00 อยู่แล้ว)

อย่างไรก็ตาม คะแนน Correctness และ Citation จาก LLM-as-a-Judge เปลี่ยนแปลงทั้งในทางบวกและลบ

ประมาณ ±0.2 ซึ่งอยู่ในระดับสัญญาณรบกวนของการประเมินเพียงรอบเดียว เช่น qwen3.6-flash ลดจาก 3.98 เป็น 3.80 ขณะที่ gemma3:4b เพิ่มจาก 3.16 เป็น 3.38 ดังนั้นไม่ควรตีความว่าการปรับปรุงช่วยเพิ่มคะแนน Judge อย่างชัดเจน หลักฐานหลักของการปรับปรุงจึงอยู่ที่ตัวชี้วัด Retrieval แบบกำหนดแน่นอนและความถูกต้องของ Zone Gate

ตารางที่ 4.21 เปรียบเทียบผลก่อนและหลังการปรับปรุง

ระบบ	**Correctness v1**	**Correctness v2**	**Citation v1**	**Citation v2**	**OOS Handled v1**	**OOS Handled v2**	**ข้อในขอบเขตที่ตอบเป็นความรู้ทั่วไป**	**Fallback v1 ⟶ v2**
qwen3.6-flash (API)	*3.98*	*3.80*	*4.06*	*3.86*	*0.75*	*1.00*	*1 ⟶ 0*	*6 ⟶ 4*
gpt-4o-mini (API)	*3.64*	*3.80*	*3.92*	*3.90*	*0.75*	*1.00*	*1 ⟶* **0**	*0 ⟶ 0*
deepseek-chat (API)	*4.18*	*4.30*	*4.02*	*3.82*	*0.75*	*1.00*	*1 ⟶ 0*	*2 ⟶ 0*
qwen3.5:4b (Local)	*3.20*	*3.26*	*3.58*	*3.50*	*0.75*	*1.00*	*1 ⟶ 0*	*0 ⟶ 0*
gemma3:4b (Local)	*3.16*	*3.38*	*3.32*	*3.66*	*0.75*	*1.00*	*1 ⟶ 0*	*0 ⟶ 0*
qwen3.6-flash + Dense	*3.50*	*3.54*	*3.48*	*3.44*	*1.00*	*1.00*	*4 ⟶ 4*	*6 ⟶ 4*

ระบบ	Correctness v1	Correctness v2	Citation v1	Citation v2	OOS Handled v1	OOS Handled v2	ข้อในขอบเขตที่ตอบเป็นความรู้ทั่วไป	Fallback v1 ⟶ v2
qwen3.6-flash + Graph	*3.88*	*3.98*	*3.78*	*4.02*	*0.75*	*1.00*	*0* ⟶ **0**	*3 ⟶ 4*

4.4.5 การวิเคราะห์ข้อผิดพลาดของระบบเวอร์ชันสุดท้าย (Failure Cases)

จากคำถาม In-scope จำนวน 46 ข้อ ระบบ Hybrid v2 ค้นพบมาตราเฉลยครบใน 5 อันดับแรกจำนวน 35 ข้อ ส่วนอีก 11 ข้อแบ่งได้เป็น 3 กลุ่มตามสาเหตุทางเทคนิค ดังแสดงในตารางที่ 4.22

ตารางที่ 4.22 กลุ่มข้อผิดพลาดของ Retrieval

กลุ่ม	ตัวอย่างข้อ	ลักษณะ	สาเหตุเชิงเทคนิค	แนวทางแก้
ก. พบมาตราแต่ลำดับต่ำ (4 ข้อ)	*A-009, A-019, B-014, B-015*	*มาตราเฉลยอยู่ใน Context แต่หลุดจาก 5 อันดับแรก*	*คำถามที่มีคำเชื่อมทำให้ dynamic_k ขยาย Context เป็น 8–14 Chunks และ Router จัดเป็น Route General ซึ่งมีน้ำหนัก Graph ต่ำ*	*ปรับน้ำหนัก Router ภายใต้ Provenance-aware Rerank*
ข. Multi-hop/Aggregation ได้ข้อมูลไม่ครบคู่ (5 ข้อ)	*A-013, A-015, A-017, A-022, B-013*	*ได้เพียงบางส่วนของคู่ “มาตราเนื้อหา + มาตราบทลงโทษ”*	*Expansion เริ่มจากเมล็ดที่ค้นพบเท่านั้น และมาตราบทลงโทษมีเนื้อหาไม่คล้ายคำถาม*	*เพิ่ม Expansion ย้อนกลับจากมาตราโทษไปยังมาตราเนื้อหา และใช้ Rerank ที่รู้โครงสร้างคู่*

กลุ่ม	ตัวอย่างข้อ	ลักษณะ	สาเหตุเชิงเทคนิค	แนวทางแก้
ค. ไม่พบมาตราเฉลย (2 ข้อ)	*A-016, A-021*	*Context ไม่มีมาตราเฉลยเลย*	*Graph ไม่มี Alias/Topic ที่จับคำถามได้ และ Dense ไม่สามารถเชื่อมภาษาพูดกับตัวบทได้*	*พัฒนา Semantic Entity Linking หรือ Text-to-Cypher*

Recall@5 อาจประเมินคุณภาพที่ผู้ใช้ได้รับต่ำกว่าความเป็นจริง เนื่องจาก LLM ได้รับบริบทมากกว่า 5 Chunks โดย Context Recall เฉลี่ยเท่ากับ 0.906 ขณะที่ Recall@5 เท่ากับ 0.823 และมี 39 จาก 46 ข้อที่มาตราเฉลยปรากฏครบในบริบท กลุ่มข้อผิดพลาดอีกส่วนหนึ่งเกิดจากข้อจำกัดของ Entity Linking แบบ Rule-based ซึ่งยังไม่สามารถเชื่อมคำถามที่ไม่มี Alias ตรงตัวกับ Topic ใน Graph ได้

4.4.6 ข้อผิดพลาดระดับ LLM เมื่อ Retrieval ถูกต้องแต่คำตอบผิดพลาด (Case Study and Baseline)

ในรอบพัฒนาแรก qwen3.6-flash ให้คำตอบว่างประมาณร้อยละ 34 แม้ Retrieval ถูกต้อง เนื่องจาก Hidden Reasoning ใช้ Completion Token ประมาณ 1,700–1,800 ก่อนเริ่มสร้างคำตอบ ขณะที่ NUM_PREDICT เดิมเท่ากับ 512 ทำให้การสร้างคำตอบถูกตัดจบก่อนมีเนื้อหา

การเพิ่มค่า NUM_PREDICT เป็น 2,500 ทำให้อัตราคำตอบว่างลดเหลือร้อยละ 6 และเพิ่ม Judge Correctness จาก 1.40 เป็น 2.82 ในรอบพัฒนาดังกล่าว ส่วนการวัดรอบสุดท้ายยังพบ Fallback 4 จาก 50 ข้อ หรือร้อยละ 8 ซึ่งระบบแก้ไขด้วยการสลับไปใช้ผู้ให้บริการรายอื่นโดยอัตโนมัติ

บทเรียนสำคัญคือควรประเมิน Retrieval และ Generation แยกกัน เพราะการวัดเฉพาะ Retrieval จะไม่ตรวจพบปัญหาที่เกิดในชั้น LLM

การทดสอบจริงบน LINE พบข้อผิดพลาดที่ชุดประเมินอัตโนมัติไม่พบ ได้แก่ (1) Zone Gate ส่งคำถามที่มีมาตราเฉลย เช่น “ต้องใช้หลักฐานอะไรเมื่อร้องเรียนค่าจ้างค้างจ่าย” ไปตอบด้วยความรู้ทั่วไป เพราะคะแนน Rerank ถูกคำนวณก่อน Safety Net เติมมาตรา 123 และ (2) LLM สร้างคำตอบซ้ำหลายบรรทัด

ระบบจึงเพิ่มการตรวจสอบ _is_degenerate_body และเรียกใช้ Fallback ไปยังผู้ให้บริการรายอื่นเมื่อพบคำตอบผิดปกติ

การเปรียบเทียบ No-RAG และ Full-context แสดงให้เห็นว่า RAG มีความจำเป็นต่อการอ้างอิง โดย No-RAG มี Citation Accuracy ต่ำที่สุดและไม่สามารถจัดการคำถามนอกขอบเขตได้อย่างสม่ำเสมอ ขณะที่ Full-context ให้ Correctness ใกล้เคียง Hybrid แต่ใช้ Prompt Token มากกว่าประมาณ 13 เท่าและมี Citation Accuracy ต่ำกว่า จึงไม่เหมาะกับการขยายระบบไปยังกฎหมายหลายฉบับ

4.4.7 สรุปการอภิปราย

1. *Hybrid RAG ให้ผลดีกว่า Dense อย่างมีนัยสำคัญทั้งด้าน Retrieval และการอ้างอิง โดย Ablation มี Hit@1 เพิ่มขึ้น 0.217 ($p=0.002$) และ Engine จริงเพิ่มขึ้น 0.130 ($p=0.014$) ขณะที่ Citation Recall เพิ่มขึ้น 0.221 และ Citation Precision เพิ่มขึ้น 0.214 โดยมีค่า $p \leq 0.003$*
2. *ประโยชน์ของ Hybrid เกิดจากการเสริมกันของ Retriever แต่ละแบบ Dense ช่วยจับความหมาย BM25 และ Graph ช่วยเรื่องเลขมาตราและภาษาพูด ส่วน Graph ยังเพิ่มข้อมูลเชิงขั้นตอน การนำรายการมาต่อกันโดยไม่ใช้ Fusion จึงไม่เพียงพอ โดย H1 ให้ผลเท่ากับ Dense*
3. *Cross-encoder Rerank ให้ผลได้ทั้งด้านบวกและด้านลบตามลักษณะข้อมูล เมื่อไม่รับรู้ที่มาของ Candidate อาจลดอันดับมาตราที่ Graph ยืนยัน การแก้ไขด้วย Provenance-aware Rerank ช่วยเพิ่ม Hit@1 บนชุด Held-out จาก 0.739 เป็น 0.826*
4. *ควรแยกชุด Held-out เนื่องจากการปรับ Graph หรือพารามิเตอร์จากคำถามชุดเดียวกับที่ใช้ประเมินอาจทำให้ผลลัพธ์สูงเกินจริง การรายงานผลแยกชุดช่วยให้เห็นประสิทธิภาพของระบบกับข้อมูลที่ไม่ได้ใช้พัฒนาอย่างชัดเจนขึ้น*
5. *ในการใช้งานจริง คอขวดด้านเวลาอยู่ที่ Cross-encoder บน CPU มากกว่า LLM จึงทำให้เวลาโดยรวมของ Local 4B ใกล้เคียงกับ API LLM การลดเวลา Rerank จึงเป็นแนวทางสำคัญกว่าการเปลี่ยนแบบจำลองภาษาเพียงอย่างเดียว*

บทที่ 5: สรุปผลและข้อเสนอแนะ (Conclusion & Future Work)

5.1 สรุปผลการดำเนินโครงงาน (Project Conclusion)

โครงงานนี้พัฒนา ThaiLaw Assistant ซึ่งเป็นระบบตอบคำถามกฎหมายแรงงานไทยบน LINE โดยใช้สถาปัตยกรรม Hybrid RAG ที่บูรณาการ Dense Retrieval ด้วย BAAI/bge-m3 และ ChromaDB, Lexical Retrieval ด้วย BM25 ภาษาไทย และ Graph Retrieval จากกราฟความรู้จำนวน 326 โหนดและ 720 เส้นเชื่อม (จัดเก็บบน Neo4j และ graph snapshot สำหรับ Retrieval)

ระบบประกอบด้วย Query Router, Weighted Reciprocal Rank Fusion, Graph-seeded Expansion, Provenance-aware Rerank และ Safety Nets ก่อนส่ง Section Card ให้แบบจำลองภาษา ทั้ง Local LLM ผ่าน Ollama และ API LLM ผ่าน dotBlue พร้อมระบบ Fallback และ Error Handling ที่ออกแบบจากปัญหาซึ่งพบในการทดลองและการทดสอบใช้งานจริง

สรุปตามวัตถุประสงค์

1. **ด้านการพัฒนาระบบ:** *ระบบมี Pipeline ครบตั้งแต่ User Request, Query Processing, Multi-Retrieval, Hybrid Fusion, Context Aggregation, LLM Execution จนถึง Response และผ่าน Unit Test จำนวน 133 กรณี นอกจากนี้ยังทดสอบการใช้งานจริงบน LINE ซึ่งช่วยตรวจพบข้อผิดพลาดที่ชุดประเมินอัตโนมัติไม่สามารถตรวจพบได้*
2. **ด้านการเปรียบเทียบ Retrieval:** *Hybrid ให้ผลดีกว่า Dense อย่างมีนัยสำคัญในหลายระดับ โดย Retrieval Ablation มี Hit@1 เพิ่มจาก 0.500 เป็น 0.717 ($p=0.002$) และ Engine จริงมี Recall@5 เพิ่มขึ้น 0.197 พร้อม Hit@1 เพิ่มขึ้น 0.130 ($p\leq0.014$) ในระดับคำตอบ Citation Recall เพิ่มจาก 0.63 เป็น 0.85 และ Citation Precision เพิ่มจาก 0.54 เป็น 0.75 ($p\leq0.003$)*

สำหรับชุด Held-out, Hybrid มี Hit@1 เท่ากับ 0.826 สูงกว่า Graph ที่ 0.783 และ Dense ที่ 0.739 แต่ความแตกต่างระหว่าง Hybrid กับ Graph ยังไม่มีนัยสำคัญ เนื่องจากชุด Held-out มีเพียง 23 ข้อ ผลการวิเคราะห์ยังแสดงให้เห็นว่า Graph ช่วยเติมข้อจำกัดของ Dense ในด้านเลขมาตรา ภาษาพูด และข้อมูลเชิงขั้นตอน ขณะที่ Rerank ที่ไม่รับรู้ที่มาของ Candidate อาจลดอันดับมาตราที่ Graph ยืนยัน จึงปรับปรุงด้วยการรวมหลักฐานจาก Fusion และ Graph เข้ากับ Cross-encoder

3. **ด้านการเปรียบเทียบ LLM:** *API LLM ให้ Correctness สูงกว่า Local 4B ในการทดลอง โดย deepseek-chat ได้คะแนนสูงสุด 4.30 เทียบกับ 3.26–3.38 ของ Local LLM Local LLM ใช้ VRAM ประมาณ 4.0 GB และ RAM ประมาณ 4.1 GB บน GPU ขนาด 6 GB และมีความเร็วประมาณ 20–26 Tokens/s*

ข้อค้นพบที่สำคัญคือ Latency รวมของ Local LLM ประมาณ 25 วินาที ใกล้เคียงกับ API LLM เนื่องจากคอขวดอยู่ที่ Cross-encoder Rerank บน CPU ซึ่งใช้เวลาประมาณ 20 วินาที ไม่ใช่ขั้นตอนการสร้างคำตอบของ LLM ส่วน qwen3.6-flash ใช้ Completion Token มากที่สุดประมาณ 1,650 Token ต่อคำถาม เนื่องจากมีกระบวนการให้เหตุผลภายในที่ไม่แสดงผล

ผลงานสำคัญเพิ่มเติม ได้แก่ (1) การวิเคราะห์ข้อผิดพลาดที่นำไปสู่การปรับปรุงระบบ 3 จุดและวัดผลซ้ำ (หัวข้อ 4.4.4) (2) การตรวจพบข้อจำกัดเรื่อง Test-set Contamination และการแยกชุด Held-out เพื่อให้การประเมินมีความเป็นธรรมมากขึ้น (หัวข้อ 4.2.5) (3) การรายงานผลลบที่มีคุณค่า เช่น Threshold แบบ Cosine ที่ไม่เหมาะกับโดเมนนี้ และการปรับน้ำหนัก Router ที่ไม่ส่งผลเมื่อ Rerank ไม่รับรู้ที่มาของ Candidate และ (4) การใช้ตัวชี้วัดแบบกำหนดแน่นอน ได้แก่ Retrieval, Citation Precision/Recall, Token และ Latency ควบคู่กับ LLM-as-a-Judge เพื่อไม่พึ่งพาผู้ประเมินประเภทเดียว

5.1.1 ความสอดคล้องระหว่างเกณฑ์ประเมินและหลักฐานในรายงาน

ตารางที่ 5.1 ความสอดคล้องระหว่างเกณฑ์ประเมินและหลักฐานในรายงาน

ด้านประเมิน	สิ่งที่เกณฑ์ระดับสูงต้องการ	หลักฐานในรายงาน
Data & Knowledge Base (10)	*Dataset และ Knowledge Base เป็นระบบ มี Cleaning, Chunking, Metadata และเตรียมทั้ง Vector กับ Graph พร้อมอธิบายเหตุผล*	*หัวข้อ 3.2, ตาราง 3.1–3.3, การ Parse 187/187 มาตรา, Continuity Check, 195 Chunks, Metadata และ Graph ที่ผ่านการตรวจสอบ*
Dense RAG (15)	*อธิบายกระบวนการ Embedding, Retrieval, Context และ LLM รวมถึงการ*	*หัวข้อ 3.3.1 และ 4.2.3, ตาราง 4.8, รูปที่ 4.3 และผล End-to-end ในตารางที่*

ด้านประเมิน	สิ่งที่เกณฑ์ระดับสูงต้องการ	หลักฐานในรายงาน
	ปรับ Top-K, Threshold และ Rerank พร้อมผลการทดลอง	*4.15*
Graph RAG (15)	*ออกแบบ Graph เหมาะสม มีความหมายของ Node/Relationship ใช้ Retrieval จริง และอธิบายการแก้ข้อจำกัดของ Dense*	*หัวข้อ 2.3, 3.2.4, 3.3.2 และ 4.4.1, ตารางที่ 4.19*
Hybrid RAG (20)	*มี Fusion, Ranking, Routing และ Aggregation ที่เหมาะสม และมีผลการทดลองสนับสนุน*	*หัวข้อ 3.3.3, Weighted RRF, Expansion, Safety Nets, Section Card, ตารางที่ 4.5, 4.7, 4.10 และ 4.12 รวมถึงการวิเคราะห์ในหัวข้อ 4.4.1–4.4.4*
Local LLM และ API LLM (15)	*เลือกแบบจำลองให้เหมาะกับ Hardware และวัดคุณภาพ Latency Resource Token Cost และ Error*	*หัวข้อ 3.4 และ 4.3 ครอบคลุมคุณภาพคำตอบ Latency, VRAM/RAM, num_ctx, Token, ต้นทุน และข้อผิดพลาดระดับ LLM ในหัวข้อ 4.4.6*
System Integration (10)	*ระบบทำงานเป็นระบบเดียว มี Error Handling และสถาปัตยกรรมชัดเจน*	*รูปที่ 3.1, ตารางที่ 3.7, LINE Webhook, Flex Message, Rich Menu, Fallback Chain, Observability และ Unit Test*
Evaluation & Analysis (10)	*ออกแบบการทดลองเป็นระบบ เปรียบเทียบ Dense, Graph, Hybrid และ Local/API พร้อมวิเคราะห์สาเหตุ*	*หัวข้อ 4.1–4.3, ชุดทดสอบ 50 ข้อ 7 ประเภท, Ground Truth, Metrics, Failure Cases และการทดสอบนัยสำคัญ*
Documentation / Presentation (5)	*จัดทำเอกสารครบตามโครงร่างที่กำหนด*	*รายงานครอบคลุมหัวข้อหลักของโครงร่าง รวมภาคผนวก A และภาคผนวก B*

5.2 ข้อจำกัดของระบบและปัญหาที่พบ (Limitations & Challenges)

ข้อจำกัดด้านข้อมูลและขอบเขต

1. *ระบบครอบคลุมกฎหมายเพียงฉบับเดียว คือพระราชบัญญัติคุ้มครองแรงงาน พ.ศ. 2541 คำถามเกี่ยวกับพระราชบัญญัติประกันสังคม พระราชบัญญัติเงินทดแทน หรือประมวลกฎหมายแพ่งและพาณิชย์ จะถูกตอบด้วยความรู้ทั่วไปของ AI พร้อมป้ายเตือน ไม่ใช่คำตอบจากตัวบทที่ผ่านการตรวจสอบในฐานความรู้ ข้อจำกัดนี้เกิดจากขอบเขตและระยะเวลาของโครงงาน*
2. *ตัวบท PDF ต้นฉบับมีปัญหา Glyph Corruption เช่น สระหรือวรรณยุกต์อยู่ผิดตำแหน่ง โดยพบอย่างน้อยร้อยละ 1.5 ของ Chunks ที่ตรวจสอบ และพบชัดเจนในมาตราที่เกี่ยวกับการลา ส่งผลให้การค้นคืนด้วย Keyword แบบตรงตัวพลาดบางกรณี ระบบใช้ Topic Alias เพื่อบรรเทาปัญหา แต่ยังไม่ได้แก้ไขที่แหล่งข้อมูลต้นฉบับ แนวทางแก้ไขคือเปลี่ยนไปใช้ตัวบท HTML จากสำนักงานคณะกรรมการกฤษฎีกา หรือแหล่งข้อมูลที่รักษาโครงสร้างภาษาไทยได้ถูกต้องกว่า*
3. *กราฟความรู้ยังไม่สมบูรณ์ โดย Edge ABOUT ระหว่าง Topic และ Section ได้รับการตรวจสอบกับตัวบทด้วยมือเพียง 11 จาก 30 Topics ความสัมพันธ์ PENALIZED_BY สร้างจาก Heuristic และยังไม่ได้วัด Precision อย่างเป็นระบบ นอกจากนี้มี Section Stub จำนวน 23 ตัวที่ไม่มี Property text/status จากการอ้างอิงล่วงหน้า และมีมาตรา 583 ที่เกิดจากการจับเลขมาตราของกฎหมายอื่นซึ่งถูกอ้างถึง อย่างไรก็ตามไม่กระทบการค้นคืนหลัก เนื่องจากเนื้อหามาตราดึงจาก sections.json โดยตรง Triple ที่สกัดด้วย LLM มี Precision ร้อยละ 50 หรือ 15 จาก 30 รายการ และยังไม่มี Node ประเภท Penalty*
4. *Entity Linking ของ Graph เป็นแบบ Rule-based ซึ่งใช้ Regex ของเลขมาตราและ Alias และยังไม่มี Semantic Fallback จึงอาจไม่สามารถจับคำถามที่ไม่มีคำหรือ Alias ตรงกับฐานความรู้ได้ เช่น A-016 และ A-021 ซึ่งทุก Configuration พลาดในลักษณะเดียวกัน แม้ความสัมพันธ์ PENALIZED_BY ที่เกี่ยวข้องจะมีอยู่ใน Graph แล้ว*

ข้อจำกัดด้านการทดลอง

1. ชุดทดสอบมี 50 ข้อ โดยเป็นคำถาม In-scope 46 ข้อ และแต่ละประเภทมีเพียง 4–10 ข้อ ทำให้ผลการวิเคราะห์รายประเภทมีความแปรปรวนสูง แม้รายงานช่วงความเชื่อมั่นและค่า p ประกอบแล้ว จึงควรตีความผลรายประเภทเป็นหลักฐานประกอบมากกว่าข้อสรุปทั่วไป
2. แบบจำลองภาษาแต่ละ Configuration รันเพียงหนึ่งรอบและไม่ได้ทำซ้ำหลาย Seed เนื่องจากข้อจำกัดด้านเวลาและเครดิต อีกทั้งผลของ LLM ไม่เป็น Deterministic ดังนั้นผล Generation ควรตีความเป็นแนวโน้ม และไม่ควรสรุปความแตกต่างเล็กน้อยว่าเป็นความแตกต่างถาวร
3. ผู้ประเมินเป็น LLM และยังไม่ได้ตรวจสอบเทียบกับผู้ประเมินที่เป็นมนุษย์ เช่น Cohen’s Kappa หรือ Spearman Correlation ผู้ประเมินอาจมีอคติต่อความยาว รูปแบบภาษา หรือโมเดลในตระกูลเดียวกัน

โดย qwen/qwen3.6-plus เป็นตระกูลเดียวกับ qwen3.6-flash จึงอาจเกิด Self-preference Bias คะแนนจึงควรใช้เปรียบเทียบสัมพัทธ์ระหว่าง Configuration มากกว่าตีความเป็นความถูกต้องสัมบูรณ์

4. Ablation Harness ไม่ได้จำลอง Production ครบทุกส่วน โดย H3 ไม่ได้ฉีด Edge ของ Graph ให้กับขั้น Expansion, H4 ใช้น้ำหนักคงที่ 0.4:0.6 แทน Router จริง และ H6 จำลอง Penalty-partner เพียงบางส่วน ดังนั้นผล Ablation ไม่ควรใช้แทนผลของระบบใช้งานจริงทั้งหมด จึงมีการประเมินซ้ำด้วย Engine จริงในตารางที่ 4.9–4.12
5. ยังไม่ได้วัดต้นทุนจากราคาจริงของ dotBlue ซึ่งเป็นเกตเวย์ของมหาวิทยาลัยและคิดค่าใช้บริการเป็นเครดิต โครงงานจึงรายงาน Token Usage ที่วัดได้จริงต่อคำถาม และประมาณต้นทุนด้วยราคาอ้างอิงสาธารณะ พร้อมระบุสมมติฐานไว้ในหัวข้อ 4.3.4
6. ค่า VRAM เป็นการใช้ GPU ทั้งเครื่อง โดยในสภาพแวดล้อมนี้ Embedding และ Reranker ทำงานบน CPU จึงเป็นการใช้ VRAM ของ Ollama เป็นหลัก ส่วนค่า RAM เป็นการใช้ของกระบวนการประเมินทั้งหมด ได้แก่ Embedding, Reranker, ChromaDB, BM25 และ Graph ไม่ใช่การใช้ของ LLM เพียงตัวเดียว นอกจากนี้ การวัด Latency จากการรัน 50 คำถามมีงานอื่นทำงานร่วมบางช่วง จึงใช้ผลการวัดแบบเครื่องว่าง 10 คำถามเป็นค่าประกอบหลัก
7. Graph มี Test-set Contamination เนื่องจาก Edge ABOUT บางส่วนถูกเพิ่มจากคำถาม*ที่ระบบพลาดในชุด A โดย Edge ที่ปรับปรุงครอบคลุมมาตราเฉลย 16 จาก 23 ข้อในชุด A เทียบกับ 3 จาก 23 ข้อในชุด B ผลของ Graph และ Hybrid บนชุดรวมจึงอาจสูงกว่าความสามารถกับคำถามใหม่ โครงงานจึงรายงานชุด B เป็น Held-out แยกต่างหาก แต่ชุด B มีเพียง 23 ข้อ ทำให้ส่วนต่างระหว่าง Hybrid กับ Graph ยังไม่มีนัยสำคัญทางสถิติ*
8. *คอขวดด้าน Latency ยังไม่ได้รับการแก้ไข โดย Rerank บน CPU ใช้เวลาประมาณ 20 วินาทีต่อคำถาม ทำให้เวลาในการตอบรวมประมาณ 25 วินาที แม้ Local LLM จะสร้างคำตอบภายในประมาณ 3 วินาที สาเหตุสำคัญคือ PyTorch ในสภาพแวดล้อมทดลองเป็นรุ่น CPU-only และยังไม่ได้ติดตั้งรุ่น CUDA เพื่อรักษาสภาพแวดล้อมให้คงที่ตลอดการทดลอง*

ปัญหาทางวิศวกรรมที่พบและแก้ไข

ปัญหาทางวิศวกรรมที่พบและแก้ไข ได้แก่ (1) Hidden Reasoning ของ qwen3.6-flash ใช้ Token จนคำตอบว่าง (2) Rerank ลดอันดับมาตราที่ Graph ยืนยัน (3) Zone Gate ตัดสินก่อน Safety Net (4) LLM สร้างคำตอบซ้ำ (5) JSON ของ Judge เสียหายจาก SSE Stream (6) Race Condition ของประวัติสนทนา (7) Regression ระหว่าง Branch ที่ทำให้ไฟล์จริงถูกเขียนทับด้วย Stub และ (8) ข้อจำกัดของการตัดคำภาษาไทยบน LINE รายละเอียดของแต่ละปัญหาและแนวทางแก้ไขแสดงในภาคผนวก A

5.3 ข้อเสนอแนะสำหรับการพัฒนาต่อยอด (Future Recommendations)

ตารางที่ 5.2 ข้อเสนอแนะสำหรับการพัฒนาระบบในอนาคต

ลำดับ	ข้อเสนอแนะ	เหตุผลหรือที่มาจากผลการทดลอง
1	*ต่อยอด Provenance-aware Rerank ที่ใช้แล้วใน v2 โดยเรียนรู้น้ำหนัก (w, b) จากข้อมูลที่มากขึ้น หรือ Rerank เฉพาะ Dense Candidate แล้วรวมกับ Graph Candidate ภายหลัง*	*H4 ให้ผลดีกว่า H5 เนื่องจาก Rerank เดิมลดอันดับมาตราที่ Graph ค้นพบ และ v2 จูนน้ำหนักจากชุด A เพียง 23 ข้อ*
2	*พัฒนา Semantic Entity Linking หรือ Text-to-Cypher เพื่อช่วยจับ Topic จากคำถามที่ไม่มี Alias ตรงตัว*	*พบจาก Failure Cases เช่น A-016 และ A-021 รวมถึงข้อจำกัดของการใช้ Graph Retrieval เพียงอย่างเดียว*
3	*ขยายและตรวจสอบ Graph โดยตรวจสอบ ABOUT ให้ครบทั้ง 30 Topics วัด Precision ของ PENALIZED_BY เพิ่ม Penalty Node และสกัด Triple เพิ่มโดยใช้ Prompt ที่กำหนดให้ระบุ Actor อย่างชัดเจน*	*สอดคล้องกับข้อจำกัดด้าน Knowledge Graph*
4	*เปลี่ยนแหล่งข้อมูลตัวบทเป็น HTML จากสำนักงานคณะกรรมการกฤษฎีกา*	*ช่วยลดปัญหา Glyph Corruption ในเอกสาร PDF*
5	*ขยายขอบเขตกฎหมายไปยังกฎหมายประกันสังคมและกฎหมายเงินทดแทน พร้อมเพิ่มความสัมพันธ์ AMENDED_BY และ Cross-law Reference*	*ขอบเขตปัจจุบันครอบคลุมกฎหมายเพียงฉบับเดียว*
6	*ตรวจสอบความสอดคล้องของ Judge กับผู้ประเมินที่เป็นมนุษย์ โดยใช้ Cohen's Kappa หรือ Spearman Correlation และใช้ผู้ประเมินต่างตระกูลกับโมเดลที่ถูกประเมิน*	*ลดความเสี่ยงจากอคติของ LLM-as-a-Judge*

ลำดับ	ข้อเสนอแนะ	เหตุผลหรือที่มาจากผลการทดลอง
7	*รันการทดลองซ้ำหลาย Seed และเพิ่มชุดทดสอบเป็นอย่างน้อย 200 ข้อ พร้อมคำถามแบบ Paraphrase*	*ลดความแปรปรวนและเพิ่มความน่าเชื่อถือของผลการทดลอง*
8	*Quantize หรือ Distill Local LLM และพิจารณาใช้โมเดลภาษาไทยเฉพาะทาง รวมถึง Cache ผลลัพธ์ตาม Hash ของ Prompt*	*ลด Latency และ Token Usage ของ Local LLM*
9	*ลด Latency โดยติดตั้ง PyTorch รุ่น CUDA และรัน Reranker แบบ FP16 บน GPU จำกัดจำนวน Candidate ที่เข้า Rerank และ Cache คะแนน Rerank*	*คอขวดหลักอยู่ที่ Rerank บน CPU ไม่ใช่ LLM*
10	*ปรับน้ำหนัก Router ภายใต้ Provenance-aware Rerank และเพิ่ม Expansion แบบย้อนกลับจากมาตราโทษไปยังมาตราเนื้อหา*	*ช่วยแก้ Failure Cases กลุ่มที่พบมาตราแต่ลำดับต่ำและกลุ่ม Multi-hop/Aggregation*
11	*เพิ่ม Monitoring ใน Production โดยใช้ data/traces.jsonl ทำ Dashboard สำหรับติดตาม Zone Distribution, Fallback Rate และ Empty-answer Rate พร้อมสุ่มตรวจคำตอบ*	*ช่วยตรวจพบข้อผิดพลาดจากการใช้งานจริงที่ชุดประเมินอัตโนมัติอาจไม่พบ*

บรรณานุกรม (References)

- *[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.*
- *[2] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," arXiv:2402.03216, 2024.*
- *[3] BAAI, "bge-reranker-v2-m3," Hugging Face model card, 2024. [Online]. Available: https://huggingface.co/BAAI/bge-reranker-v2-m3*
- *[4] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.*
- *[5] W. Phatthiyaphaibun, K. Chaovavanich, C. Polpanumas, A. Suriyawongkul, L. Lowphansirikul, and P. Chormai, "PyThaiNLP: Thai Natural Language Processing in Python," in Proc. 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS), 2023.*
- *[6] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," arXiv:2404.16130, 2024.*
- *[7] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in Proc. 32nd Int. ACM SIGIR Conf. on Research and Development in Information Retrieval, 2009, pp. 758–759.*
- *[8] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P.*

Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in NeurIPS Datasets and Benchmarks Track, 2023.

- *[9] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv:2312.10997, 2023.*
- *[10] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," arXiv:2309.15217, 2023.*
- *[11] Neo4j, Inc., "Neo4j Graph Database and Cypher Query Language Documentation (v5)." [Online]. Available: https://neo4j.com/docs/*
- *[12] Chroma, "Chroma: the open-source embedding database." [Online]. Available: https://docs.trychroma.com/*
- *[13] Ollama, "Ollama: Run large language models locally." [Online]. Available: https://ollama.com/*
- *[14] LINE Corporation, "LINE Messaging API Reference." [Online]. Available: https://developers.line.biz/en/reference/messaging-api/*
- *[15] พระราชบัญญัติคุ้มครองแรงงาน พ.ศ. 2541 (ฉบับปรับปรุงล่าสุด), ตัวบทกฎหมายที่ใช้เป็นฐานความรู้ของโครงงาน (เก็บใน data/raw/). [Online]. Available: https://legal.labour.go.th/images/law/Protection2541/2568_protectionpdf.pdf*

ภาคผนวก A: รายงานสรุปผลงานและการวิเคราะห์ส่วนบุคคล (Individual Technical Report & Reflection)

A-1 นักศึกษาคนที่ 1: Data / Knowledge Graph / Retrieval Evaluation

ชื่อ-นามสกุลนักศึกษา: อลงกรณ์ จงยิ่งยศ

รหัสนักศึกษา: 6610110693

โมดูลและหน้าที่รับผิดชอบหลัก: Data Pipeline, Knowledge Graph, Graph Retrieval, Retrieval Ablation และการวิเคราะห์ผลเชิงสถิติ

A-1.1 รายละเอียดการพัฒนาระบบย่อย (Individual Module Architecture and Implementation)

ตารางที่ A.1 ระบบย่อย ไฟล์ซอร์สโค้ด และหน้าที่รับผิดชอบ

ระบบย่อย	ไฟล์ซอร์สโค้ด	สิ่งที่พัฒนา
Ingest	src/ingest/scrape.py, pdf_extract.py, clean.py	ดึงตัวบท PDF และ FAQ จากกรมสวัสดิการและคุ้มครองแรงงาน รวมถึงล้างข้อความ เช่น เลขไทย Header/Footer Footnote และบล็อกท้ายเล่ม
Parse & Chunk	src/ingest/parse_sections.py, chunk.py	แยกโครงสร้างหมวดและมาตรา ได้ 187/187 มาตรา พร้อมตรวจสถานะยกเลิกหรือแก้ไข และแบ่งข้อมูลเป็น 195 Chunks พร้อม Metadata Schema
Knowledge Graph	src/index/graph_core.py, triples.py, review_triples.py	สร้างกราฟแบบ Deterministic ได้แก่ REFERS_TO, PENALIZED_BY และ DEFINES รวมข้อมูล Topic, Agency, Evidence, Form และ Step จาก Curated CSV ได้ 326 Nodes และ 720 Edges
Graph Retrieval	src/retrieval/graph.py	พัฒนา Entity Linking ด้วย Regex เลขมาตราและ Alias ไปยัง Topic ใช้แม่แบบ Cypher 5 ชนิด และพัฒนา Offline Traversal จาก graph.json รวมถึง Aggregation Fallback
Evaluation	eval/testset_a.jsonl, eval/run_retrieval.py, eval/analyze.ipynb	จัดทำชุดทดสอบ 25 ข้อร่วมกับนักศึกษาคนที่ 2 รวมเป็น 50 ข้อ ทำ Retrieval Ablation จำนวน 9 Configurations, Dense Tuning, Bootstrap CI, Wilcoxon Test และ Error Analysis

ระบบย่อย	ไฟล์ซอร์สโค้ด	สิ่งที่พัฒนา
Data Quality	tests/test_pipeline.py, tests/test_graph_retrieval.py	ตรวจสอบคุณภาพข้อมูล (สรุปในหัวข้อ 3.2) และพัฒนา Unit Test สำหรับตรวจสอบ Data Pipeline และ Knowledge Graph

อัลกอริทึมสำคัญ: Sequential-continuity Check

ตัวบทมีบรรทัดอ้างอิงที่ขึ้นต้นด้วย “มาตรา N” ซึ่งอาจทำให้ Regular Expression นับเป็นหัวมาตราใหม่ เช่น กรณี Wrapped Citation ที่ข้อความถูกตัดบรรทัดระหว่างการอ้างอิง ส่งผลให้ Parser นับได้ 209 มาตราแทน 187 มาตรา

ระบบจึงยอมรับหัวมาตราเมื่อเลขมาตราเรียงต่อจากมาตราก่อนหน้าตามลำดับของตัวบท ผลคือสามารถ Parse ได้ถูกต้อง 187/187 มาตรา

อัลกอริทึมสำคัญ: Graph Retrieval แบบ Offline

Graph Retrieval แบบ Offline ใน GraphRetriever._offline() ทำ Entity Linking แล้วเดินกราฟจากไฟล์ข้อมูลด้วยตรรกะเดียวกับ Cypher เช่น การเชื่อม Section กับ Topic ผ่านความสัมพันธ์ ABOUT ทำให้ระบบยังค้นคืนได้เมื่อไม่มี Neo4j ขณะ Runtime และช่วยให้การประเมิน Retrieval ทำซ้ำได้ในสภาพแวดล้อมเดียวกัน

A-1.2 การวิเคราะห์ปัญหาทางเทคนิคและข้อแลกเปลี่ยน (Technical Trade-off Analysis and Lessons Learned)

ตารางที่ A.2 ปัญหาทางเทคนิค แนวทางแก้ไข และข้อแลกเปลี่ยน

ปัญหาที่พบ	สาเหตุ	แนวทางแก้ไข	Trade-off และเหตุผลที่เลือก
Parse มาตราเกิน 209 แทน 187	Wrapped Citation ถูกมองเป็นหัวมาตราใหม่	ใช้ Sequential-continuity Check	มีความเสี่ยงหากกฎหมายมีเลขมาตราข้ามจริง แต่ตรวจสอบแล้วว่ากฎหมายฉบับนี้ไม่มีช่องว่าง จึงเรียบง่ายและเหมาะสมกว่าการพัฒนา Parser ด้วยการฝึกโมเดล

ปัญหาที่พบ	สาเหตุ	แนวทางแก้ไข	Trade-off และเหตุผลที่เลือก
DEFINES จับได้ 0 คำ	Regex เดิมบังคับให้มีคำว่า “คำว่า” นำหน้า แต่ตัวบทจริงใช้รูปแบบ “TERM หมายความว่า”	ทำ Prefix ให้เป็น Optional และได้คำนิยาม 23 รายการ	แสดงให้เห็นว่าควรตรวจสอบข้อมูลจริงก่อนตั้งสมมติฐานจากรูปแบบข้อความ
LLM สกัด BINDS/HELD_BY ผิดทั้งหมด	โมเดลขนาดเล็กกำหนด Subject Type ผิด เช่น จัด “นายจ้าง” เป็น Duty แม้ผ่าน Ontology Check	ไม่ใช้ Triple กลุ่มดังกล่าว และสร้าง Edge ใหม่จาก Subject ของ Triple ที่ผ่านการตรวจสอบ	ลดความครบถ้วนของการสกัดอัตโนมัติ แต่เพิ่มความถูกต้องของ Graph โดยแลกกับการต้องมีมนุษย์ตรวจสอบ
Graph Retrieval เพียงอย่างเดียวมี Recall@5 เพียง 0.16	ABOUT Edge มีเพียง 1 เส้น และ _offline() คืน Section แบบสุ่มแทนการเดินกราฟจริง	เพิ่มคอลัมน์ Sections ใน Curated CSV และแก้ _offline() ให้เดินตาม ABOUT จาก 1 เป็น 35 เส้น ทำให้ Recall@5 เพิ่มจาก 0.16 เป็น 0.31	ใช้เวลาตรวจสอบด้วยมือเพิ่มขึ้น แต่แก้ปัญหาที่ต้นเหตุ ทั้งนี้ยังครอบคลุมเพียง 11 จาก 30 Topics
คำถามรวบยอด เช่น “ลาประเภทใดบ้าง” ได้ 0/3	ไม่มี Keyword ของ Topic เฉพาะ และการจำกัด Top-k=5 ตัดผลลัพธ์ที่จำเป็นออก	ใช้ AGGREGATION_ROOTS และขยาย Top-k อย่างน้อย 20 เฉพาะคำถามประเภท Aggregation	เป็นการใช้ Rule เฉพาะกิจ เช่น Root “ลา” และ “วันหยุด” ขณะที่แนวทางทั่วไปคือ Text-to-Cypher ซึ่งอยู่นอกขอบเขตเวลา
ผล Ablation ไม่สะท้อนระบบจริง	ชุดทดลองเดิมมี graph_search() แยกจาก Production	ให้ RealGraphBackend เรียกใช้ GraphRetriever เดียวกับ Production	ได้ผลที่ตรงกับระบบจริงมากขึ้น แต่ต้องรันการทดลองใหม่ทั้งชุด
ผลรวมทางสถิติไม่แตกต่างอย่างมีนัยสำคัญ (p=0.30)	ผลดีในกลุ่ม Lookup และ Procedure ถูกหักล้างด้วยผลด้อยในกลุ่ม Multi-hop	วิเคราะห์ผลแยกตามประเภทคำถาม	เป็นบทเรียนว่าควรรายงานผลแยกตาม Category ไม่ควรพิจารณาเฉพาะค่าเฉลี่ยรวม

บทเรียนที่ได้รับ

ประการแรก ควรตรวจสอบผลลัพธ์จริงก่อนสรุปจากโค้ดเพียงอย่างเดียว เนื่องจากปัญหา DEFINES และ Offline Retrieval ไม่ปรากฏจาก Unit Test ที่ผ่านเพียงอย่างเดียว

ประการที่สอง การมีเทคโนโลยีหลายชนิดไม่ได้รับประกันคุณภาพ หากข้อมูลและความสัมพันธ์ไม่ครอบคลุม การใช้ Neo4j และ Graph ที่มี Edge ไม่เพียงพออาจให้ผลต่ำกว่าที่คาด จึงต้องวัดผลและปรับปรุงข้อมูลอย่างต่อเนื่อง

ประการที่สาม ข้อมูลที่สกัดด้วย LLM ขนาดเล็กควรผ่านการตรวจสอบโดยมนุษย์ เนื่องจากการทดลองพบ Precision เพียงร้อยละ 50 หากไม่ตรวจสอบ อาจทำให้ข้อมูลที่ผิดพลาดถูกนำไปปนเปื้อนใน Knowledge Graph.

A-2 นักศึกษาคนที่ 2: Retrieval Fusion / LLM Integration / LINE Application

ชื่อ-นามสกุลนักศึกษา: คีตศิลป์ คงสี

รหัสนักศึกษา: 6610110425

โมดูลและหน้าที่รับผิดชอบหลัก: Dense RAG, Hybrid Fusion Engine, LLM Integration (Local/API), LINE Application และ Generation Evaluation

A-2.1 รายละเอียดการพัฒนาระบบย่อย (Individual Module Architecture and Implementation)

ตารางที่ A.3 ระบบย่อย ไฟล์ซอร์สโค้ด และหน้าที่รับผิดชอบของนักศึกษาคนที่ 2

ระบบย่อย	ไฟล์ซอร์สโค้ด	สิ่งที่พัฒนา
Vector/BM25 Index	src/index/build_vector.py, build_bm25.py	สร้าง Embedding ด้วย BAAI/bge-m3 และจัดเก็บใน ChromaDB รวมถึงสร้าง BM25 แบบ Word-level ด้วย PyThaiNLP newmm และแบบ Character 3-gram โดยใช้ passage_text() ที่มีป้าย “มาตรา N”
Dense Retrieval	src/retrieval/retriever.py, reranker.py, thai.py	พัฒนา Dense Retrieval ร่วมกับ BM25, RRF, dynamic_k และ Cross-encoder Rerank บน CPU
Router / Fusion / Context	src/retrieval/router.py, fusion.py, context.py	พัฒนา Query Router, Weighted RRF 4 ทาง, Graph-seeded Expansion, Safety Nets และ Section Card

ระบบย่อย	ไฟล์ซอร์สโค้ด	สิ่งที่พัฒนา
LLM Client	src/llm/client.py	พัฒนา Interface เดียวสำหรับ Ollama และ dotBlue รองรับ SSE Streaming, Retry และ Token Accounting ตาม Thread
Engine	src/app/engine.py, generator.py, tracing.py	พัฒนา Pipeline ครบวงจร, Zone Gate, Fallback Chain, การตอบคำถามนอกขอบเขตด้วยความรู้ทั่วไปพร้อมป้ายเตือน และ Lock สำหรับประวัติสนทนา
LINE Application	src/app/app_line.py, flex.py, chat_history.py, setup_richmenu.py	พัฒนา Webhook และการตรวจสอบ Signature, คำสั่ง /mode, /llm, /debug, /reset, /intro, Flex Message, Rich Menu และการจัดเก็บประวัติบน Neo4j
Generation Evaluation	eval/testset_b.jsonl, eval/run_generation.py, eval/judge.py	จัดทำชุดทดสอบ 25 ข้อ รวมเป็น 50 ข้อ เปรียบเทียบ Local และ API LLM, ทดลอง num_ctx Sweep, LLM-as-a-Judge แบบ Shard/Merge/Retry และวัด Token/RAM/VRAM
Tests	tests/test_engine.py, test_fusion.py, test_context.py, test_generator.py, test_app_line.py และไฟล์ที่เกี่ยวข้อง	พัฒนาและตรวจสอบ Unit Test จำนวน 133 กรณี

อัลกอริทึมสำคัญ: Weighted RRF และ Query Router

ระบบคำนวณคะแนนของ Chunk ตามสมการ

$$\mathrm{RRF}(d) = \sum_{r} \frac{w_r}{60 + \mathrm{rank}_r(d)}$$

โดย w_r มาจากตารางน้ำหนักตามประเภทคำถามตามที่อธิบายในหัวข้อ 3.3.3 การใช้ลำดับแทนคะแนนดิบช่วยรวมผลลัพธ์จาก Cosine Similarity, BM25 และ Graph ซึ่งมีช่วงคะแนนแตกต่างกัน โดยไม่ต้องปรับสเกลหรือ Normalize คะแนนโดยตรง

Query Router ทำหน้าที่จำแนกประเภทคำถามและกำหนดน้ำหนักให้ Retriever แต่ละแบบ เพื่อให้ระบบตอบสนองต่อคำถามเลขมาตรา คำถามเชิงนิยาม คำถามหลายขั้นตอน และคำถามเชิงปฏิบัติได้เหมาะสมขึ้น

อัลกอริทึมสำคัญ: Safety Nets หลัง Rerank

Safety Nets หลัง Rerank ประกอบด้วย complete_penalty_partners และ ensure_graph_hits_survive ซึ่งเป็น Post-processing แบบ Append-only ที่ไม่แก้ไขคะแนนของ Cross-encoder โดยตรง ฟังก์ชัน complete_penalty_partners ช่วยดึงมาตราคู่บทลงโทษกลับเข้ามาเมื่อพบมาตราเนื้อหาที่เกี่ยวข้อง ส่วน ensure_graph_hits_survive ตรวจสอบว่ามาตราที่ Graph ยืนยันไม่ถูก Rerank ตัดออกจากผลลัพธ์ ทั้งสองกลไกช่วยรักษาหลักฐานเชิงโครงสร้างก่อนสร้าง Context และส่งต่อให้ LLM

A-2.2 การวิเคราะห์ปัญหาทางเทคนิคและข้อแลกเปลี่ยน (Technical Trade-off Analysis and Lessons Learned)

ตารางที่ A.4 ปัญหาทางเทคนิค แนวทางแก้ไข และข้อแลกเปลี่ยน

ปัญหาที่พบ	สาเหตุ	แนวทางแก้ไข	Trade-off และเหตุผลที่เลือก
qwen3.6-flash ตอบว่างร้อยละ 34 ของคำถาม	Hidden Reasoning ใช้ Completion Budget ประมาณ 1,700–1,800 Tokens ก่อนเริ่มตอบ และ NUM_PREDICT เดิมเท่ากับ 512 ทำให้คำตอบถูกตัดจบ	เพิ่ม NUM_PREDICT จาก 512 เป็น 2,500 ตรวจสอบ Body ว่าง และ Retry ไปยังผู้ให้บริการอื่น ทำให้คำตอบว่างลดเหลือร้อยละ 6 และ Judge Correctness เพิ่มจาก 1.40 เป็น 2.82	Latency และ Token เพิ่มขึ้นในโมเดลที่ใช้ Reasoning แต่โมเดลอื่นหยุดการสร้างคำตอบเองจึงได้รับผลกระทบน้อย
Rerank ลดผลลัพธ์ใน คำถาม Multi-hop โดย Hit@1 ลดจาก 0.40 เป็น 0.10	Cross-encoder พิจารณา Text Similarity เป็นหลัก และไม่ทราบว่ามาตราบทลงโทษมาจาก Graph จึงทำให้คู่มาตราเนื้อหาและมาตราโทษหลุดจาก Top-k	ใช้ complete_penalty_partners หลัง Rerank	เป็นการแก้ที่ปลายทาง ซึ่งทำได้ง่ายและมีความเสี่ยงต่ำกว่า ต่อมาจึงแก้ที่ตรงสาเหตุด้วย Provenance-aware Rerank ใน v2 (หัวข้อ 4.4.4)

ปัญหาที่พบ	สาเหตุ	แนวทางแก้ไข	Trade-off และเหตุผลที่เลือก
การปรับน้ำหนัก Router ของ Route General ไม่ส่งผล	Weight มีผลเฉพาะลำดับก่อน Rerank แต่ Rerank จัดอันดับใหม่จากข้อความ ทำให้ Candidate Pool เดิมให้ผลเหมือนเดิม	บันทึกเป็น Negative Result และใช้ ensure_graph_hits_survive แทน	ลดเวลาที่ใช้กับการปรับน้ำหนักซึ่งพิสูจน์แล้วว่าไม่ส่งผลภายใต้ Rerank แบบเดิม
Zone Gate ตัดคำตอบที่ถูกต้อง เช่น คำถามเกี่ยวกับหลักฐานร้องเรียนค่าจ้างค้างจ่าย	คะแนน Rerank ถูกคำนวณก่อน Safety Net เติมมาตรา 123	หาก Graph ยืนยันตรง ให้ปรับสถานะหรือคะแนนเพื่อให้ผ่าน TAU_ANSWER	อาจทำให้คำถามที่ไม่ควรตอบผ่านได้หาก Graph Link ผิด จึงลดความเสี่ยงด้วยการใช้ Exact Match เช่น เลขมาตราหรือ Alias ที่ยืนยันแล้ว
LLM สร้างคำตอบซ้ำ	เกิด Degenerate Generation จากแบบจำลองภาษา	ใช้ _is_degenerate_body ตรวจจับคำตอบผิดปกติและเรียก Retry	Heuristic ที่ตรวจบรรทัดซ้ำมากกว่าร้อยละ 30 อาจไม่ครอบคลุมรูปแบบความผิดพลาดทุกชนิด
Judge ได้ JSON เสียหายร้อยละ 36	SSE Stream ของ dotBlue ส่งข้อมูลไม่ครบหรือมีการสลับ Chunk	Retry ไม่เกิน 3 ครั้งต่อแถว, Checkpoint ทุก 10 แถว และประมวลผล 7 Processes แบบขนาน ทำให้เวลาลดจากประมาณ 4.4 ชั่วโมงเหลือประมาณ 40 นาที	เพิ่มจำนวนการเรียก API เล็กน้อย แต่เพิ่มความสมบูรณ์ของผลประเมิน
ข้อความภาษาไทยถูกตัดกลางคำบน LINE	LINE ไม่มี Thai Line-breaker โดยอัตโนมัติ	แทรก Zero-width Space ตามขอบคำด้วย PyThaiNLP	ขนาดข้อความเพิ่มขึ้นเล็กน้อย แต่ช่วยให้การแสดงผลภาษาไทยอ่านง่ายขึ้น
Race	Webhook แตก Thread	ใช้ threading.Lock และคืนสำเนาของ	ไม่ล็อกระหว่าง

ปัญหาที่พบ	สาเหตุ	แนวทางแก้ไข	Trade-off และเหตุผลที่เลือก
Condition ของประวัติสนทนา	ต่อข้อความและมีการอ่าน–เขียนประวัติพร้อมกัน	ข้อมูล โดยล็อกเฉพาะช่วง Copy/Append	Retrieve หรือ Generate จึงไม่เพิ่ม Latency ของกระบวนการหลัก
Regression ข้าม Branch โดยไม่แสดงข้อผิดพลาดอย่างชัดเจน	สคริปต์ Scaffold เขียนทับ chat_history.py และ scrape.py กลับเป็น Stub แล้วถูก Merge	กู้คืนจาก Git History และเพิ่ม Dependency ที่ขาด เช่น python-dotenv, requests และ bs4	เป็นบทเรียนให้เพิ่ม CI และ Test ที่ครอบคลุมการ Import และการทำงานของไฟล์จริง

บทเรียนที่ได้รับ

ประการแรก ควรวัด Retrieval และ Generation แยกกัน เพราะ LLM อาจให้คำตอบว่างแม้ Retrieval ถูกต้อง ตัวอย่างเช่น qwen3.6-flash มี Retrieval ถูกต้อง แต่เกิดปัญหาในชั้น LLM หากวัดเฉพาะ Retrieval จะไม่สามารถตรวจพบปัญหานี้ได้

ประการที่สอง การทดสอบ Live ช่วยค้นพบปัญหาที่ชุด Evaluation อัตโนมัติไม่พบ เนื่องจาก Test Set ใช้ประโยคคำถามที่กำหนดไว้ล่วงหน้า ขณะที่ผู้ใช้จริงอาจใช้คำถามแบบ Paraphrase ซึ่งทำให้คะแนน Rerank แตกต่างและอาจทำให้ Zone Gate ตัดสินผิด

ประการที่สาม ผลลบจากการทดลองมีคุณค่า การพิสูจน์ว่าน้ำหนัก Router ไม่มีผลภายใต้ Rerank แบบเดิมช่วยชี้ว่าปัญหาหลักอยู่ที่ Rerank มากกว่าการปรับน้ำหนัก Router

ประการที่สี่ การออกแบบระบบจริงต้องพิจารณาความถูกต้อง ความทนทานต่อข้อผิดพลาด และต้นทุนการประมวลผลควบคู่กัน โดยเฉพาะระบบที่ต้องทำงานผ่าน LINE และรองรับทั้ง Local LLM กับ API LLM.

ภาคผนวก B: คู่มือการติดตั้งและการใช้งานระบบ (Installation & User Manual)

B-1 ข้อกำหนดระบบและ Dependencies (System Requirements and Dependencies)

ตารางที่ B.1 ข้อกำหนดระบบและทรัพยากรที่ใช้ติดตั้ง

รายการ	**ข้อกำหนด**
ระบบปฏิบัติการ	Windows 10/11 โดยทดสอบบน Windows 11 ทั้งนี้ Linux และ macOS สามารถใช้งานได้ด้วยคำสั่งเทียบเท่า
Python	Python 3.12
Docker Desktop	ใช้สำหรับ Neo4j 5.24 Community และ APOC
Ollama	ใช้สำหรับ Local LLM ได้แก่ qwen3.5:4b และ gemma3:4b
cloudflared	ใช้สร้าง HTTPS Tunnel ให้ LINE เรียก Webhook โดยมี cloudflared.exe อยู่ใน Repository
ฮาร์ดแวร์ที่ใช้ทดสอบ	NVIDIA RTX 3050 Laptop ขนาด 6 GB VRAM และ RAM 16 GB โดย Embedding และ Reranker ประมวลผลบน CPU ส่วน Local LLM ผ่าน Ollama ใช้ VRAM ประมาณ 3.9–4.1 GB
บัญชีและคีย์	LINE Messaging API Channel และ dotBlue API Key ของมหาวิทยาลัย

ไลบรารี Python ที่ระบุใน requirements.txt ได้แก่ Flask, line-bot-sdk==3.21.0, neo4j, chromadb, sentence-transformers, rank_bm25, pythainlp, pdfplumber, requests, beautifulsoup4, lxml, pydantic, openai, ollama, tenacity, tiktoken, python-dotenv, psutil, nbconvert และ ipykernel

B-2 ขั้นตอนการติดตั้งสภาพแวดล้อมและการเรียกใช้ระบบ (Setup Environment and Run Application)

1) เตรียม Python Environment

ติดตั้ง Python 3.12, Docker Desktop, Ollama และ cloudflared ก่อนเริ่มต้น จากนั้นสร้าง Virtual Environment และติดตั้ง Dependencies

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

2) ตั้งค่าไฟล์ .env

คัดลอกไฟล์ตัวอย่างและสร้างไฟล์ .env

```
copy .env.example .env
```

กรอกค่าต่อไปนี้ในไฟล์ .env

```
LINE_CHANNEL_SECRET=
LINE_CHANNEL_ACCESS_TOKEN=
DOTBLUE_API_KEY=
NEO4J_PASSWORD=
```

ค่า NEO4J_PASSWORD ต้องตรงกับค่าที่ใช้ใน Docker Compose และห้ามเผยแพร่หรือ Commit ไฟล์ .env ลงใน Repository

3) ดาวน์โหลด Local LLM

```
ollama pull qwen3.5:4b
ollama pull gemma3:4b
```

4) เริ่มต้น Neo4j

```
docker compose up -d neo4j
```

ตรวจสอบสถานะให้เป็น Healthy จากนั้นเปิด Neo4j Browser ที่

```
http://localhost:7474
```

เข้าสู่ระบบด้วยผู้ใช้ neo4j และรหัสผ่านที่กำหนดไว้ในไฟล์ .env

5) สร้างฐานความรู้ใหม่ตั้งแต่ต้น

ขั้นตอนนี้เป็นทางเลือก หากใช้ข้อมูลที่ประมวลผลแล้วในโฟลเดอร์ data/ ได้แก่ chunks.jsonl, sections.json, graph.json, bm25.pkl และ index/chroma สามารถข้ามขั้นตอนนี้ได้

```
python -m src.ingest.scrape
python -m src.ingest.pdf_extract
```

```
python -m src.ingest.clean
python -m src.ingest.parse_sections
python -m src.ingest.chunk
python -m src.index.build_vector
python -m src.index.build_bm25
python -m src.index.graph_core data/chunks.jsonl --output data/graph.json --triples
data/triples.jsonl --review data/curated/triples_review_approved.csv --topics
data/curated/topic_agency_evidence_form_step.csv
```

6) รันระบบบน LINE

เปิด Terminal แรกสำหรับ Application

```
python -m src.app.app_line
```

เปิด Terminal ที่สองเพื่อสร้าง HTTPS Tunnel

```
.\cloudflared.exe tunnel --url http://localhost:5000
```

นำ URL ที่ได้ไปกำหนดเป็น Webhook URL ที่มี Path /callback ใน LINE Developers Console
หากต้องการสร้าง Rich Menu ให้ดำเนินการครั้งเดียวด้วยคำสั่ง

```
python -m src.app.setup_richmenu --upload
```

7) รัน Unit Test และการประเมินระบบ

ตรวจสอบ Unit Test จำนวน 133 กรณี

```
python -m unittest discover -s tests
```

รัน Retrieval Ablation และ Dense Tuning

```
python -m eval.run_retrieval --production --tune
```

รัน Generation Evaluation

```
python -m eval.run_generation --matrix --label H5_api_qwen-flash
```

ให้คะแนนด้วย Judge

```
python -m eval.judge --score --only-config H5_api_qwen-flash --output
eval/results/judge_scores_H5_api_qwen-flash.csv
python -m eval.judge --merge
```

สร้างผลวิเคราะห์จาก Notebook

```
jupyter nbconvert --to notebook --execute eval/analyze.ipynb
```

B-3 ตัวอย่าง User Interface

คำสั่งควบคุมผ่านแชต LINE

ตารางที่ B.2 คำสั่งควบคุมระบบผ่านแชต LINE

คำสั่ง	ความหมาย
/mode dense|graph|hybrid	สลับ Retrieval Mode โดยค่าเริ่มต้นคือ Hybrid
/llm local|api	สลับระหว่าง Local LLM ผ่าน Ollama และ API LLM ผ่าน dotBlue
/debug	เปิดหรือปิดโหมดแสดง Route, Top Sections, คะแนน, Graph Path, Latency และ Token
/reset	ล้างประวัติการสนทนา
/intro	แสดงการ์ดแนะนำระบบและตัวอย่างคำถาม ซึ่งเชื่อมกับ Rich Menu

ตัวอย่างการสนทนา

ตารางที่ B.3 ตัวอย่างคำถามและพฤติกรรมของระบบ

คำถามผู้ใช้	พฤติกรรมของระบบ
“ฝ่าฝืนมาตรา 61 มีโทษอย่างไร”	Router จัดเป็น Lookup เนื่องจากมีเลขมาตรา จากนั้นค้นคืนมาตรา 61 และมาตราบทลงโทษผ่าน PENALIZED_BY และตอบพร้อม Citation รวมถึง Flex Message ที่มีปุ่ม “อ่านตัวบทต้นฉบับ”
“ต้องใช้หลักฐานอะไรเมื่อร้องเรียนค่าจ้างค้างจ่าย”	Router จัดเป็น Procedure และ Graph เชื่อม Topic ไปยังหน่วยงาน หลักฐานแบบฟอร์ม และขั้นตอน พร้อมอ้างอิงมาตรา 123

คำถามผู้ใช้	พฤติกรรมของระบบ
“วันนี้อากาศเป็นอย่างไร”	หากคะแนน Rerank ต่ำกว่า TAU_REJECT ระบบตอบด้วยความรู้ทั่วไปของ AI พร้อมป้ายเตือน และไม่อ้างเลขมาตรา



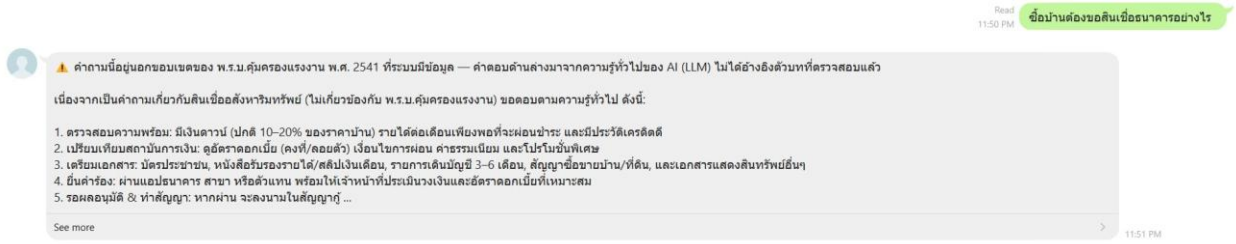
รูปที่ B.1 หน้าจอ Rich Menu หลักของระบบ ThaiLaw Assistant บน LINE



รูปที่ B.2 ตัวอย่างคำตอบด้านบทลงโทษ พร้อมการอ้างอิงมาตราและปุ่มอ่านตัวบทต้นฉบับ



รูปที่ B.3 ตัวอย่างคำตอบเกี่ยวกับเงื่อนไขการทำงานล่วงเวลา พร้อมรายการมาตราที่เกี่ยวข้อง



รูปที่ B.4 ตัวอย่างการแจ้งเตือนเมื่อผู้ใช้ถามคำถามนอกขอบเขตระบบ